SOFTWARE REQUIREMENT SPECIFICATION (SRS)

FPGA-Accelerated Arcade Gaming Platform

Version 1.0

Muhammad Ahmed Qazi (CS-24045)

Huzaifa Hanif (CS-24039)

Mujtaba Jawaid Rao (CS-24047)

Department of Computer and Information Systems Engineering
NED University of Engineering and Technology

Submitted to
Dr. Fakhra Aftab

October 19, 2026

Contents

1 Introduction **1**

1.1 Document Purpose . . . 1
1.2 Product Scope . . . 1
1.3 Intended Audience and Document Overview . . . 2
1.4 Definitions, Acronyms and Abbreviations . . . 2
1.5 Document Conventions . . . 5
1.6 References and Acknowledgments . . . 6

2 Overall Description **8**

2.1 Product Perspective . . . 8
2.2 Product Functionality . . . 8
2.2.1 User Requirements . . . 10
2.3 Users and Characteristics . . . 11
2.3.1 Stakeholders and Viewpoints . . . 11
2.3.2 User Classes . . . 12
2.4 Operating Environment . . . 12
2.5 Design and Implementation Constraints . . . 14
2.5.1 Design Decisions . . . 18
2.6 User Documentation . . . 18
2.7 Assumptions and Dependencies . . . 19

3 Specific Requirements **20**

3.1 External Interface Requirements . . . 20
3.1.1 User Interfaces . . . 20
3.1.2 Hardware Interfaces . . . 23
3.1.3 Software Interfaces . . . 26
3.1.4 Communications Interfaces . . . 27
3.2 System Functional Requirements . . . 29
3.2.1 SR-FW — Game framework, Arcade API and game loop . . . 29
3.2.2 SR-UI — Screens and navigation . . . 31
3.2.3 SR-GAM — Game rules . . . 32
3.2.4 SR-VGA — VGA timing and display output (VGA Controller) . . . 34
3.2.5 SR-INP — USB keyboard input (Input Manager) . . . 35
3.2.6 SR-AUD — Sound effects (Tone Generator) . . . 37
3.2.7 SR-HAL — Hardware Access Layer . . . 38

3.2.8 SR-REN — Tile Renderer . . . 40
3.3 Behaviour Requirements . . . 41
3.3.1 Use Case View . . . 41
3.3.2 State Machines and Interaction Models . . . 41

4 Other Non-functional Requirements 55
4.1 Performance Requirements . . . 55
4.2 Safety and Security Requirements . . . 56
4.2.1 Security . . . 57
4.3 Software Quality Attributes . . . 58
4.3.1 Maintainability . . . 58
4.3.2 Reusability (product line) . . . 59
4.3.3 Portability . . . 59
4.3.4 Testability . . . 59
4.3.5 Reliability . . . 59
4.3.6 Usability . . . 59

5 Other Requirements 60
5.1 Database Tables (Hardware) . . . 60
5.1.1 TBL-Tilemap . . . 60
5.1.2 TBL-TilePattern . . . 60
5.1.3 TBL-Palette . . . 61
5.1.4 TBL-RegisterMap . . . 61
5.2 Database Tables (Software) . . . 61
5.2.1 TBL-GameRegistry . . . 61
5.2.2 TBL-HighScore . . . 62
5.2.3 TBL-KeyBinding . . . 63
5.2.4 TBL-SoundEffect . . . 63
5.3 Future Scope . . . 64

A Data Dictionary 65
A.1 Constants . . . 65
A.2 State variables . . . 67
A.3 Inputs and outputs . . . 68

B Traceability Matrix 70

List of Figures

2.1 System context of the FPGA Arcade Platform . . . 9
2.2 Context diagram (DFD level 0) . . . 10
2.3 DFD level 1 (processes 1.0–6.0, data stores D1–D4) . . . 13
2.4 DFD level 2, process 4.0 Render Display (HPS and FPGA) . . . 15
2.5 Physical setup . . . 16

3.1 Screen 1 – Boot / splash . . . 23
3.2 Screen 2 – Main menu (reference screen) . . . 23
3.3 Screen 3 – Snake . . . 23
3.4 Screen 4 – Breakout . . . 23
3.5 Screen 5 – Paused . . . 24
3.6 Screen 6 – Game over . . . 24
3.7 Screen 7 – Initials entry . . . 24
3.8 Screen 8 – High scores . . . 24
3.9 Screen 9 – Keyboard disconnected . . . 25
3.10 Screen 10a – Shut down, confirm . . . 25
3.11 Screen 10b – Shut down, safe to switch off . . . 25
3.12 Use case diagram . . . 42
3.13 Application state machine, part 1: start-up, menu and shutdown . . . 44
3.14 Application state machine, part 2: gameplay . . . 45
3.15 Snake state machine . . . 46
3.16 Breakout state machine . . . 47
3.17 Keypress sequence: one key press (LEFT in Breakout) to the Game Engine . . . 48
3.18 VGA Controller timing state machine, part 1: horizontal . . . 49
3.19 VGA Controller timing state machine, part 2: vertical . . . 49
3.20 Input Manager state machine . . . 50
3.21 HAL lifecycle state machine . . . 51
3.22 Tile Renderer state machine . . . 51
3.23 Frame-update sequence, part 1: request . . . 52
3.24 Frame-update sequence, part 2: confirm (messages numbered from 12) . . . 53
3.25 Frame-update sequence, part 3: display . . . 54

5.1 Hardware tables entity-relationship diagram . . . 62
5.2 Software tables entity-relationship diagram . . . 62

List of Tables

1.1 Intended readers and suggested reading order . . . 2
1.2 Definitions, acronyms and abbreviations . . . 2
1.3 Requirement identifier prefixes . . . 5

2.1 User requirements . . . 10
2.2 Stakeholders and viewpoints . . . 11
2.3 User classes . . . 12
2.4 Minimum platform . . . 13
2.5 On-chip memory budget (CON-04) . . . 17
2.6 Design decisions . . . 18

3.1 Common screen layout . . . 20
3.2 Screen list and key mapping . . . 20
3.3 Register map of the lightweight-bridge window . . . 28
3.4 Use cases . . . 41
3.5 Behaviour models and their figures . . . 43

4.1 Keypress-to-screen latency budget (PR-02) . . . 55

5.1 TBL-Tilemap fields . . . 60
5.2 TBL-TilePattern fields . . . 60
5.3 TBL-Palette fields . . . 61
5.4 TBL-RegisterMap fields . . . 61
5.5 TBL-GameRegistry fields . . . 62
5.6 TBL-HighScore fields . . . 62
5.7 TBL-KeyBinding fields . . . 63
5.8 TBL-KeyBinding default bindings . . . 63
5.9 TBL-SoundEffect fields . . . 64
5.10 TBL-SoundEffect contents . . . 64

A.1 Constants . . . 65
A.2 State variables . . . 67
A.3 Inputs and outputs . . . 69

B.1 Traceability matrix: user requirements to derived requirements . . . 70

1 Introduction

The FPGA Arcade Platform is a self-contained arcade machine built on a single Terasic DE1-SoC board. Game logic runs as C++ software under embedded Linux on the board's ARM processor, while a custom graphics and sound subsystem in the board's FPGA fabric draws the picture on a VGA monitor and plays sound effects. This chapter states the purpose and scope of this document, who it is written for, the terms and conventions it uses, and the sources it draws on.

1.1 Document Purpose

This Software Requirements Specification (SRS) defines the requirements for the **FPGA-Accelerated Arcade Gaming Platform, release 1.0** (the *FPGA Arcade Platform* in the rest of this document), the CS-326 Software Engineering complex engineering problem of practical group G2. It is revision 1.0 of the SRS, issued on 19 October 2026.

The SRS covers the **whole system**: the C++ program on the Hard Processor System (game framework, Arcade API, two games, Input Manager and Hardware Access Layer) and the FPGA design (Tile Renderer, VGA Controller and Tone Generator), together with the interfaces between them. It is the agreed basis for design, implementation and testing: every requirement has an identifier, a parent user requirement and a verification method, so each one can be traced from stakeholder need to test.

1.2 Product Scope

The product is a dedicated arcade cabinet on one board. A player connects a VGA monitor, a USB keyboard and, optionally, speakers or headphones, powers the board on and reaches a menu of games without any PC. Release 1.0 provides two games, **Snake** and **Breakout**, a persistent top-10 high-score table for each game, simple sound effects, and a safe shutdown.

The design separates **meaning from pixels**. The processor owns all game state, rules, scoring and menus; the FPGA owns only the picture and the sound. The two halves meet at a small, frozen interface: a memory-mapped tilemap, a set of registers and the Arcade API. The benefits are:

- **Reuse (product line):** a new game is a new C++ class and one registry entry. The FPGA design, the HAL and the framework do not change (UR-16).
- **Testability:** the same games run on a PC through an SDL2 simulation backend, so game logic can be developed and unit-tested before the hardware is ready (UR-17).
- **Real-time quality:** the FPGA produces a stable 1024 × 768 picture at 60 Hz in hardware, with double-buffered tilemaps so the display never tears (UR-06, UR-13).

Out of scope for release 1.0: Space Invaders and any other sprite-based game (future scope, because the renderer is tile-only), multiplayer, networking and recorded audio.

Change from the approved proposal: the proposal named a PS/2 keyboard as the input device. On the DE1-SoC the PS/2 port connects only to the FPGA, while the two USB host ports connect to the processor. Release 1.0 therefore uses a **USB keyboard read through Linux**, which removes a custom FPGA input decoder and lowers project risk (decision of 13 Oct, CON-07).

1.3 Intended Audience and Document Overview

Table 1.1: Intended readers and suggested reading order

Reader	Uses this SRS to	Suggested reading order
Course instructor (client and assessor)	Check that scope, stakeholders, requirements, models and data design are complete and traceable	1 → 2 → 3.2 → 3.3 → 5 → Appendix B
Team developers (C++ and SystemVerilog)	Build each component to its requirements and interfaces	2.1 → 3.1 → 3.2 (own subsystem) → 3.3 → Appendix A
Testers	Derive a test for every requirement from its verification method	3.2 → 4 → Appendix B
Future game developers	Add a game without changing hardware	1.2 → 2.2 → 3.1.3 → SR-FW → 5
Lab staff	Set up, power and safely shut down the board	2.4 → 2.6 → 4.2

Organisation. Chapter 1 introduces the document. Chapter 2 describes the product as a whole: its context, functions and user requirements, stakeholders and users, environment, constraints and assumptions. Chapter 3 gives the specific requirements: external interfaces, system functional requirements grouped by subsystem, and the behavioural models (use cases, state machines and sequence diagrams). Chapter 4 gives the non-functional requirements: performance, safety and security, and quality attributes. Chapter 5 specifies the database tables. Appendix A is the data dictionary and Appendix B the traceability matrix.

1.4 Definitions, Acronyms and Abbreviations

Table 1.2: Definitions, acronyms and abbreviations

Term	Definition
Action	A game command such as Up, Select, Back, Pause, Launch or Mute. The Input Manager produces actions from key events using TBL-KeyBinding; games react to actions, not to raw keys.
Active region	The part of each line and frame in which pixels are visible: 1024 × 768 in the operating mode.
ADC	Analog-to-digital converter.
ADV7123	Analog Devices triple 10-bit video DAC on the DE1-SoC that converts digital RGB to analog VGA.
Arcade API	The C++ interface games use to draw, present frames, read input and play sounds (`setTile`, `fillRect`, `drawText`, `clear`, `present`, `pollInput`, `playSound`, `setMute`). It hides whether the FPGA or SDL2 backend is in use.
ARM	Processor architecture of the HPS cores (ARM Cortex-A9).
ASCII	American Standard Code for Information Interchange.
Atomic replace	Replacing a file so that readers see either the whole old file or the whole new one, never a partial file.
Auto-repeat	Repeated key-press events that the OS generates while a key is held.
AVDD	Analog supply voltage of the WM8731 codec; sets the full-scale level of its line output.
AXI	Advanced eXtensible Interface; the on-chip bus protocol the lightweight HPS-to-FPGA bridge is built on.
Back porch	Blank interval between the end of a sync pulse and the start of active video.
Backend	One implementation of the Arcade API. The FPGA backend drives the DE1-SoC hardware through the HAL; the SDL2 backend draws in a window on a PC. Chosen at start-up (SR-FW-05).

(continued)

Term	Definition
Bank swap	The act of exchanging which tilemap bank is “front” (displayed) and “back” (writable), performed by hardware only at the start of vertical blanking.
BCLK	Bit clock of the codec’s digital audio interface; one pulse per data bit.
Blanking	Intervals (front porch, sync, back porch) in which no picture is sent.
Bring-up mode	640×480 @ 60 Hz video mode used during development to test the video path.
Codec	Coder/decoder chip that converts between digital audio samples and analog sound (here the WM8731).
CSB	WM8731 control-interface pin that selects the codec’s I2C device address (0x1A when low).
CSV	Comma-separated values; the text format of the high-score file (decision D2).
D-SUB	15-pin connector used for analog VGA.
DAC	Digital-to-analog converter.
Data flow diagram (DFD)	Diagram showing how data moves between the system’s processes, stores and external entities (2.2).
DDR3	Double Data Rate 3 SDRAM; the HPS main memory.
Display enable	Control-register bit set by the HAL once tiles and palette are loaded; until then the VGA output stays black (SR-VGA-09, SR-HAL-13).
DMA	Direct Memory Access; not used in this system.
ER diagram	Entity-relationship diagram showing tables and their keys (Chapter 5).
EV_KEY	Linux input-event type for key and button changes.
evdev	Linux generic input-event interface; exposes each input device as `/dev/input/eventN`.
EVIOCGRAB	ioctl that gives one process exclusive access to an input device.
Flash (WCAG)	A pair of opposite changes in relative luminance of 10% or more, where the darker image is below 0.80 relative luminance.
FPGA manager	The Linux kernel subsystem that loads the FPGA bitstream and reports fabric configuration state (e.g. via `/sys/class/fpga_manager/fpga0/state`), checked by SR-HAL-09 before any bridge access.
Front porch	Blank interval between the end of active video and the start of a sync pulse.
fsync	Linux call that writes a file’s data to the storage device before returning.
Game Engine	The framework part of the C++ program that runs the game loop, owns the application state machine, and calls the active game through the game interface.
Game interface	The five operations every game implements: `init`, `update`, `render`, `isOver`, `score` (SR-FW-03).
Game tick	One iteration of the game loop; 60 ticks per second, locked to the display frame.
GPIO	General-Purpose Input/Output.
HAL	Hardware Access Layer; the software module that maps and accesses the lightweight bridge from Linux userspace.
HDMI	High-Definition Multimedia Interface; a digital video interface. The DE1-SoC has none.
HID	Human Interface Device; the USB device class for keyboards and mice.
HPS	Hard Processor System; the dual-core ARM Cortex-A9 part of the Cyclone V SoC.
HSYNC / VSYNC	Horizontal and vertical synchronisation pulses that tell the monitor where a line or frame starts.
HUD	Heads-up display; the score bar on rows 0–1 during play, showing the game name, score, lives and high score.
I2C	Two-wire serial bus used to write the codec’s control registers.
I^2S	Serial format for digital audio samples, with a bit clock and a left/right clock.
IEEE	Institute of Electrical and Electronics Engineers.
inotify	Linux mechanism that reports file-system changes, used to detect a keyboard being plugged in.
Input Manager	The C++ module that reads the USB keyboard through evdev and turns key events into actions (SR-INP).
IP	Intellectual-property core; a reusable, pre-designed hardware block supplied with the FPGA tools.

(continued)

Term	Definition
Key code	Linux number identifying a key (`KEY_*`, for example KEY_A).
Latency budget	Worst-case time of each stage of a path, added up to check a timing requirement.
Line-out	3.5 mm analog audio output at line level, for powered speakers or headphones.
LRCK (DACLRCK)	Left/right clock of the codec; one period per stereo sample frame (48 kHz here).
LW bridge	Lightweight HPS-to-FPGA AXI bridge; the memory-mapped interface used for all HPS↔FPGA communication in this system.
M10K	The Cyclone V FPGA's on-chip block RAM primitive, used to implement tilemap, tile pattern, palette and audio-related memory.
Main menu	The screen listing the registered games and SCORES, from which a game is started.
MCLK (AUD_XCK)	Codec master clock; 12.288 MHz for a 48 kHz sample rate.
MLAB	Memory logic array block; small distributed RAM in the Cyclone V FPGA.
mmap()	POSIX system call used to map a physical memory region (here, the LW bridge window) into a process's virtual address space.
MSEL	FPGA configuration-mode pins, set by switch SW10 on the DE1-SoC.
Mute	Setting that silences all sound; toggled with M.
Photosensitive epilepsy	Seizures triggered by flashing light or patterns.
PHY	Physical-layer transceiver chip (e.g. the USB PHY).
Pixel clock	Clock at which pixels are sent to the display: 65 MHz in the operating mode.
PK / FK	Primary key / foreign key of a table.
PLL	Phase-locked loop; FPGA block that generates the pixel clock from the 50 MHz board clock.
Product line	A family of products (here, games) built from one shared framework, API and hardware design. A new game reuses all of them unchanged (UR-16, QA-02).
Quadrant mosaic tile	One of 16 tiles that each draw a 2 × 2 block of font pixels. Combined, they display 4×-scale text with only 16 tiles of the 256-tile budget (SR-FW-12).
RGB	Red, green and blue colour components.
RS-343A	Analog video level standard (0 V black to 0.7 V white) used by VGA.
RTL	Register-Transfer Level; the hardware design abstraction written in SystemVerilog.
Sample rate	Number of audio samples per second per channel: 48 kHz here.
Scan-out	Sending one frame to the monitor, line by line from top to bottom.
SDL2	Simple DirectMedia Layer 2; a cross-platform C library for windows, graphics, keyboard and audio, used by the PC simulation backend.
SDRAM	Synchronous dynamic random-access memory.
Shadow tilemap	The software-side copy of the tilemap kept in HPS process RAM; all drawing calls write here, not to the bridge, until present() copies changes to the back bank.
Shutdown confirmation	The box opened by holding Esc for 3 s on the main menu; Enter confirms a safe shutdown, Esc cancels (UR-19, SR-UI-14).
SIGINT / SIGTERM	Linux signals that ask a process to stop: SIGINT from Ctrl+C on the console, SIGTERM from a termination request (SR-FW-14).
SoC	System on Chip; here the Cyclone V with HPS and FPGA on one die.
Square wave	Waveform that switches between two levels; gives a simple beep.
SRS	Software Requirements Specification; this document.
Stakeholder viewpoint	A group of stakeholders who share an interest in the system: interactor (player), indirect (instructor, lab staff, game developer) or domain (standards, platform).
Swap request	Register write by which the HAL asks the FPGA to exchange front and back tilemap banks at the next vertical blanking interval.
System requirement (SR)	A precise, testable statement of what the system shall do, derived from one UR (3.2).
TBL-GameRegistry	Compiled-in table listing every game (ID, name, menu order, factory); the menu is built from it (SR-FW-04).
TBL-HighScore	The top-10 scores per game, stored in the high-score CSV file on the microSD card (SR-FW-08 to SR-FW-10).
TBL-KeyBinding	Table mapping each (context, key code) pair to an action; `Global` entries apply on every screen unless overridden (SR-INP-09).

(continued)

Term	Definition
TBL-SoundEffect	Compiled-in table giving each sound effect's notes (frequency and duration); used by `playSound(effectId)` (SR-HAL-11).
Tile	8 × 8 pixel graphic; the screen is a 128 × 96 grid of tiles.
Tile pattern RAM	Writable on-chip memory holding up to 256 8×8 px, 4bpp tile graphics, loaded at start-up.
Tile Renderer	The FPGA-side module that reads tilemap, tile pattern and palette memory and produces the VGA pixel stream.
Tilemap bank	One of two 12,288-byte buffers holding the current 128×96 grid of tile indices; one is displayed while the other is written to.
Tone Generator	FPGA block that makes square-wave tones and sends them to the codec.
Traceability matrix	Table linking each user requirement (UR) to the system requirements (SR) derived from it.
TRM	Technical Reference Manual (Intel Cyclone V HPS).
uinput	Linux interface that lets a program create a virtual input device and inject key events, used for testing.
UIO	Userspace I/O; a Linux driver framework, a possible future replacement for `/dev/mem`.
ULPI	UTMI+ Low Pin Interface; links the HPS USB controller to the USB PHY.
UML	Unified Modeling Language.
USB	Universal Serial Bus.
Use case	A goal-directed interaction between an actor (e.g. the player) and the system (3.3).
User requirement (UR)	A natural-language statement of what a stakeholder needs from the system (2.2).
VESA DMT	VESA Display Monitor Timing standard, which defines XGA and other modes.
VGA	Video Graphics Array; analog RGB video interface with separate sync signals.
VGA Controller	The FPGA module that generates 1024×768 @ 60 Hz VGA timing (HSYNC, VSYNC, blanking) and the pixel coordinates for the Tile Renderer.
vsync	Vertical sync; the signal/moment marking the end of one scanned-out frame and the start of vertical blanking, the only safe point to swap tilemap banks.
W3C	World Wide Web Consortium; publisher of WCAG.
WCAG	Web Content Accessibility Guidelines (W3C); 2.3.1 limits flashing content.
WM8731	Wolfson 24-bit audio codec on the DE1-SoC board.
XGA	1024 × 768 display mode; used here at 60 Hz with a 65 MHz pixel clock.

1.5 Document Conventions

Formatting. The document follows the IEEE SRS layout of the course template: Computer Modern (the LaTeX default) text, 11 pt, single spacing and 1-inch margins. Code, file paths, register names and API calls are set in `monospace`. Diagrams follow UML 2 notation as used in the course labs (StarUML style).

Requirement identifiers.

Table 1.3: Requirement identifier prefixes

Prefix	Meaning
UR-nn	User requirement (stakeholder view, section 2.2)
SR-FW-nn, SR-UI-nn, SR-GAM-nn	System requirements: framework, screens and navigation, game rules
SR-INP-nn, SR-VGA-nn, SR-AUD-nn	System requirements: keyboard input, VGA output, sound
SR-HAL-nn, SR-REN-nn	System requirements: Hardware Access Layer, Tile Renderer
PR-nn, SF-nn, SC-nn, QA-nn	Performance, safety, security and quality-attribute requirements

(continued)

Prefix	Meaning
CON-nn, AS-nn	Design constraints, assumptions and dependencies
UC-nn, TBL-name	Use cases, database tables

Identifiers are never reused. A number that was retired during review (SR-REN-02) is left unused so that cross-references stay valid.

Requirement wording. *Shall* marks a mandatory requirement, *should* a desirable one (all sound requirements derived from UR-18), and *will* a statement of fact. Each system requirement is one testable statement with a single parent user requirement, a rationale and a verification method: Test, Inspection, Analysis or Demonstration. Quantities always carry units.

Component names. The same names are used throughout: Game Engine, Arcade API, HAL, Input Manager, Tilemap RAM, Tile Renderer, VGA Controller and Tone Generator.

1.6 References and Acknowledgments

[1] ISO/IEC/IEEE, *Systems and software engineering — Life cycle processes — Requirements engineering*, ISO/IEC/IEEE 29148:2018.

[2] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998.

[3] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.

[4] Terasic Inc., *DE1-SoC User Manual*, rev. E, 2015. [Online]. Available: https://people.ece.cornell.edu/land/courses/ece5760/DE1_SOC/DE1-SoC_User_manualv.1.2.2_revE.pdf

[5] Intel Corp., *Cyclone V Hard Processor System Technical Reference Manual*. [Online]. Available: https://www.intel.com/content/www/us/en/docs/programmable/683126/

[6] Intel Corp., *Cyclone V Device Overview*. [Online]. Available: https://www.intel.com/content/www/us/en/docs/programmable/683694/current/embedded-memory-capacity-in-cyclone.html

[7] Analog Devices, *ADV7123 CMOS 330 MHz Triple 10-Bit High Speed Video DAC*, data sheet, rev. D. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ADV7123.pdf

[8] Wolfson Microelectronics, *WM8731 Portable Internet Audio CODEC with Headphone Driver*, data sheet, rev. 3.4. [Online]. Available: https://people.ece.cornell.edu/land/courses/ece5760/DE1_SOC/Wolfson-WM8731-audio-CODEC.pdf

[9] VESA, *VESA and Industry Standards and Guidelines for Computer Display Monitor Timing (DMT)*, ver. 1.0, rev. 13, 2013.

[10] The Linux Kernel Documentation, “Input subsystem” and “Input event codes.” [Online]. Available: https://www.kernel.org/doc/html/latest/input/input.html

[11] The Linux Kernel Documentation, “FPGA Manager.” [Online]. Available: https://docs.kernel.org/driver-api/fpga/fpga-mgr.html

[12] Linux man-pages project, *mmap(2)*, *mem(4)*, *rename(2)*, *fsync(2)*, *inotify(7)*. [Online]. Available: https://man7.org/linux/man-pages/

[13] W3C, *Web Content Accessibility Guidelines (WCAG) 2.1*, “Understanding SC 2.3.1: Three Flashes or Below Threshold.” [Online]. Available: https://www.w3.org/WAI/WCAG21/Understanding/three-flashes-or-below-threshold.html

[14] SDL Project, *Simple DirectMedia Layer 2.0*. [Online]. Available: https://www.libsdl.org/

[15] Google, *GoogleTest User's Guide*. [Online]. Available: https://google.github.io/googletest/

[16] NED University of Engineering & Technology, *CS-326 Software Engineering: Complex Engineering Problem brief and rubric*, Fall 2026; and *Lab Session 05: Behavioral view UML diagrams*.

[17] M. A. Qazi, H. Hanif and M. J. Rao, *FPGA-Accelerated Arcade Gaming Platform: Project Proposal*, CS-326, NED University, 10 Sep 2026.

Acknowledgments. The team thanks Dr. Fakhra Aftab, course instructor, for approving the project and guiding the requirements process.

2 Overall Description

2.1 Product Perspective

The FPGA Arcade Platform is a new, self-contained product, not an extension or follow-on to an existing system. It combines a Terasic DE1-SoC board's embedded Linux (HPS) side with its FPGA fabric side to form a dedicated arcade cabinet running two games, Snake and Breakout, with no PC required at run time (DE1-SoC User Manual).

The division of responsibility follows the hardware split: the HPS owns *meaning* and the FPGA owns *pixels*. All game logic, rules, scoring, menus and input handling run as a C++ process on embedded Linux, which decides what the screen should show by writing tile indices into a shared tilemap. The FPGA fabric owns no game state at all; it only owns the Tile Renderer and VGA Controller, which read that tilemap and the tile pattern/palette RAM and turn them into a VGA signal, synchronised to the display's vertical refresh. This split keeps the FPGA design fixed and game-agnostic (see UR-16), while all gameplay changes happen entirely in software.

System context diagram: see Figure 2.1.

2.2 Product Functionality

The system performs the following major functions:

- **Start-up and hardware check:** boot from the microSD card, check that the FPGA design is loaded, and show the main menu within 60 s.
- **Game selection:** list the registered games and the high-score viewer in a menu with a live preview.
- **Gameplay:** run Snake and Breakout at a fixed 60 ticks per second, with pause, resume and quit.
- **Keyboard input:** read a USB keyboard, map keys to actions per screen, and survive the keyboard being unplugged.
- **Display:** draw every screen on a 128 × 96 tile grid and output 1024 × 768 VGA at 60 Hz without tearing.
- **Sound effects:** play short tones for game events, with mute (desirable).
- **High scores:** keep a top-10 table per game on the microSD card, entered with three initials, safe against power loss.
- **Safe shutdown:** save, show "SAFE TO SWITCH OFF" and halt Linux before power is removed.
- **Development support:** run the same games on a PC through an SDL2 backend, and add new games without changing the FPGA design.

The data flow diagrams below use Yourdon–DeMarco notation: circles are processes, squares are external entities, open-ended boxes are data stores, and labelled arrows are data flows.

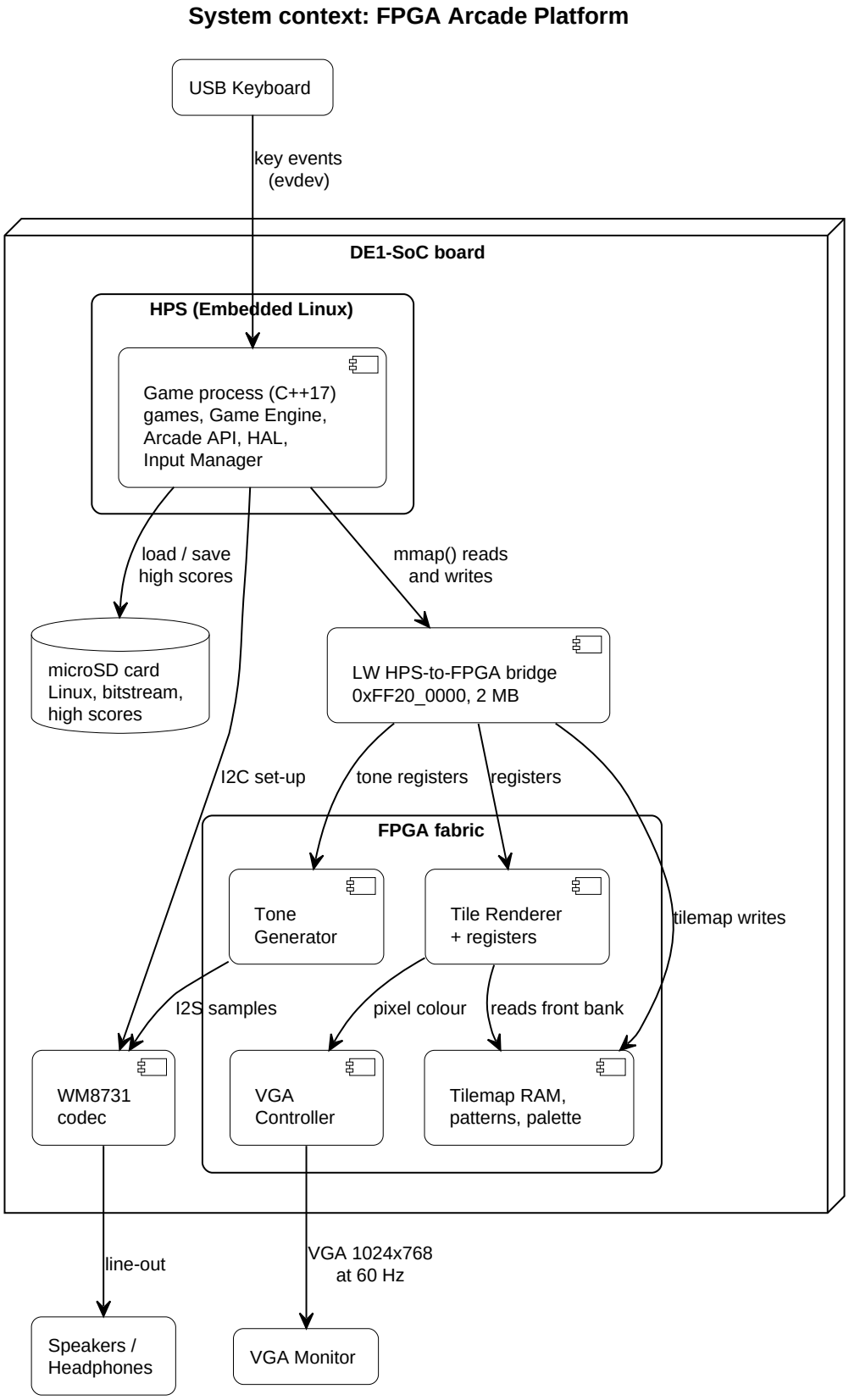


Figure 2.1: System context of the FPGA Arcade Platform

- **Context diagram (DFD level 0, Figure 2.2):** the whole system as one process, exchanging data with the Player, Lab Staff and Game Developer. The USB keyboard, VGA monitor and speakers are the physical media for the Player's flows (section 2.4).
- **DFD level 1 (Figure 2.3):** the system broken into six processes: 1.0 Read Input, 2.0 Run Game Logic, 3.0 Manage High Scores, 4.0 Render Display, 5.0 Generate Sound and 6.0 Control Lifecycle. It also shows four data stores: D1 high-score file, D2 tilemap banks, D3 tile patterns and palette, and D4 key bindings, game registry and sound effects.
- **DFD level 2 for process 4.0 (Figure 2.4):** rendering split across HPS software (4.1 update the shadow tilemap, 4.2 copy cells and request a swap) and FPGA hardware (4.3 swap banks at vblank, 4.4 fetch tile and colour, 4.5 generate VGA timing). This is where data crosses from software into hardware.

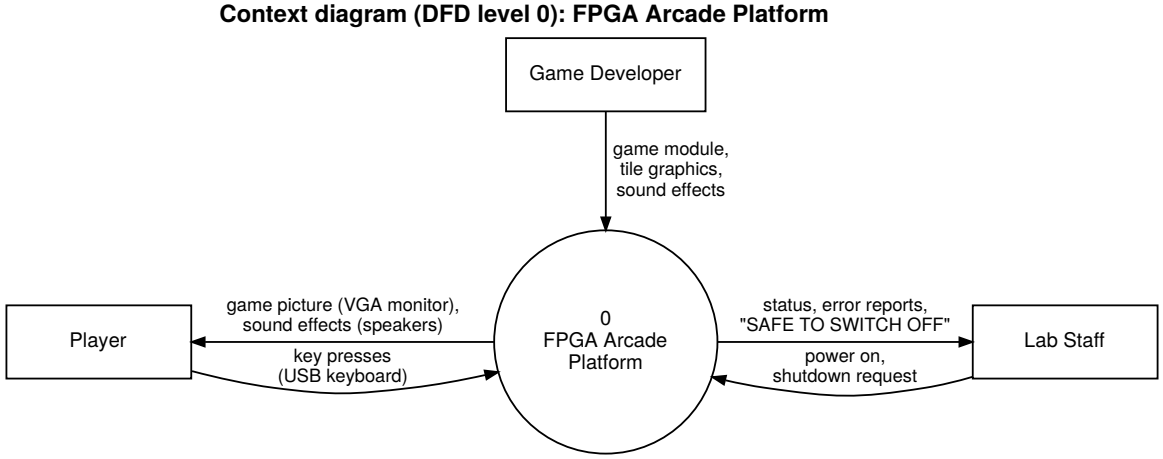


Figure 2.2: Context diagram (DFD level 0)

2.2.1 User Requirements

The user requirements below state what each stakeholder needs, in natural language. Every system requirement in section 3.2 and every non-functional requirement in chapter 4 traces to exactly one of them (Appendix B).

Table 2.1: User requirements

ID	User requirement	Stakeholder	Priority
UR-01	The system shall start from power-on to the main menu without a PC connected.	Player	Shall
UR-02	The player shall be able to choose a game from a main menu using the keyboard.	Player	Shall
UR-03	The player shall be able to play Snake.	Player	Shall
UR-04	The player shall be able to play Breakout.	Player	Shall
UR-05	The player shall control the active game with a USB keyboard, and each keypress shall take effect on screen without noticeable delay.	Player	Shall
UR-06	The system shall display the game on a standard VGA monitor at 1024×768.	Player	Shall
UR-07	The system shall show the current score, and lives where the game has them, during play.	Player	Shall

(continued)

ID	User requirement	Stakeholder	Priority
UR-08	The player shall be able to pause and resume a game.	Player	Shall
UR-09	The player shall be able to leave a game and return to the main menu at any time.	Player	Shall
UR-10	When a game ends, the system shall show the final score and let the player play again or return to the menu.	Player	Shall
UR-11	When a score qualifies for the high-score table, the player shall be able to enter three initials.	Player	Shall
UR-12	The player shall be able to view the top 10 scores for each game, and these shall be kept after power-off.	Player	Shall
UR-13	The display shall never show a torn or half-updated picture.	Player	Shall
UR-14	If the keyboard is unplugged, the system shall keep running and accept input again when it is reconnected.	Player	Shall
UR-15	If the graphics hardware is not ready, the system shall report it instead of hanging.	Player, lab staff	Shall
UR-16	A developer shall be able to add a new game without changing the FPGA design or the HAL.	Game developer	Shall
UR-17	A developer shall be able to run and debug the games on a PC without the DE1-SoC board.	Game developer	Shall
UR-18	The system should play short sound effects for game events (food eaten, brick hit, game over, menu move), and the player should be able to mute them.	Player	Should
UR-19	The player shall be able to shut the system down safely from the main menu before switching off power.	Player, lab staff	Shall

2.3 Users and Characteristics

2.3.1 Stakeholders and Viewpoints

Stakeholders are grouped by viewpoint, following the viewpoint-oriented approach of Sommerville [3]: **interactor** viewpoints interact with the system directly, **indirect** viewpoints have an interest in the system without using it during play, and **domain** viewpoints are standards and platform characteristics that constrain it.

Table 2.2: Stakeholders and viewpoints

Viewpoint	Stakeholder	Interest in the system	Key user requirements
Interactor	Player	Plays the games, reads the screen, uses the keyboard and speakers	UR-01 to UR-14, UR-18, UR-19
Interactor	USB keyboard, VGA monitor, speakers (external devices)	Exchange signals with the system at its physical interfaces	UR-05, UR-06, UR-18
Indirect	Course instructor (client and assessor)	Defines the problem, approves scope, grades the SRS and the final demonstration	All; 1.2 scope
Indirect	Game developer (team now, future students)	Adds and tests games through the Arcade API without touching hardware	UR-16, UR-17
Indirect	Lab staff	Provides boards and monitors, powers the system on and off, diagnoses faults	UR-15, UR-19

(continued)

Viewpoint	Stakeholder	Interest in the system	Key user requirements
Indirect	Development team	Builds, integrates and tests the HPS software and FPGA design within the semester	CON-08, CON-11
Domain	VESA display timing and VGA analog levels	Fix the 1024 × 768 at 60 Hz timing and the signal levels	UR-06; SR-VGA
Domain	USB HID class and the Linux input subsystem	Fix how key events reach the program	UR-05, UR-14; SR-INP
Domain	DE1-SoC board and Cyclone V SoC	Fix the bridge, memory, clocks, audio codec and pin assignments	CON-01 to CON-04
Domain	Photosensitivity guidance (WCAG 2.3.1)	Limits flashing content	SF-01

2.3.2 User Classes

Table 2.3: User classes

User class	Frequency of use	Technical expertise	Functions used	Importance
Player	Every session	None assumed; reads English text on screen	Menu, both games, pause, high scores, sound, shutdown	**Primary**: the system succeeds only if a first-time player can start a game unaided (QA-06)
Game developer	During development	C++17 and the Arcade API; no FPGA knowledge required	SDL2 backend, Arcade API, game interface, TBL-GameRegistry	**Secondary**: extensibility is a core design goal (UR-16, UR-17)
Lab staff	At setup and shutdown	Basic lab equipment handling	Power on, safe shutdown, serial-console error messages	**Secondary**
Course instructor	At assessment	Software engineering expert	Observes all functions during the demonstration	**Primary** for acceptance, not for daily use

The system has a single privilege level for players; there are no logins or accounts. Developers and lab staff use the serial console, which is outside the player interface (4.2.1).

2.4 Operating Environment

The FPGA Arcade Platform runs on one Terasic DE1-SoC board connected to a VGA monitor and a USB keyboard. The game software runs as a user-space process on Embedded Linux on the board's ARM Cortex-A9 Hard Processor System (HPS), booted from a microSD card. The FPGA fabric on the same chip generates the 1024×768 @ 60 Hz VGA picture and the sound-effect tones, which the board's audio codec plays through powered speakers or headphones. No PC, network connection or other device is needed once the board is powered on (UR-01). During development, a PC is connected over USB for the serial console and for FPGA programming, but it is not part of the operating environment.

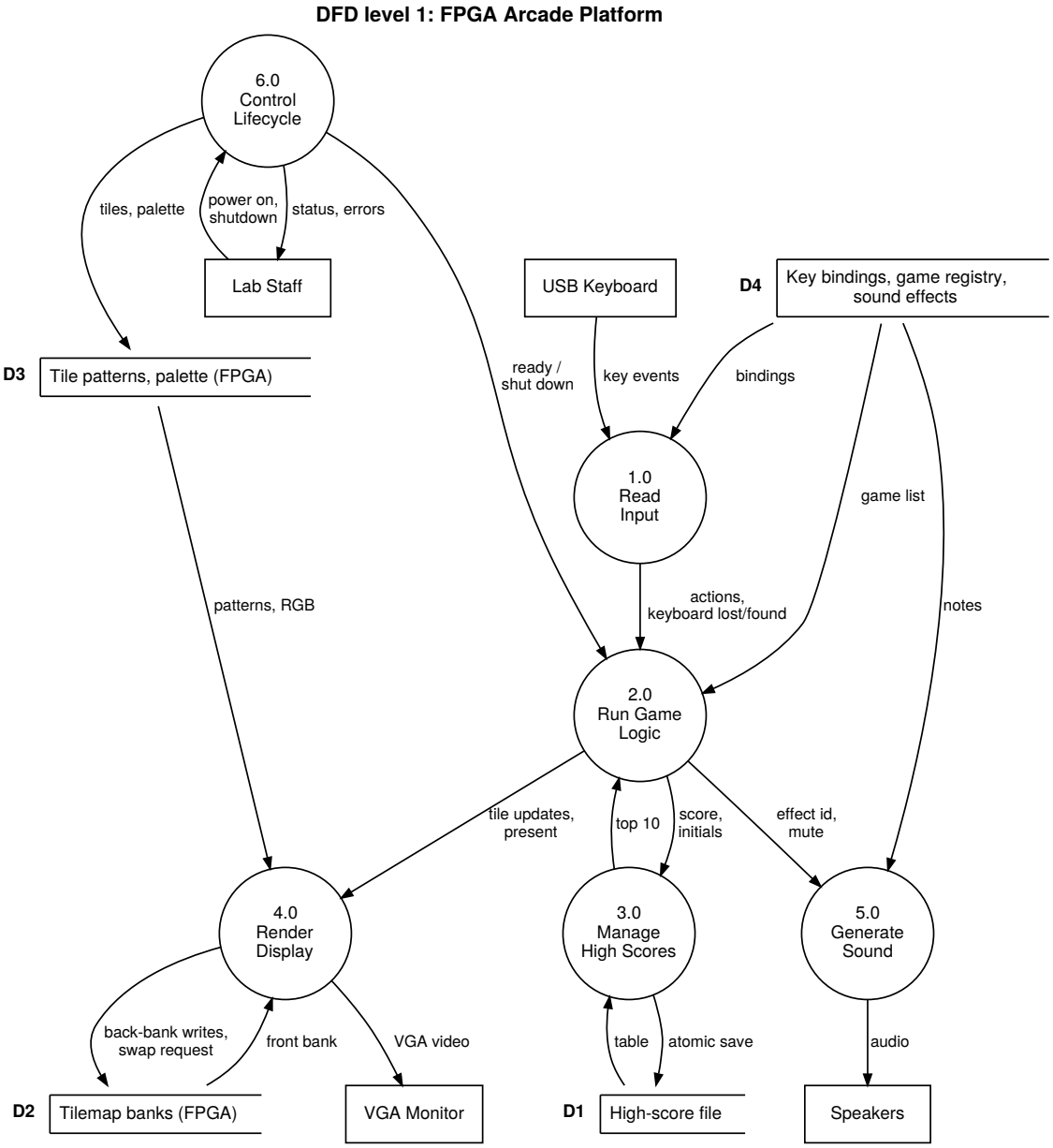


Figure 2.3: DFD level 1 (processes 1.0–6.0, data stores D1–D4)

Minimum platform

Table 2.4: Minimum platform

Item	Requirement	Source
Board	Terasic DE1-SoC with Cyclone V SE 5CSEMA5F31C6. HPS: 800 MHz dual-core ARM Cortex-A9, 1 GB DDR3 SDRAM (32-bit bus)	Manual §2.1, §3.7.4
Power	12 V DC adapter, supplied in the DE1-SoC kit	Manual §1.1, §2.2
Storage	microSD card holding the Linux image, the game program, the FPGA configuration file and the high-score file	Manual §3.7.5
Display	Monitor with an analog VGA (15-pin D-SUB) input that accepts 1024×768 @ 60 Hz	Manual §3.6.6
Cable	VGA cable, 15-pin D-SUB male to male	—

(continued)

Item	Requirement	Source
Input	USB keyboard (HID class), plugged into either of the board's two USB 2.0 Type-A host ports	Manual §3.7.6
Audio (for UR-18)	Powered (active) speakers or headphones with a 3.5 mm stereo plug, in the green line-out jack. The board has no built-in speaker. Without them the games still run, silently	Manual §2.1, §3.6.4
Configuration switch	SW10 (MSEL[4:0]) set for FPGA configuration by the HPS: `01010` when Linux loads the FPGA, `00000` when U-Boot loads it from the SD card	Manual §3.1, Table 3-2

The Linux interfaces the software relies on are specified in 3.1.3 Software Interfaces; the Linux image and FPGA configuration method are those of the lab's DE1-SoC set-up (2.7).

The physical setup is shown in Figure 2.5: mains → 12 V adapter → DE1-SoC; DE1-SoC VGA port → VGA cable → monitor; USB keyboard → HPS USB host port; line-out jack → speakers; microSD card in the HPS socket.

> ***Open item:*** *lab monitor support for 1024×768 @ 60 Hz over VGA is being checked with lab staff. If the lab monitors have no VGA input, the team brings or arranges a VGA monitor.*

2.5 Design and Implementation Constraints

CON-01 Single hardware platform. The system shall run on one Terasic DE1-SoC board with the Cyclone V SE 5CSEMA5F31C6 device and no other processing hardware.

Rationale the course provides the DE1-SoC, and the product is defined as a self-contained arcade on one board.
Source Manual §2.1

CON-02 Hard real-time video timing. VGA timing shall be generated entirely in FPGA logic at a 65.000 MHz pixel clock (15.38 ns per pixel), with no part of the timing depending on software.

Rationale Linux on the HPS is not a real-time OS; one late pixel or sync pulse makes the monitor lose lock.
Source Manual §3.6.6, Tables 3-14, 3-15

CON-03 Pixel clock source. The 65.000 MHz pixel clock shall be derived by an FPGA PLL from a 50 MHz on-board clock (50 MHz × 13 ÷ 10).

Rationale the board provides four 50 MHz clocks to the FPGA and no 65 MHz oscillator; 13/10 gives 65 MHz exactly.
Source Manual §3.5, Table 3-5

CON-04 On-chip memory budget. All video memory (two tilemap banks, tile patterns and palette) shall fit in the 5CSEMA5's on-chip memory, which has 397 M10K blocks (3,970 Kb) plus 480 Kb of MLAB.

Budget (estimate, to be confirmed by the Quartus fit report)

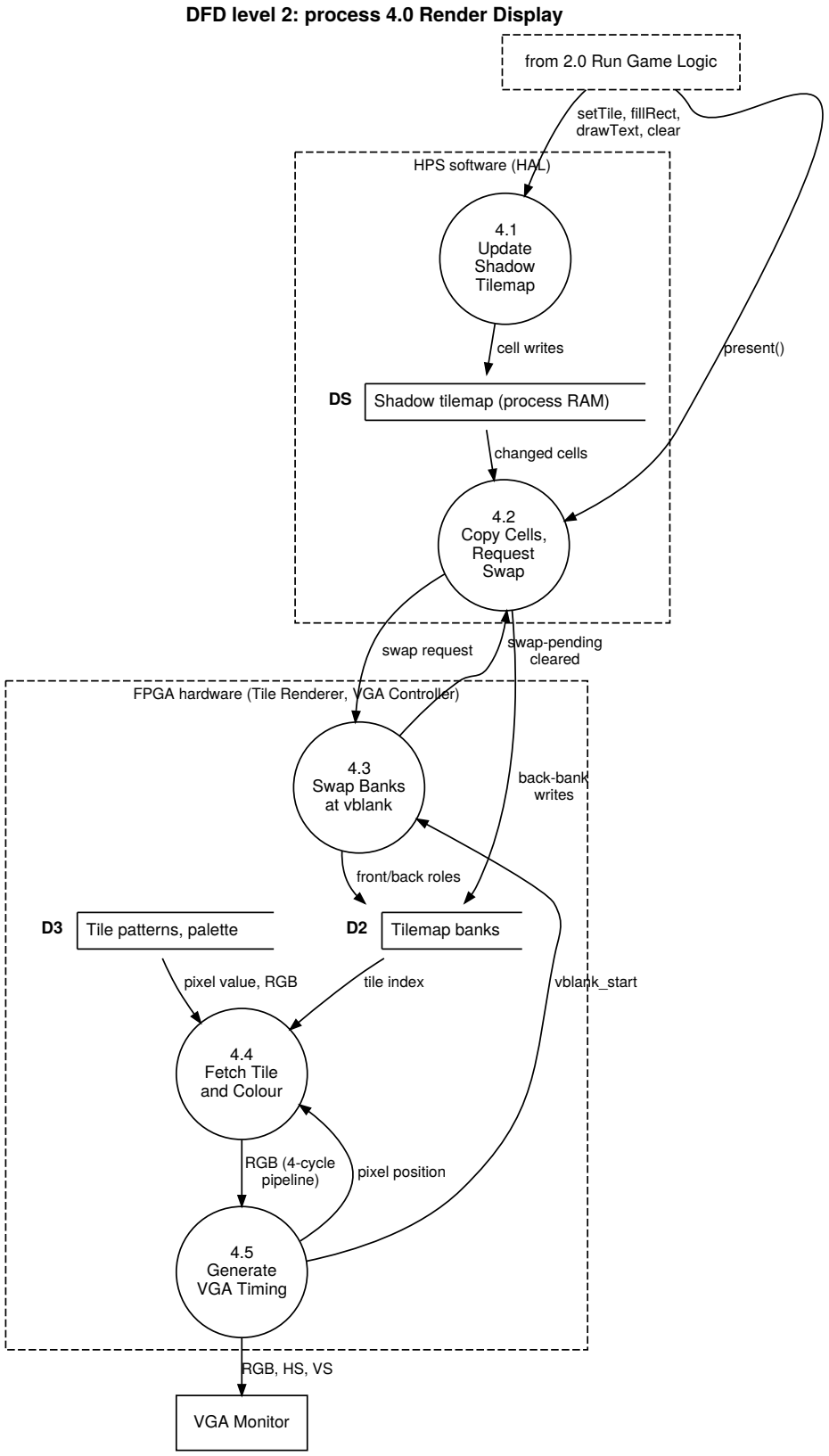


Figure 2.4: DFD level 2, process 4.0 Render Display (HPS and FPGA)

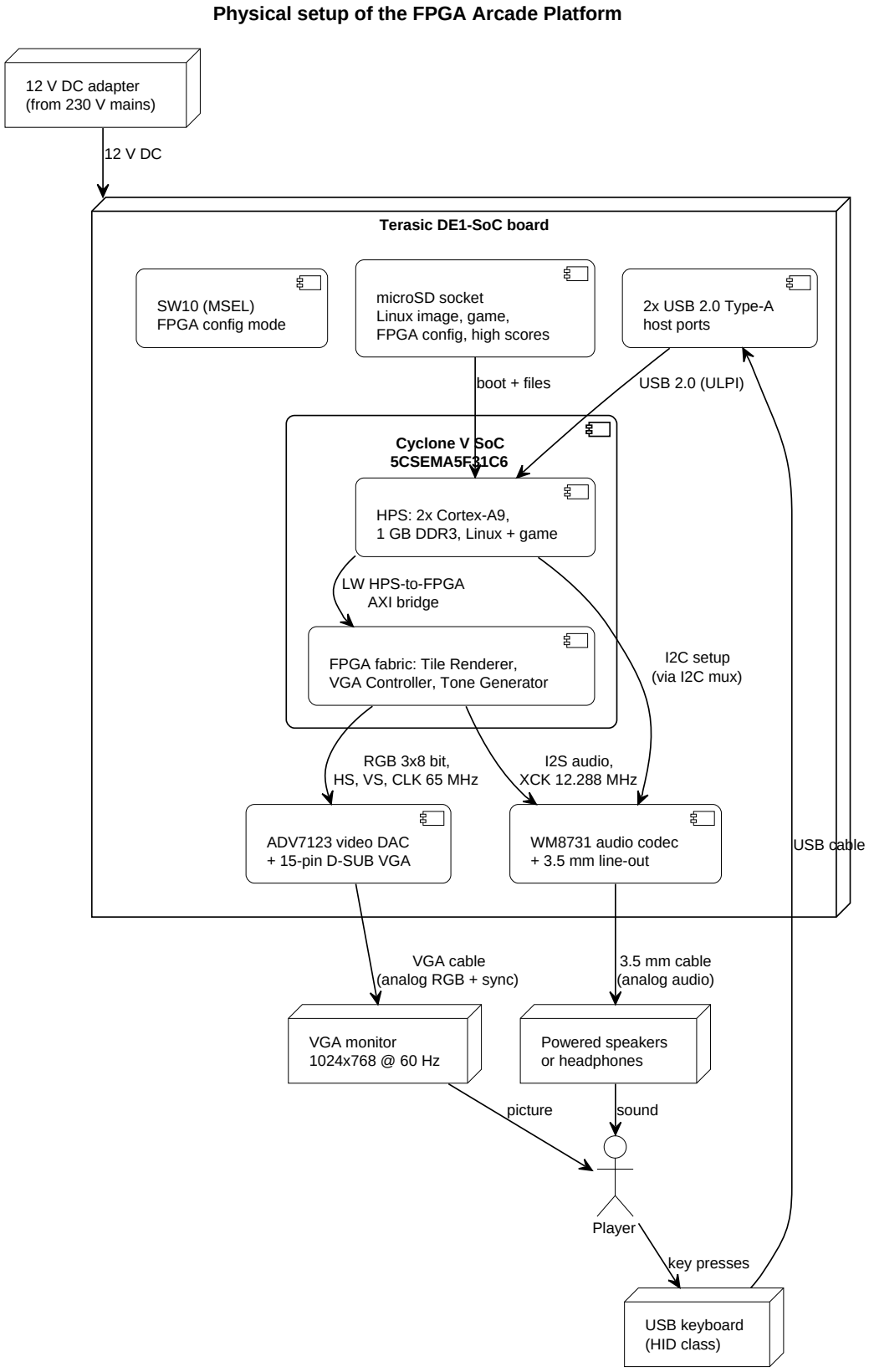


Figure 2.5: Physical setup

Table 2.5: On-chip memory budget (CON-04)

Memory	Size	M10K blocks (estimate)
Tilemap bank A	12,288 B (128 × 96 × 8 bit)	12 (1K × 8 each)
Tilemap bank B	12,288 B	12
Tile-pattern RAM	8,192 B (256 tiles × 64 px × 4 bit)	8 (256 × 32 each)
Palette	64 B (16 entries × 32 bit, `0x00RRGGBB`)	1 (or MLAB)
Tone Generator	3 registers, no sample memory	0
Total	32,832 B	≈ 33 of 397 (≈ 8 %)

Rationale external SDRAM adds latency and a memory controller; the budget leaves over 90 % of block RAM free. Sound adds no block RAM because tones are generated, not stored (CON-13).
Source Intel Cyclone V Device Overview, embedded memory table

CON-05 Tile-only graphics. The display shall be built only from 8×8 px tiles on a 128 × 96 grid, with 16 on-screen colours. There are no hardware sprites, so every object moves in whole-tile (8 px) steps.

Rationale design baseline decision (tile size 8×8, 16-entry palette); keeps the Tile Renderer and memory small.

CON-06 Colour depth. Colour output shall use the board's ADV7123 DAC at 8 bits per channel; the 16 palette entries are chosen from this 24-bit range.

Rationale the board wires only the upper 8 of the DAC's 10 bits per channel.
Source Manual §3.6.6

CON-07 Keyboard path. Keyboard input shall come only from a USB HID keyboard on an HPS USB host port, read through the Linux input subsystem. The board's PS/2 port shall not be used.

Rationale design baseline decision (13 Oct). This changes Objective 5 of the approved proposal; 1.2 Scope records the change.
Source Manual §3.7.6

CON-08 Languages and toolchain. Hardware shall be written in SystemVerilog and built with Quartus Prime and Platform Designer, then simulated with QuestaSim. Software shall be written in C++17 with GoogleTest for unit tests and SDL2 for the PC simulation backend.

Rationale course requirement and team decision.
Version Quartus Prime 25.1 Std.

CON-09 Offline operation. The system shall provide no network services. The board's Gigabit Ethernet port shall not be used at run time.

Rationale a single-player arcade needs no network; this also removes remote attack paths (see 4.2 Security).

CON-10 Analog VGA only. Video output shall be analog VGA through the board's 15-pin D-SUB connector, so the monitor must have a VGA input (or a VGA-to-HDMI active converter).

Rationale the DE1-SoC has no digital video output.
Source Manual §3.6.6

CON-11 Schedule. The SRS shall be submitted by 19 Oct 2026, and the implementation shall be completed within the CS-326 semester by a team of three.

Rationale course deadline; limits scope to two games (Space Invaders is future scope).

CON-12 Audio path. Sound samples shall be generated in FPGA logic and sent to the

WM8731 codec over its digital audio interface. The codec shall be configured by the HPS over I2C.

Rationale the codec's audio data pins connect only to the FPGA, while its I2C control bus can be switched to the HPS. Configuring it from Linux needs no version-specific Quartus IP and no FPGA rebuild to change settings.

Source Manual §3.6.4, Table 3-12; §3.6.5, Table 3-13

CON-13 Tone-only sound. Sound shall be one square-wave tone at a time, with no recorded sound samples.

Rationale tones need no sample memory and are enough for game-event sounds; sound is a "should" feature (UR-18), so its cost is kept small.

2.5.1 Design Decisions

Table 2.6: Design decisions

ID	Decision	Detail	Traced to
D1	Boot-to-menu time	$\leq$ 60 s from power-on to the main menu	SR-FW-01, PR-07
D2	High-score file	`/var/lib/arcade/highscores.csv`, one line per entry: `game_id,rank,initials,score,seq`. It is written to a temp file, then `fsync()`, then `rename()` over the old file	TBL-HighScore, SF-04
D3	Sound at boot	On, at the capped amplitude of SR-AUD-07; M toggles it. The cap, not muting, protects headphone users	SR-FW-11, SR-AUD-07
D4	Game end	A score that qualifies for the top 10 goes straight to Initials, then High Scores; any other score goes to Game Over	App state machine, SR-UI
D5	Keyboard lost outside a game	In Menu, GameOver, Initials or HighScores the state stays the same and screen 9's notice shows until a keyboard is found. Only Playing and Paused move to KeyboardLost	SR-UI-11, SR-UI-15
D6	Safe shutdown	Hold Esc for 3 s on the menu to open the confirm box, then: mute, save, show "SAFE TO SWITCH OFF" and `present()`, `shutdown()` (display enable stays set), `poweroff`. SIGTERM/SIGINT run the same clean-up without `poweroff`	UR-19, SR-UI-14, SR-FW-13/14, SF-07

2.6 User Documentation

The system will be delivered with four documents. A one-page **quick-start card** shows how to connect power, monitor, keyboard and speakers, set SW10, and start the arcade. It also tells users to place the board on a non-conductive surface, handle it only by its edges, and connect or disconnect the 12 V supply only with the power switch off (SF-06). It gives the shutdown steps: on the main menu, hold Esc for 3 s, press Enter at "SHUT DOWN?", wait for "SAFE TO SWITCH OFF", then switch off the power. It warns that switching off at any other time can corrupt the microSD card (SF-07). A **player manual** describes each screen, the controls for Snake and Breakout (the key table in 3.1.1), scoring, sound on/off, and how high scores are saved. A **developer guide** explains how to add a new game: implement the game interface (`init`, `update`, `render`, `isOver`, `score`), register it in TBL-GameRegistry, load its tiles with `loadTiles`, and test it on a PC with the SDL2 backend before running it on the board. A **README** in the GitHub repository covers building the FPGA design and the C++ program, preparing the microSD card, and running the unit tests. All documents are written in English,

in Markdown, and kept in the repository.

2.7 Assumptions and Dependencies

AS-01 The Linux image running on the HPS permits root access to `/dev/mem`, which the HAL requires to `mmap()` the lightweight bridge window.

AS-02 The FPGA bitstream implementing the Tile Renderer, VGA Controller and bridge interface is loaded and the lightweight bridge is enabled before the game process starts.

AS-03 The lab's VGA monitors accept 1024×768 @ 60 Hz input over the DE1-SoC's VGA output, per the agreed design baseline.

AS-04 The microSD card has sufficient free space and write endurance for periodic, atomic rewrites of the high-score file over the project's demo lifetime.

AS-05 Only one game process accesses the lightweight bridge at a time; the system does not need to arbitrate between multiple concurrent HPS-side clients.

AS-06 The USB keyboard is HID-compliant and is recognised by the Linux kernel's standard input subsystem (evdev) without a custom driver.

AS-07 The on-chip M10K memory budget on the 5CSEMA5F31C6 is sufficient to hold two 12 KB tilemap banks plus the 8 KB tile pattern RAM concurrently; this is not re-verified in this SRS and is treated as a constraint handed down from the design baseline.

AS-08 The SDL2 simulation backend is available on the development PC used for off-board testing (UR-17) and is not required on the DE1-SoC itself.

AS-09 Powered speakers or headphones are available at the demo for the board's line-out jack; the DE1-SoC has no built-in speaker (UR-18). Without them the games run silently.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

All screens are drawn on the 128 × 96 tile grid (columns 0–127, rows 0–95, 8 × 8 px per tile). Positions are given as (column, row) in tiles. Each screen has a clean mockup in this section, drawn to scale by `3.1.1-mockups.py`, which extends the reference menu script. The mockups use the reference palette (entries 0–14); the final colour values are set in TBL-Palette.

Text. `drawText` takes a scale of 1 or 4:

- **Scale 1:** one font tile per character (ASCII 0x20–0x7E, 5 × 7 glyphs). Used for HUDs, captions and hint bars.
- **Scale 4:** each character covers 4 × 4 tiles and is built from 16 "quadrant mosaic" tiles. Each mosaic tile shows 2 × 2 font pixels, so one font pixel is 4 × 4 screen pixels. Used for logos, menu items and screen titles.
- **Tile budget:** the font (95 tiles) and the mosaic set (16 tiles) use 111 of the 256 tile patterns, leaving 145 for game graphics.

Common layout. Every screen has a border frame on rows 0 and 95 and columns 0 and 127, with a blue accent tile every 8 columns on rows 0 and 95.

Table 3.1: Common screen layout

Region	During play (screens 3–6, 9)	Other screens (1, 2, 7, 8)
Top	Row 0 frame; HUD text on row 1	Row 0 frame
Play field / content	Rows 2–93 (walls on rows 2 and 93 where the game has them)	Rows 1–92
Hint bar	Text on row 94; frame on row 95	Text on row 93 (band over columns 1–126); frame on row 95

Screen list and key mapping

Table 3.2: Screen list and key mapping

#	Screen	Shown when	Keys	Mockup
1	Boot / splash	From the first frame after the HAL is ready until the menu loads	—	Figure 3.1
2	Main menu	After boot, after quitting a game, or after Esc on Game over	↑/↓ choose item · Enter start game or open scores · H high scores · M sound on/off · hold Esc 3 s shut down	Figure 3.2
3	Snake	A Snake game is running	Arrow keys turn · P pause · Esc quit to menu · M sound on/off	Figure 3.3
4	Breakout	A Breakout game is running	←/→ move paddle · Space launch ball · P pause · Esc quit to menu · M sound on/off	Figure 3.4

(continued)

#	Screen	Shown when	Keys	Mockup
5	Paused	P pressed during a game	P resume · Esc quit to menu · M sound on/off	Figure 3.5
6	Game over	A game ends and the score is not in the top 10 (decision D4)	Enter play again · Esc main menu · M sound on/off	Figure 3.6
7	Initials entry	A game ends and the score is in the top 10 (decision D4)	A–Z type · Backspace delete · Enter confirm (3 letters)	Figure 3.7
8	High scores	H on the main menu, Enter on SCORES, or after initials entry	←/→ switch game · Esc back · M sound on/off	Figure 3.8
9	Keyboard disconnected	The keyboard is unplugged during play (UR-14, SR-UI-11, decision D5)	(plug in a keyboard)	Figure 3.9
10	Shut down (10a confirm, 10b safe to switch off)	Esc held for 3 s on the main menu (UR-19, SR-UI-14)	10a: Enter confirm · Esc cancel; 10b: none	Figure 3.10, Figure 3.11

The key defaults are stored in TBL-KeyBinding, so remapping a key needs no code change. M (sound on/off, UR-18) is bound in the Global context of TBL-KeyBinding, with the action Mute, so it works on every screen except Initials entry, where M types a letter (SR-INP-09). The on/off setting lasts until the system is powered off.

Screen layouts

1. **Boot / splash.** "FPGA ARCADE" at scale 4 on rows 34–37, centred (columns 42–85), yellow with a red shadow. Subtitle "SILICON TRIAD - DE1-SOC - 1024X768" at scale 1 on row 44. Progress bar on row 50, columns 32–95, filling in green as start-up proceeds. "STARTING..." on row 53. Only if no keyboard is found, row 60 shows "NO KEYBOARD FOUND - PLUG IN A USB KEYBOARD" in red. Before the HAL has mapped the hardware, the VGA Controller outputs a black screen (SR-VGA-09), so this screen is the first picture the player sees.

2. **Main menu (reference screen).**

   - Logo "FPGA ARCADE" at scale 4 on rows 6–9, subtitle on row 16, divider on row 19 (columns 16–111).
   - Menu items at scale 4 from column 13: "SNAKE" on row 28, "BREAKOUT" on row 38, "SCORES" on row 48. The selected item has a highlight band (columns 6–55, from 2 rows above to 5 rows below the text), a green bar in column 6 and a scale-4 ">" at column 8.
   - Preview panel on the right: a green frame over columns 60–121 and rows 24–77, showing the selected game in 3 × 3-tile cells. Row 74 inside the panel shows the best score and the game's own caption. A tagline sits under the panel on row 80.
   - "PRESS ENTER TO PLAY" on row 62 (column 13), status line "SOUND ON - 2 GAMES - TOP 10 SAVED" on row 86.
   - Hint bar on row 93: "^/v CHOOSE ENTER PLAY H HIGH SCORES M SOUND: ON HOLD ESC: SHUT DOWN".

3. **Snake.** HUD on row 1: “SNAKE” at column 2, “SCORE 000120” centred, “HI 000560” right-aligned to column 125. The board is 42 × 30 Snake cells, each drawn as 3 × 3 tiles (24 × 24 px), covering columns 1–126 and rows 3–92. The walls are column 0, column 127, row 2 and row 93. Snake body in dark green, head in bright green, food in red. Hint bar on row 94: “ARROWS TURN P PAUSE ESC MENU M SOUND: ON”.

4. **Breakout.** HUD on row 1: “BREAKOUT” at column 2, “SCORE 000340” centred, “LIVES 3” right-aligned to column 125. Walls are column 0, column 127 and row 2; the bottom is open. Each brick sits in a 6 × 2-tile slot (5 tiles of brick plus a 1-tile gap): 21 bricks per row across columns 1–126, 6 rows on rows 8–19, one colour per row (red, orange, yellow, green, cyan, blue). The paddle is 16 × 1 tiles on row 90 (columns 56–71 when centred); the ball is 2 × 2 tiles (16 × 16 px), drawn round with 4 ball tiles. Hint bar on row 94: “</> MOVE SPACE LAUNCH P PAUSE ESC MENU M SOUND: ON”.

5. **Paused.** The game picture stays in place. A box with a blue frame covers columns 40–87 and rows 38–57: “PAUSED” at scale 4 on rows 41–44, then “P RESUME” on row 48, “ESC QUIT TO MENU” on row 50 and “M SOUND ON/OFF” on row 52. Hint bar on row 94: “P RESUME ESC MENU M SOUND: ON”.

6. **Game over.** The final game picture stays in place. A box with a red frame covers columns 36–91 and rows 34–61: “GAME OVER” at scale 4 on rows 37–40, “SCORE 000340” on row 45, “HI 000560” on row 47, “ENTER PLAY AGAIN” on row 52 and “ESC MAIN MENU” on row 54. Hint bar on row 94.

7. **Initials entry.** “NEW HIGH SCORE!” at scale 4 on rows 18–21. Game, score and rank (“SNAKE - SCORE 000560 - RANK 2”) on row 28; divider on row 32. Three letter slots at scale 4, starting at columns 50, 60 and 70, with text on rows 40–43, each on a highlight background (rows 38–45) with an underline on row 46. The current slot’s underline is green, the others grey. “ENTER YOUR INITIALS” on row 52. Hint bar on row 93: “A-Z TYPE BACKSPACE DELETE ENTER CONFIRM”.

8. **High scores.** “HIGH SCORES” at scale 4 on rows 6–9, divider on row 16. Game selector on rows 20–23: the game name at scale 4, centred, with green “<” and “>” at columns 40 and 84. Column headings on row 30 (“RANK” column 36, “NAME” column 56, “SCORE” column 80), divider on row 32. Ten entries on rows 34–52, one every 2 rows; rank 1 is shown in yellow. Hint bar on row 93: “</> SWITCH GAME ESC BACK M SOUND: ON”.

9. **Keyboard disconnected.** The game is paused and its picture stays in place. A box with a red frame covers columns 30–97 and rows 38–57: “NO KEYBOARD” at scale 4 on rows 41–44, “PLUG IN A USB KEYBOARD TO CONTINUE” on row 49 and “GAME PAUSED” on row 52. The hint bar on row 94 is empty because no keys work. When a keyboard is detected, the box is replaced by the Paused screen (5), and the game continues only when the player presses P.

10. **Shut down (UR-19, SR-UI-14).**

    - **10a, confirm:** the main menu stays in place. A box with a blue frame covers columns 40–87 and rows 38–57, the same as the Paused box: “SHUT DOWN?” at scale 4 on rows 41–44, “ENTER YES” on row 48 and “ESC NO” on row 50. The hint bar on row 93 reads “ENTER CONFIRM ESC CANCEL”. Esc closes the box and returns to the menu.
    - **10b, safe to switch off:** after Enter, the program mutes, saves and releases the hardware (SR-FW-13/14). The whole screen is black (palette entry 0, no frame),



Figure 3.1: Screen 1 – Boot / splash



Figure 3.2: Screen 2 – Main menu (reference screen)

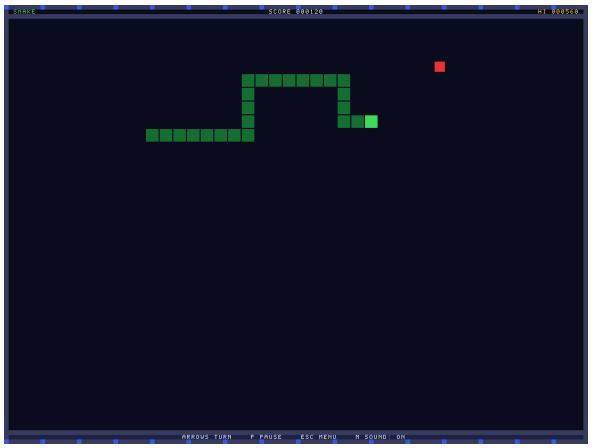


Figure 3.3: Screen 3 – Snake

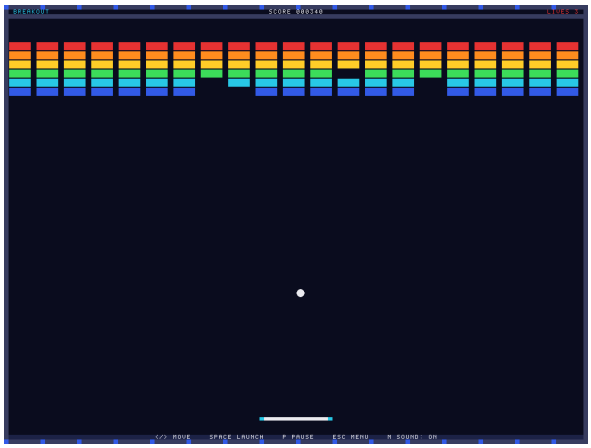


Figure 3.4: Screen 4 – Breakout

with “SAFE TO SWITCH OFF” at scale 4 on rows 46–49, centred (columns 28–99) in white. No keys work. The power may be switched off only after this screen appears (SF-07).

3.1.2 Hardware Interfaces

3.1.2.1 VGA output

The VGA Controller in the FPGA drives the board’s Analog Devices ADV7123 video DAC, which drives the 15-pin D-SUB VGA connector (Manual §3.6.6).

- **DAC:** ADV7123 triple 10-bit video DAC; the board connects only the upper 8 bits per channel, so colour is 24-bit RGB. The board is rated up to 1280×1024 at a 100 MHz pixel clock (Manual §3.6.6).
- **Data latching:** pixel data, BLANK and SYNC are latched on the rising edge of the DAC clock, with a pipeline delay of 1 clock cycle (ADV7123 datasheet, Tables 6 and 8).
- **Analog levels:** the R, G and B outputs drive RS-343A video levels into a doubly terminated 75 Ω load: 0 V for black and about 0.7 V for full white (ADV7123 datasheet, Figure 23, Analog Outputs).



Figure 3.5: Screen 5 – Paused



Figure 3.6: Screen 6 – Game over



Figure 3.7: Screen 7 – Initials entry



Figure 3.8: Screen 8 – High scores

- **Sync signals:** HSYNC and VSYNC go directly from the FPGA to the connector as 3.3 V digital signals (Manual Table 3-16).

All signals use the 3.3 V I/O standard.

Pixel clock 65.000 MHz. Line rate = 65 MHz ÷ 1344 = 48.36 kHz. Frame rate = 48.36 kHz ÷ 806 = 60.004 Hz. Source: Manual Tables 3-14 and 3-15 (pixel and line counts follow from the 65 MHz clock).

Pixel clock 25.175 MHz nominal (the manual lists 25 MHz). Source: Manual Tables 3-14 and 3-15.

3.1.2.2 USB keyboard

- **Ports:** the board has two USB 2.0 Type-A host ports. An SMSC USB3300 PHY connects to the HPS USB 2.0 controller over ULPI, through an SMSC USB2512B 2-port hub; the PHY runs in host mode, powering the two ports (Manual §3.7.6).
- **Keyboard type:** any USB keyboard of the HID class.
- **Protocol handling:** USB enumeration and the HID protocol are handled by the Linux kernel's USB host and HID drivers, which present the keyboard as an event device `/dev/input/eventN` (Linux input subsystem).
- **Events:** each key change arrives as a `struct input_event` of type `EV_KEY` with a key

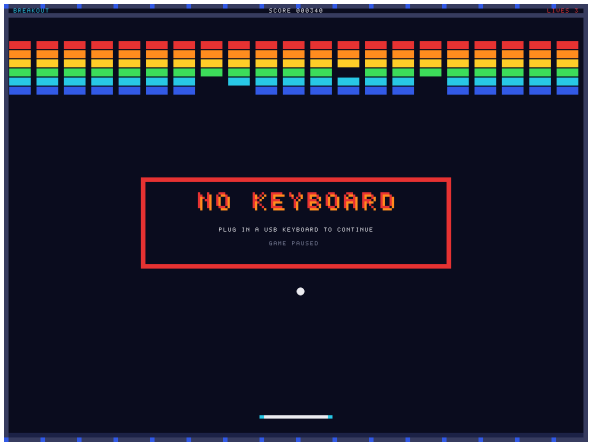


Figure 3.9: Screen 9 – Keyboard disconnected



Figure 3.10: Screen 10a – Shut down, confirm



Figure 3.11: Screen 10b – Shut down, safe to switch off

code (`KEY_*`) and a value of 1 for key press, 0 for key release and 2 for auto-repeat (input event codes).

- **No FPGA involvement:** the FPGA has no part in keyboard input, and there is no PS/2 or USB logic in the FPGA design.

3.1.2.3 Board power and reset

- **Power:** 12 V DC input; the 12 V power LED (D14) lights when power is on (Manual §2.2, Table 3-3).
- **Cold reset — KEY5 (HPS_RESET_N):** resets the HPS, the Ethernet PHY and the USB host. This restarts Linux and the game (Manual §3.4, Table 3-4).
- **Warm reset — KEY7 (HPS_WARM_RST_N):** resets the HPS system domain, for debugging (Manual Table 3-4).
- **FPGA push-buttons KEY0–KEY3 and switches SW0–SW9:** not used by the games.
- **Power loss:** removing power at any time must not corrupt the high-score file. The file is written atomically (see SR-FW and 4.2 Safety).

3.1.2.4 Audio output

The Tone Generator in the FPGA sends sound to the board's Wolfson WM8731 audio codec, which drives the 3.5 mm line-out jack (Manual §3.6.4).

- **Codec:** WM8731, 24-bit, with microphone-in, line-in and line-out jacks and a sample rate adjustable from 8 kHz to 96 kHz. Only the DAC and line-out are used; line-in and microphone-in are not used (Manual §3.6.4).
- **Speaker:** the board has no built-in speaker. Powered speakers or headphones plug into the green line-out jack (2.4).
- **Output level:** the line output has a fixed 0 dB gain; full scale is 1.0 × AVDD / 3.3 Vrms. The codec's analog volume control applies only to its headphone output, so on line-out the volume is set by the sample amplitude (SR-AUD-07) (WM8731 datasheet, Electrical Characteristics, Table 9).
- **Clock and format:** the codec runs in slave mode, so the FPGA supplies the bit clock and the left/right clock. Samples are sent in I^2S format, 16 bits per channel, at 48 kHz with a 12.288 MHz master clock (256 × 48 kHz) (WM8731 datasheet, Digital Audio Interfaces; Table 18).

All signals use the 3.3 V I/O standard.

Control interface (codec set-up)

- **Bus:** the codec's registers are written over I2C. An I2C multiplexer connects this bus to the FPGA (PIN_J12/PIN_K12) or to the HPS. The HPS gets the bus only when HPS_I2C_CONTROL is high (Manual §3.6.5, Table 3-13).
- **Switching the bus to the HPS:** HPS_I2C_CONTROL is HPS_GPIO48, bit 19 of the GPIO1 controller, whose registers are at 0xFF708000 (Manual §6.2, §6.4).
- **Linux access:** the HAL writes the codec registers through the Linux I2C driver at `/dev/i2c-0`, as in the manual's I2C multiplexer demo (Manual §6.4).
- **Registers used:** Reset (R15), Analogue Audio Path Control (R4), Digital Audio Path Control (R5), Digital Audio Interface Format (R7) and Sampling Control (R8), plus the power-down and activation registers (R6, R9; Power Down Control and Active Control). Each write carries a 7-bit register address and 9 bits of data (WM8731 datasheet, register descriptions).
- **Device address:** 0x1A (7-bit), set by the CSB pin being low (WM8731 datasheet, software control interface).

3.1.3 Software Interfaces

The game runs as a single C++17 process on embedded Linux (Terasic/Intel DE1-SoC image) on the HPS. At start-up, the process:

1. Opens `/dev/mem` and calls `mmap()` to map the 2 MB lightweight HPS-to-FPGA bridge window (base 0xFF20_0000) into its own virtual address space (Cyclone V HPS Technical Reference Manual, "HPS-FPGA Bridges"; mmap(2)). This requires root privileges, since `/dev/mem` is owned by root (mem(4)). *(Decision: `/dev/mem` is used, not UIO. UIO is noted only as a possible future improvement, not an open item.)*

2. Assumes the FPGA bitstream has already been loaded onto the fabric before the process starts, either by the Linux boot process (preloaded bitstream) or by a prior manual step; this SRS does not define the bitstream-loading mechanism itself (see AS-02).

3. Opens `/dev/i2c-0` to configure the WM8731 audio codec (SR-HAL-10, CON-12). Like `/dev/mem`, this needs root (or membership of the `i2c` group).

4. Links against: the C++ standard library, GoogleTest (unit testing only, not present in the shipped binary), and SDL2 (used exclusively by the PC-side simulation backend for development and debugging, per UR-17 — not linked into the FPGA-backend build).

No other software interfaces (databases, external APIs, other processes) are required; the game process is self-contained once started.

3.1.4 Communications Interfaces

The HPS and FPGA communicate exclusively through the **lightweight HPS-to-FPGA AXI bridge**, a memory-mapped interface exposing a 2 MB window at physical base address `0xFF20_0000` (Cyclone V HPS Technical Reference Manual, "HPS-FPGA Bridges"). The C++ game process maps this window with `mmap()` on `/dev/mem` and accesses it thereafter as ordinary memory reads and writes — no DMA and no interrupts are used.

The window exposes the tilemap (two banks), tile pattern RAM, a 16-entry colour palette, and a small set of control/status registers, detailed in the register map below.

Synchronisation method: all drawing operations (`setTile`, `fillRect`, `drawText`, `clear`) write only to a **shadow tilemap kept in HPS process RAM** — no bridge access occurs on these calls. On `present()`, the HAL, in order: copies every shadow cell that changed since the back bank was last written into the back bank over the bridge; requests a bank swap by writing the **Swap Request** register; then polls the Status register until the **swap pending** bit clears (or times out after 100 ms). present() does not wait for vsync before this sequence: waiting first would delay the swap by a full frame and break the 61 ms latency budget. Polling begins only after the Swap Request write, so the swap lands on the very next vblank. The actual swap is performed in hardware: the FPGA applies the swap only at the start of the next vertical blanking interval. This moves the race-prone part of the operation (deciding exactly when the front/back roles flip) into hardware, so software timing jitter can never cause a mid-frame swap.

Data rate: rewriting the entire 12,288-byte tilemap every frame at 60 Hz gives a worst-case bridge write load of approximately 737 kB/s (12,288 × 60). In practice, each tick only rewrites the cells that changed (e.g. the snake's head/tail, a moving ball and paddle), so typical load is expected to be well below this figure. *(Open item: a measured figure for LW-bridge write throughput from Linux userspace, sourced from the DE1-SoC manual or an on-board benchmark, is still needed to confirm this is well within the available bandwidth.)*

There is no network interface anywhere in this system; all communication is local, over the on-chip bridge only.

3.1.4.1 Register map

These offsets are our own Platform Designer address assignment within the 2 MB lightweight-bridge window; only the bridge's base address (`0xFF20_0000`) and 2 MB window size come from the HPS Technical Reference Manual. The offsets below are frozen design, not something to verify against the TRM.

Table 3.3: Register map of the lightweight-bridge window

Offset	Name	R/W	Width	Reset Value	Description
0x0000–0x2FFF	Tilemap Bank 0	R/W	8 bits/cell	0x00	128×96 tile-index grid, bank 0 (12,288 bytes)
0x3000–0x5FFF	Tilemap Bank 1	R/W	8 bits/cell	0x00	128×96 tile-index grid, bank 1 (12,288 bytes)
0x6000–0x7FFF	Tile Pattern RAM	R/W	4 bits/pixel	0x0	256 tile patterns, 8×8 px, 4bpp (8 KB). Packed 2 pixels/byte; low nibble = left pixel, high nibble = right pixel.
0x8000	Palette Base	R/W	32 bits ×16	0x00000000	16-entry colour palette; each entry packed as `0x00RRGGBB`, 8 bits per channel (final — matches the ADV7123 DAC's 8 wired bits/channel, CON-06)
0x8040	Control Register	R/W	32 bits	0x00000000	Bit 0: reserved; Bit 1: reset tilemap; Bit 2: display enable (HAL sets this only after tile patterns and palette are loaded — SR-HAL-13, ties to SR-VGA-09); others reserved
0x8044	Status Register	R	32 bits	0x00000000	Bit 0: vsync flag (cleared on read); Bit 1: FPGA ready; Bit 2: swap pending (set on Swap Request, cleared by hardware once the swap is applied)
0x8048	Frame Counter	R	32 bits	0x00000000	Increments once per displayed frame since power-on
0x804C	Swap Request / Front Bank	W: Swap Request (1 bit); R: Front Bank (1 bit)	1 bit	0x0	Write 1 to request a bank swap at the next vertical blanking start. Read returns which bank (0/1) is currently front (displayed), so the HAL knows which is the writable back bank.
0x8050	ID / Version	R	32 bits	0xA2CADE01	Fixed magic value confirming this design (not some other bitstream) is loaded on the fabric
0x8054	Tone Frequency	R/W	16 bits	0x0000	Frequency in Hz for the Tone Generator, 100–4,000 (SR-AUD-04); written by the HAL on playSound() (SR-HAL-11)

(continued)

Offset	Name	R/W	Width	Reset Value	Description
0x8058	Tone Duration	R/W	16 bits	0x0000	Duration in ms for the current tone, 10–2,000 (SR-AUD-05); written by the HAL on playSound() (SR-HAL-11)
0x805C	Audio Control Register	R/W	32 bits	0x00000000	Bit 0: audio enable (HAL sets after codec I2C setup succeeds, SR-HAL-10); Bit 1: mute (set by setMute(), SR-HAL-12); others reserved

Register offsets, the tile-pattern nibble order and the palette packing format above are this team's own design decisions in Platform Designer, now frozen; they do not require a source citation to the TRM.

3.2 System Functional Requirements

3.2.1 SR-FW — Game framework, Arcade API and game loop

SR-FW-01 At start-up, the Game Engine shall initialise the HAL, load the tile patterns and palette, set display enable and show the main menu within 60 s of power being applied.

Rationale decision D1 fixes the boot-to-menu time; PR-07 measures it. Display enable is set only after tiles and palette are loaded, so no undefined picture is shown (SR-HAL-13, SR-VGA-09).

Parent UR-01 *Verification* Test (stopwatch from power switch to menu, 5 power cycles)

SR-FW-02 The Game Engine shall run exactly one game tick per displayed frame, 60 ticks per second, each tick performing in order: `pollInput()`, the active screen's `update()`, `render()` and `present()`; `present()` shall lock the loop to the display refresh.

Rationale all game speeds are counted in ticks, so gameplay speed is independent of processor load, and input is read once per frame, which bounds input latency (PR-02, PR-03).

Parent UR-05 *Verification* Test (over 10 minutes, the tick count equals the change in `frameCount()`)

SR-FW-03 Every game shall implement the game interface `init()`, `update(input)`, `render(api)`, `isOver()` and `score()`, and the Game Engine shall call games only through this interface.

Rationale one fixed interface makes games interchangeable parts of a product line; the engine never depends on a specific game.

Parent UR-16 *Verification* Inspection (code review: no game-specific calls in the Game Engine)

SR-FW-04 The main menu shall be built from TBL-GameRegistry at start-up, so that adding a game requires only a new class and one registry row, with no change to the Game Engine, the HAL or the FPGA design.

Rationale makes extensibility a property of the design rather than of developer discipline (QA-02).

Parent UR-16 *Verification* Demonstration (add a test game; it appears in the menu after rebuilding the C++ program only)

SR-FW-05 The rendering backend, FPGA or SDL2, shall be selected by a start-up option, and the game code shall be identical for both backends.

Rationale proves the Arcade API is a real abstraction and allows development before the hardware is complete.
Parent UR-17 *Verification* Test (the same game build runs on both backends)

SR-FW-06 The SDL2 backend shall display the 128 × 96 tilemap in a single 1024 × 768 window and take keyboard input from the PC keyboard, mapped to the same actions as on the board.

Rationale the PC simulation must look and behave like the board so that game behaviour seen on the PC is the behaviour on the hardware.
Parent UR-17 *Verification* Demonstration (play both games on a PC)

SR-FW-07 The SDL2 backend shall accept every `playSound()` and `setMute()` call without error; producing audible sound on the PC is optional.

Rationale games must not need backend-specific code paths for sound (SR-FW-05).
Parent UR-17 *Verification* Test (unit test calls every effect on the SDL2 backend)

SR-FW-08 At start-up, the Game Engine shall load TBL-HighScore from `/var/lib/arcade/highscores.csv`; if the file is missing or unreadable it shall start with an empty table, and it shall discard invalid lines (SC-04).

Rationale a missing or damaged file must never stop the arcade from starting.
Parent UR-12 *Verification* Test (start with no file, an empty file and a corrupted file)

SR-FW-09 The Game Engine shall save TBL-HighScore by writing a temporary file, calling `fsync()`, and then calling `rename()` over the existing file.

Rationale `rename()` replaces the file atomically, so a power loss leaves either the complete old or the complete new table (decision D2, SF-04).
Parent UR-12 *Verification* Test (power cut during repeated saves, 20 times)

SR-FW-10 The Game Engine shall keep the top 10 scores per game, sorted by score in descending order; equal scores shall be ranked by entry order, the older entry first.

Rationale defines a single, deterministic ranking for the high-score screen.
Parent UR-12 *Verification* Test (unit test with ties and an 11th entry)

SR-FW-11 Sound should be on at start-up, and each Mute action should toggle it through `setMute()`; the setting should last until power-off.

Rationale decision D3; the amplitude cap (SR-AUD-07, SF-05), not muting, protects headphone users.
Parent UR-18 *Verification* Test

SR-FW-12 `drawText()` shall support scale 1, one font tile per character, and scale 4, built from the 16 quadrant-mosaic tiles in which each tile shows 2 × 2 font pixels.

Rationale readable titles at 1024 × 768 while using only 16 of the 256 tile patterns for large text.
Parent UR-06 *Verification* Inspection (screens match the 3.1.1 mockups)

SR-FW-13 On a confirmed shutdown, the Game Engine shall, in order: mute sound; complete any pending high-score save; draw "SAFE TO SWITCH OFF" and `present()` it; call the HAL's `shutdown()` without clearing display enable; and run `poweroff`.

Rationale halting Linux before power is removed protects the microSD card; keeping display enable set leaves the message on screen until power-off (SR-HAL-07, SF-07).
Parent UR-19 *Verification* Test (shutdown, then check the message stays and the card's file system is clean on the next boot)

SR-FW-14 On receiving SIGTERM or SIGINT, the Game Engine shall perform the steps of SR-FW-13 except `poweroff`, and then exit.

Rationale stopping the program from the serial console during development must also leave the hardware released and the scores saved.

Parent UR-19 *Verification* Test (Ctrl+C on the serial console during play)

3.2.2 SR-UI — Screens and navigation

SR-UI-01 The main menu shall list every game in TBL-GameRegistry followed by SCORES; Up and Down shall move the selection, wrapping at both ends.

Rationale one consistent selection model for any number of games.
Parent UR-02 *Verification* Test

SR-UI-02 The main menu shall show an animated preview of the selected game in its preview panel.

Rationale lets a first-time player recognise each game before starting it (QA-06).
Parent UR-02 *Verification* Inspection (against 3.1.1 screen 2)

SR-UI-03 Select on a game entry shall start that game within 1 tick.

Rationale immediate response to the start key.
Parent UR-02 *Verification* Test

SR-UI-04 During play, the HUD shall show the game name, the score as 6 digits, the lives (Breakout only) and the game's current high score, updated every tick.

Rationale the player always sees progress; 6 digits cover SCORE_MAX = 999,999.
Parent UR-07 *Verification* Inspection (against 3.1.1 screens 3 and 4)

SR-UI-05 Pause during play shall stop all game updates and show the Paused box over the frozen game picture.

Rationale the game state must not change while paused.
Parent UR-08 *Verification* Test (game state identical before and after a 60 s pause)

SR-UI-06 Pause in the Paused state shall resume the game from exactly the state in which it was paused.

Rationale completes the pause cycle without loss.
Parent UR-08 *Verification* Test

SR-UI-07 Back in the Playing or Paused state shall return to the main menu without saving the score.

Rationale quitting is the player's choice to abandon the game.
Parent UR-09 *Verification* Test

SR-UI-08 When a game ends, the system shall go to Initials entry if the score qualifies for that game's top 10, and to Game Over otherwise.

Rationale decision D4; a qualifying player is asked for initials straight away.
Parent UR-10 *Verification* Test (one qualifying and one non-qualifying score)

SR-UI-09 The Game Over screen shall show the final score; Select shall restart the same game and Back shall return to the main menu.

Rationale offers both choices required by UR-10.
Parent UR-10 *Verification* Test

SR-UI-10 Initials entry shall accept letters A to Z up to three letters, Delete shall remove the last letter, and Select shall be accepted only when three letters have been entered; the entry shall then be saved and the High Scores screen shown with the new entry highlighted.

Rationale initials are exactly three letters (TBL-HighScore); highlighting confirms the save.
Parent UR-11 *Verification* Test

SR-UI-11 On a KeyboardLost event in the Playing or Paused state, the system shall pause the game and show the Keyboard Disconnected screen within 1 tick; when a keyboard is found

again, the game shall stay paused until the player presses Pause.

Rationale the player must never lose a game to a pulled cable (decision D5, SF-03).
Parent UR-14 *Verification* Test (unplug during play)

SR-UI-12 The High Scores screen shall show the rank, initials and score of the top 10 entries for one game; Left and Right shall switch game and Back shall return to the main menu.

Rationale shows the table required by UR-12 one game at a time.
Parent UR-12 *Verification* Inspection (against 3.1.1 screen 8)

SR-UI-13 Every screen shall show a hint bar listing that screen's keys and the current sound state.

Rationale the controls are always visible, so no manual is needed to play (QA-06).
Parent UR-02 *Verification* Inspection (all 11 screens)

SR-UI-14 Holding Back for 3 s (180 ticks, timed from KeyDown using the held-key state) on the main menu shall show the shutdown confirmation; Select shall confirm the shutdown and Back shall cancel it.

Rationale a held key with confirmation prevents an accidental shutdown; timing from KeyDown works because auto-repeat is discarded (SR-INP-07).
Parent UR-19 *Verification* Test (hold 2 s: no box; hold 3 s: box appears)

SR-UI-15 On a KeyboardLost event in the Menu, Game Over, Initials or High Scores state, the system shall remain in that state and show the "no keyboard" notice until a keyboard is found.

Rationale decision D5; outside gameplay there is nothing to pause, so only a notice is needed.
Parent UR-14 *Verification* Test (unplug on the main menu)

3.2.3 SR-GAM — Game rules

Snake

SR-GAM-01 The Snake board shall be 42 × 30 cells, each drawn as 3 × 3 tiles, occupying tile columns 1–126 and rows 3–92, with walls on its border.

Rationale 24-pixel cells are clearly visible at 1024 × 768 and the board fills the play field exactly.
Parent UR-03 *Verification* Inspection (against 3.1.1 screen 3)

SR-GAM-02 A Snake game shall start in the Ready state with a snake of length 4 in the centre of the board heading right; the first arrow key shall start movement.

Rationale the player starts the game when ready.
Parent UR-03 *Verification* Test

SR-GAM-03 The snake shall move one cell every `stepTicks` ticks; `stepTicks` shall start at 6 and decrease by 1 after every 5 food items eaten, to a minimum of 3.

Rationale speed rises from 10 to 20 cells per second as the game progresses.
Parent UR-03 *Verification* Test (unit test of speed against food count)

SR-GAM-04 An arrow key shall set the direction for the snake's next move; a key that reverses the current direction shall be ignored, and at most one direction change shall apply per move.

Rationale prevents the snake from turning into itself through two quick key presses.
Parent UR-05 *Verification* Test (unit test: Up then Left within one move)

SR-GAM-05 Food shall appear at a random cell not occupied by the snake; eating it shall grow the snake by one cell and add 10 points.

Rationale core Snake rule.
Parent UR-03 *Verification* Test

SR-GAM-06 The game shall end when the snake's head moves into a wall or into the snake's body; the final frame shall then be held, without flashing, for 30 ticks.

Rationale core Snake rule; the still frame satisfies SF-01.
Parent UR-03 *Verification* Test

Breakout

SR-GAM-07 The Breakout wall shall be 21 × 6 bricks, each 5 × 2 tiles in a 6 × 2-tile slot, on tile rows 8–19; the paddle shall be 16 × 1 tiles on row 90 and the ball 2 × 2 tiles.

Rationale fills the screen width (columns 1–126) and keeps the ball clearly visible.
Parent UR-04 *Verification* Inspection (against 3.1.1 screen 4)

SR-GAM-08 In the Serve state the ball shall rest on the paddle and move with it; Launch shall start the ball moving upward diagonally.

Rationale the player chooses when each ball starts.
Parent UR-04 *Verification* Test

SR-GAM-09 The paddle shall move one tile per tick while Left or Right is held, and shall stop at the side walls.

Rationale held-key movement independent of the keyboard's auto-repeat rate (SR-INP-07, SR-INP-08).
Parent UR-05 *Verification* Test

SR-GAM-10 The ball shall move one tile diagonally every 2 ticks on level 1 and every tick from level 2 onwards.

Rationale difficulty increases after the first wall is cleared.
Parent UR-04 *Verification* Test

SR-GAM-11 The ball shall bounce off the side walls, the top wall, the paddle and bricks; after a paddle hit its horizontal direction shall be set by the half of the paddle it hit.

Rationale lets the player aim the ball.
Parent UR-04 *Verification* Test (unit test of each bounce case)

SR-GAM-12 A ball hitting a brick shall remove the brick and add points by row: 60 for the top row down to 10 for the bottom row.

Rationale higher bricks are harder to reach and are worth more.
Parent UR-07 *Verification* Test

SR-GAM-13 A Breakout game shall start with 3 lives; when the ball passes below the paddle one life shall be lost and the game shall return to Serve, and when no lives remain the game shall end.

Rationale core Breakout rule.
Parent UR-04 *Verification* Test

SR-GAM-14 When the last brick is removed, the game shall pause for 60 ticks, draw a new full wall and increase the level by 1.

Rationale the game continues with rising difficulty instead of ending.
Parent UR-04 *Verification* Test

Sound events (desirable; all with parent UR-18 and verification by Test)

SR-GAM-15 Moving the menu selection should play the MenuMove effect.

SR-GAM-16 Starting a game should play the MenuSelect effect.

SR-GAM-17 The snake eating food should play the Eat effect.

SR-GAM-18 The snake dying should play the GameOver effect.

SR-GAM-19 The Breakout ball hitting a brick should play the BrickHit effect.

SR-GAM-20 The Breakout ball hitting the paddle should play the PaddleHit effect.

SR-GAM-21 Losing a Breakout life with lives remaining should play the LifeLost effect.

SR-GAM-22 Losing the last Breakout life should play the GameOver effect.

Rationale audible feedback for each event in UR-18; effects are defined once in TBL-SoundEffect.
Parent (SR-GAM-15 to 22) UR-18 *Verification* Test (each event triggers exactly one `playSound()` call with the listed effect)

3.2.4 SR-VGA — VGA timing and display output (VGA Controller)

SR-VGA-01 The VGA Controller shall generate video with a pixel clock of 65.000 MHz ± 0.5 %.

Rationale 65 MHz is the XGA 1024×768 @ 60 Hz pixel clock; the PLL ratio 13/10 from 50 MHz is exact (CON-03).
Parent UR-06 *Verification* Test (measure VGA_CLK with an oscilloscope or frequency counter)

SR-VGA-02 The VGA Controller shall produce each video line as 1024 active pixels, 24 front-porch pixels, 136 HSYNC pixels and 160 back-porch pixels, for 1344 pixel clocks per line.

Rationale XGA horizontal timing (Manual Table 3-14); a monitor loses lock if the line length is wrong.
Parent UR-06 *Verification* Test (QuestaSim testbench counts pixel clocks per region)

SR-VGA-03 The VGA Controller shall produce each frame as 768 active lines, 3 front-porch lines, 6 VSYNC lines and 29 back-porch lines, for 806 lines per frame.

Rationale XGA vertical timing (Manual Table 3-15); 806 lines at 48.36 kHz gives 60.004 Hz.
Parent UR-06 *Verification* Test (QuestaSim testbench counts lines per region)

SR-VGA-04 The VGA Controller shall drive VGA_HS and VGA_VS low during their sync pulses and high at all other times.

Rationale XGA uses active-low (negative) sync on both signals; the manual describes the sync as an active-low pulse (Manual §3.6.6).
Parent UR-06 *Verification* Test (simulation waveform check)

SR-VGA-05 The VGA Controller shall drive VGA_BLANK_N low for every pixel clock outside the 1024 × 768 active region.

Rationale BLANK low forces the DAC outputs to the blanking level during the porches and sync, as the monitor requires (ADV7123 Table 8).
Parent UR-06 *Verification* Test (simulation waveform check)

SR-VGA-06 The VGA Controller shall hold VGA_SYNC_N at logic 0 at all times.

Rationale sync is sent on the separate HSYNC and VSYNC wires; the datasheet says to tie SYNC to logic 0 when no sync on green is needed (ADV7123 Table 8).
Parent UR-06 *Verification* Inspection (RTL review)

SR-VGA-07 During the active region, the VGA Controller shall output on VGA_R[7:0], VGA_G[7:0] and VGA_B[7:0] the 24-bit colour supplied by the Tile Renderer for that pixel.

Rationale the DAC takes 8 bits per channel on this board (Manual §3.6.6); the Tile Renderer owns the palette lookup (SR-REN).
Parent UR-06 *Verification* Test (simulation compares output colour with expected palette value)

SR-VGA-08 The VGA Controller shall signal the start of vertical blanking to the Tile Renderer on the first pixel clock after the last active pixel of line 767.

Rationale the tilemap bank swap and the vsync flag read by the HAL happen at this point, so a new frame is never shown half-updated.
Parent UR-13 *Verification* Test (simulation)

SR-VGA-09 From FPGA configuration until the HAL sets the display-enable bit in the control register, the VGA Controller shall output correct sync timing with every active pixel black (R = G = B = 0).

Rationale the monitor stays locked and shows a defined black screen instead of random tilemap contents while Linux boots.
Parent UR-15 *Verification* Test (power on without starting the game; monitor shows black, no "no signal" message)

SR-VGA-10 The VGA Controller shall provide a 640×480 @ 60 Hz bring-up mode with an 800-pixel × 525-line frame and a pixel clock between 25.0 MHz and 25.2 MHz.

Rationale 640×480 is accepted by almost every VGA monitor, so it is used to bring up the video path before XGA (Manual Tables 3-14, 3-15).
Parent UR-06 *Verification* Test (monitor reports 640×480 @ 60 Hz)

SR-VGA-11 The video mode shall be selected by a synthesis-time parameter of the VGA Controller, with 1024×768 as the default value.

Rationale a fixed PLL per build keeps timing simple; the bring-up mode is a development build only.
Parent UR-06 *Verification* Inspection (RTL review)

3.2.5 SR-INP — USB keyboard input (Input Manager)

SR-INP-01 At start-up, the Input Manager shall select as the keyboard the first `/dev/input/event*` device that reports `EV_KEY` support for KEY_A to KEY_Z, KEY_ENTER, KEY_ESC and the four arrow keys.

Rationale other input devices (for example a mouse) also appear as event devices; these keys are the ones the games need (input event codes).
Parent UR-05 *Verification* Test (start with a keyboard and a mouse connected; the keyboard is chosen)

SR-INP-02 The Input Manager shall open the keyboard device in non-blocking mode (`O_NONBLOCK`).

Rationale the game loop must never wait for a key (baseline: `pollInput()` never blocks).
Parent UR-05 *Verification* Inspection (code review)

SR-INP-03 The Input Manager shall request exclusive access to the keyboard device with the `EVIOCGRAB` ioctl.

Rationale stops keystrokes from also reaching the Linux console, where a key combination such as Ctrl+Alt+Del could reboot the board (linux/input.h).
Parent UR-05 *Verification* Test (press Ctrl+Alt+Del while playing; the board does not reboot)

SR-INP-04 Once per game tick, the Input Manager shall read every queued `input_event` from the keyboard device until `read()` returns `EAGAIN`.

Rationale draining the queue every tick (60 per second) keeps input latency within one tick and loses no events.
Parent UR-05 *Verification* Test (inject 10 events in one tick; all 10 are returned)

SR-INP-05 The Input Manager shall convert each `EV_KEY` event with value 1 into a KeyDown event carrying the key code.

Rationale value 1 means key press (input event codes).
Parent UR-05 *Verification* Test (unit test with recorded events)

SR-INP-06 The Input Manager shall convert each `EV_KEY` event with value 0 into a KeyUp event carrying the key code.

Rationale value 0 means key release; Breakout needs releases to stop the paddle.
Parent UR-05 *Verification* Test (unit test)

SR-INP-07 The Input Manager shall discard `EV_KEY` events with value 2 (auto-repeat).

Rationale baseline decision: held keys are tracked from key-down and key-up events, not from the OS auto-repeat, so speed does not depend on the repeat rate.
Parent UR-05 *Verification* Test (hold a key for 2 s; exactly one KeyDown is produced)

SR-INP-08 The Input Manager shall keep a pressed/released state for every key code, updated by each KeyDown and KeyUp event.

Rationale the Breakout paddle moves 1 tile per tick while its key is held.
Parent UR-05 *Verification* Test (unit test)

SR-INP-09 The Input Manager shall translate each KeyDown event into an action using the TBL-KeyBinding entry for the current context (Menu, Snake, Breakout, Paused, GameOver, Initials, HighScores, ShutdownConfirm) or, if the key has no entry there, the entry for the Global context.

Rationale one mapping table gives every screen its keys (3.1.1) and allows remapping without code changes. Global holds keys that work on every screen, such as M (sound on/off). Checking the current context first lets a screen override a Global key: in Initials entry, M types the letter M (SR-INP-11) instead of muting.
Parent UR-02 *Verification* Test (unit test per context, including M in Snake → Mute and M in Initials → Char 'M')

SR-INP-10 The Input Manager shall discard a KeyDown event whose key code has no binding in either the current context or the Global context.

Rationale unbound keys must have no effect on the game.
Parent UR-05 *Verification* Test (unit test)

SR-INP-11 In the Initials context, the Input Manager shall translate a KeyDown on KEY_A to KEY_Z into a Char action carrying the letter A to Z.

Rationale initials are three letters A–Z (TBL-HighScore constraint).
Parent UR-11 *Verification* Test (unit test for all 26 letters)

SR-INP-12 When `read()` on the keyboard device returns `ENODEV`, the Input Manager shall enter the Disconnected state.

Rationale the kernel's evdev driver returns `ENODEV` once the device has been removed (evdev.c); the stale file descriptor is closed on entry to this state.
Parent UR-14 *Verification* Test (unplug the keyboard during play)

SR-INP-13 On entering the Disconnected state, the Input Manager shall set every key to released.

Rationale a key held at the moment of unplugging would otherwise stay "pressed" (for example, a paddle that keeps moving).
Parent UR-14 *Verification* Test (unplug while holding →; the paddle stops)

SR-INP-14 On entering the Disconnected state, the Input Manager shall report a KeyboardLost event to the Game Engine within the same game tick.

Rationale the Game Engine handles KeyboardLost as in SR-UI-11: during play it pauses the game and shows screen 9 (3.1.1); on other screens nothing changes (decision D5).
Parent UR-14 *Verification* Test

SR-INP-15 In the Disconnected and No-keyboard states, the Input Manager shall select a newly plugged-in keyboard (as in SR-INP-01) within 1 s of it being plugged in.

Rationale the Input Manager watches `/dev/input` with inotify, which reports new device files without polling the directory (inotify(7)).
Parent UR-14 *Verification* Test (unplug, wait 5 s, plug in; input works within 1 s)

SR-INP-16 If no keyboard is found at start-up, the Input Manager shall enter the No-keyboard state.

Rationale start-up continues without a keyboard instead of hanging or exiting; screen 1 tells the player to plug one in.
Parent UR-14 *Verification* Test (boot with no keyboard)

3.2.6 SR-AUD — Sound effects (Tone Generator)

UR-18 is a "should" requirement, so every SR-AUD requirement uses "should". The Tone Generator is a new FPGA block alongside the Tile Renderer and VGA Controller.

SR-AUD-01 The Tone Generator should drive AUD_XCK at 12.288 MHz ± 0.5 %.

Rationale 12.288 MHz is the codec master clock for a 48 kHz sample rate (256 × 48 kHz) (WM8731 datasheet, Table 18); it comes from an FPGA PLL fed by a 50 MHz board clock. A ±0.5 % error only shifts pitch by an amount players cannot hear in a beep.
Parent UR-18 *Verification* Test (measure AUD_XCK with an oscilloscope or frequency counter)

SR-AUD-02 The Tone Generator should drive AUD_BCLK and AUD_DACLRCK so that the codec receives 48,000 stereo frames per second ± 0.5 %.

Rationale in slave mode the codec takes its bit clock and left/right clock from the FPGA (WM8731 datasheet, pin descriptions).
Parent UR-18 *Verification* Test (QuestaSim testbench counts AUD_DACLRCK periods per second of simulated time)

SR-AUD-03 The Tone Generator should send each sample on AUD_DACDAT as a 16-bit two's-complement value in I^2S format, with the same value on the left and right channels.

Rationale matches the codec format set in R7 (I^2S, 16-bit) (WM8731 datasheet, Digital Audio Interfaces); one value on both channels gives mono sound from both speakers.
Parent UR-18 *Verification* Test (testbench decodes the serial stream and compares it with the expected samples)

SR-AUD-04 While a tone is playing, the Tone Generator should output a square wave at the frequency in the tone-frequency register (written by the HAL from TBL-SoundEffect), for any value from 100 Hz to 4,000 Hz, within ± 1 %.

Rationale 100–4,000 Hz covers the beeps the games need; ±1 % is below the pitch difference most listeners notice.
Parent UR-18 *Verification* Test (simulation at 100, 440 and 4,000 Hz; measure the period of the output)

SR-AUD-05 The Tone Generator should stop a tone (output value 0) when the time in the tone-duration register has passed, for any value from 10 ms to 2,000 ms, within ± 1 ms.

Rationale games call `playSound(effectId)`; the HAL looks the effect up in TBL-SoundEffect and writes its frequency and duration to the Tone Generator registers. The hardware then ends the tone by itself, so no software has to stop it and the game loop is never delayed.
Parent UR-18 *Verification* Test (simulation at 10, 100 and 2,000 ms)

SR-AUD-06 When the HAL writes a new tone while another is playing, the Tone Generator should start the new tone within 1 sample period (20.8 µs) and drop the old one.

Rationale one voice only (CON-13); the newest game event is the one the player should hear.
Parent UR-18 *Verification* Test (simulation)

SR-AUD-07 The square-wave amplitude should not exceed ± 8,192 (25 % of 16-bit full scale).

Rationale the line output has fixed gain, so the sample amplitude is the only volume control (3.1.2.4); the cap protects players using headphones (SF-05 in 4.2).

Parent UR-18 *Verification* Inspection (RTL review of the amplitude constant) and Test (simulation checks no sample exceeds ±8,192)

SR-AUD-08 While the mute bit in the audio control register is 1, the Tone Generator should output the value 0.

Rationale the player turns sound off with M (3.1.1); muting in hardware means the games need no mute logic of their own.

Parent UR-18 *Verification* Test (simulation, then on the board with speakers)

SR-AUD-09 From FPGA configuration until the HAL sets the audio-enable bit in the audio control register, the Tone Generator should output the value 0.

Rationale no noise or click is played while Linux boots and the HAL configures the codec over I2C; this mirrors SR-VGA-09 for the display.

Parent UR-18 *Verification* Test (power on with speakers connected and the game not started; no sound)

3.2.7 SR-HAL — Hardware Access Layer

SR-HAL-01 The HAL shall map the lightweight HPS-to-FPGA bridge's 2 MB window at physical base address 0xFF20_0000 into the game process's virtual address space at start-up, using mmap() on /dev/mem.

Rationale the game process needs direct, call-free access to bridge registers and the tilemap region.

Parent UR-01 *Verification* Test

SR-HAL-02 If the bridge mapping fails, the HAL shall return a distinct error code identifying the failure rather than causing the process to hang or crash.

Rationale lab staff must be able to diagnose an unconfigured or misconfigured FPGA without a silent failure.

Parent UR-15 *Verification* Test

SR-HAL-03 The HAL shall write a single tile index to a specified (x, y) cell of the shadow tilemap in process RAM; no bridge access occurs on this call.

Rationale drawing calls must be cheap and frequent; only present() touches the bridge, keeping bridge traffic bounded and bank-swap timing predictable.

Parent UR-13 *Verification* Test

SR-HAL-04 The HAL shall write a rectangular region of up to 128×96 cells to the shadow tilemap in process RAM in a single HAL call; no bridge access occurs on this call.

Rationale large-area redraws must be equally cheap and must not touch the bridge outside present(), for the same reason as SR-HAL-03.

Parent UR-13 *Verification* Test

SR-HAL-05 After a Swap Request has been written, the HAL shall provide a blocking call that polls the Status register and returns as soon as the swap-pending bit clears (hardware has applied the swap), or with a timeout error if it has not cleared within 100 ms (6 frame periods).

Rationale Present() must confirm the requested swap actually completed, but must not block beforehand waiting for a separate vsync pulse — polling only begins after the Swap Request write, so the swap is applied at the very next vblank rather than one frame later. Waiting for vsync first, then requesting the swap, would delay the swap by a full frame and break the 61 ms keypress-to-screen latency budget (PR-02).

Parent UR-13 *Verification* Test

SR-HAL-06 The HAL shall provide a non-blocking read of the Front Bank bit and the Frame Counter register at any time without waiting for vsync.

Rationale the renderer and diagnostics need current frame state without stalling the caller.
Parent UR-13 *Verification* Test

SR-HAL-07 The HAL shall unmap the bridge region and release all associated resources on shutdown(), without clearing the Display Enable bit, so the last presented frame ("SAFE TO SWITCH OFF", SR-FW-13) stays on screen.

Rationale clean teardown is required for repeated start/stop during debugging; leaving display enable set keeps the shutdown message visible until power is removed (SF-07).
Parent UR-17 *Verification* Test

SR-HAL-08 On present(), the HAL shall, in order: (1) copy to the back bank every cell that changed in the shadow tilemap since that bank was last written; (2) request a bank swap by writing the Swap Request register; (3) call the blocking poll of SR-HAL-05 and return once it confirms the swap or times out.

Rationale With double buffering the back bank is two frames old, so copying only the most recent frame's changes would leave stale cells from the frame before that. The copy-then-request-then-poll order (not waiting for vsync beforehand) keeps the swap on the very next vblank, which the 61 ms latency budget (PR-02) depends on.
Parent UR-13 *Verification* Test

SR-HAL-09 Before any bridge access, the HAL shall confirm that the FPGA is configured (via the Linux FPGA manager state) and that the ID/Version register returns the expected value; if either check fails, it shall report "graphics hardware not ready" and exit without accessing the tilemap, pattern or palette regions of the bridge.

Rationale accessing the lightweight bridge while the FPGA fabric is unconfigured can hang or fault the HPS rather than return a clean error, so this must be checked before any other bridge access is attempted. Configuration state is read from the FPGA manager (Linux FPGA Manager).
Parent UR-15 *Verification* Test (boot with no bitstream loaded)

SR-HAL-10 Before the first sound is played, the HAL should set HPS_I2C_CONTROL high and configure the WM8731 codec over `/dev/i2c-0`, then set the audio-enable bit in the Audio Control Register; if the codec does not acknowledge, the HAL should report a codec configuration error rather than hang.

Rationale "should" because sound is a "should" requirement (UR-18); mirrors SR-HAL-09's "report, don't hang" principle for the audio path.
Parent UR-18 *Verification* Test

SR-HAL-11 On playSound(effectId), the HAL should look up effectId in TBL-SoundEffect and play the effect's notes in order: write the first note's frequency (Hz) and duration (ms) to the Tone Frequency and Tone Duration registers, then write each following note in the game tick in which the previous note ends. A new playSound() call replaces any effect still playing.

Rationale games call playSound() with only an effect ID; the HAL is what translates that into register writes, keeping Hz/ms values out of game code. TBL-SoundEffect holds one row per note (effect_id, note_no), every note a whole number of ticks, so tick-by-tick sequencing needs no extra hardware.
Parent UR-18 *Verification* Test

SR-HAL-12 On setMute(on), the HAL should write the mute bit in the Audio Control Register to match the requested state.

Rationale the Arcade API's setMute() must have a concrete effect on the Tone Generator; one bit write is sufficient since the FPGA owns actually silencing output (SR-AUD-08).
Parent UR-18 *Verification* Test

SR-HAL-13 After loading tile patterns and the palette at start-up, the HAL shall set the Display Enable bit (Control Register bit 2).

Rationale pairs with SR-VGA-09 — the VGA Controller outputs black until this bit is set, so the player never sees uninitialised tilemap/pattern/palette contents during boot.
Parent UR-15 *Verification* Test

3.2.8 SR-REN — Tile Renderer

SR-REN-01 The Tile Renderer shall maintain two tilemap banks of 12,288 bytes each (128×96 cells, 1 byte/cell), addressed as bank 0 and bank 1.

Rationale double buffering exists specifically to prevent a torn or half-updated picture.
Parent UR-13 *Verification* Inspection

SR-REN-03 The Tile Renderer shall apply a requested bank swap (Swap Request register) only at the start of vertical blanking, and shall clear the swap-pending status bit once the swap has been applied.

Rationale performing the swap in hardware, gated to vertical blanking, removes the software-timing race that could otherwise cause a swap to land mid-frame and tear the displayed image.
Parent UR-13 *Verification* Test

SR-REN-04 The Tile Renderer shall store up to 256 tile patterns of 8×8 pixels at 4 bits per pixel in writable RAM, loaded at start-up rather than fixed at synthesis time.

Rationale a developer must be able to add a new game's graphics without resynthesising the FPGA bitstream.
Parent UR-16 *Verification* Inspection

SR-REN-05 The Tile Renderer shall maintain a 16-entry colour palette, with each pixel in a tile pattern selecting one entry by a 4-bit index.

Rationale fixes a small, FPGA-resource-efficient colour model for all games.
Parent UR-06 *Verification* Inspection

SR-REN-06 The Tile Renderer shall output one pixel per 65 MHz pixel-clock cycle, with a fixed pipeline latency of N = 4 cycles from tilemap read to colour output, and shall delay HSYNC/VSYNC by the same 4 cycles to keep sync aligned with pixel data.

Rationale the renderer is pipelined, not combinational, so a one-cycle fetch is not physically realistic. N = 4: tilemap read → pattern read → palette read → colour output, one pipeline stage per cycle, all four stages busy every cycle on different pixels.
Parent UR-06 *Verification* Analysis (testbench waveform)

SR-REN-07 Setting Control Register bit 1 (reset) shall clear both tilemap banks to tile index 0 within one frame period (≤ 16.7 ms).

Rationale a defined, fully-cleared state on both banks is required so the system never displays stale or undefined tile data; clear() in the Arcade API is a software-side shadow-tilemap operation and is covered separately under SR-HAL.
Parent UR-13 *Verification* Test

SR-REN-08 In 640×480 bring-up mode, the Tile Renderer shall display only tilemap columns 0–79 and rows 0–59 (80×60 visible tiles); the remainder of each tilemap bank shall still be written and held by the renderer but not displayed.

Rationale 640×480 is a smaller visible area than the 128×96 tilemap: this mode exists only to bring up the video path before switching to the full 1024×768 mode, so cropping rather than rescaling the tilemap keeps the renderer simple.
Parent UR-06 *Verification* Inspection

3.3 Behaviour Requirements

3.3.1 Use Case View

Figure 3.12 shows the use case diagram for the whole system. The **Player** is the primary actor. The **Game Developer** uses the system through the SDL2 backend and the Arcade API. **Lab Staff** set up and shut down the board and read fault reports. The **USB Keyboard** and **VGA Monitor** are secondary actors that the system reads from and writes to.

Table 3.4: Use cases

ID	Use case	Actors	Description	URs
UC-01	Start System	Player, Lab Staff	Power on; the system boots, checks the FPGA design and shows the main menu, or reports “graphics hardware not ready”.	UR-01, UR-15
UC-02	Select Game	Player	Move through the menu and start a game.	UR-02
UC-03	Play Snake	Player	Steer the snake to eat food without hitting a wall or itself.	UR-03, UR-05, UR-07
UC-04	Play Breakout	Player	Move the paddle to keep the ball in play and clear the wall.	UR-04, UR-05, UR-07
UC-05	Pause and Resume	Player	Extends Play Game: freeze and continue the game.	UR-08
UC-06	Quit to Menu	Player	Extends Play Game: abandon the game.	UR-09
UC-07	Enter Initials	Player	Extends Play Game when the score is in the top 10: type three initials.	UR-10, UR-11
UC-08	View High Scores	Player	Browse the top 10 of each game.	UR-12
UC-09	Toggle Sound	Player	Mute or unmute sound effects from any screen except Initials entry.	UR-18
UC-10	Reconnect Keyboard	Player	Plug the keyboard back in after it was unplugged; the game waits paused.	UR-14
UC-11	Shut Down System	Player, Lab Staff	Hold Esc on the menu, confirm, wait for “SAFE TO SWITCH OFF”, switch off.	UR-19
UC-12	Add New Game	Game Developer	Implement the game interface, register the game and rebuild the C++ program only.	UR-16
UC-13	Run Games on PC	Game Developer	Run and debug games in an SDL2 window without the board.	UR-17

UC-03 and UC-04 specialise the abstract use case *Play Game*; UC-05, UC-06 and UC-07 extend Play Game, so they apply to every game, including games added later (UC-12). Play Game exchanges key events with the USB Keyboard and pictures with the VGA Monitor. The detailed flows of these use cases are given by the application state machines and the sequence diagrams below; full use case descriptions belong to the test plan.

3.3.2 State Machines and Interaction Models

The system’s behaviour is specified by the following models, each in its own figure:

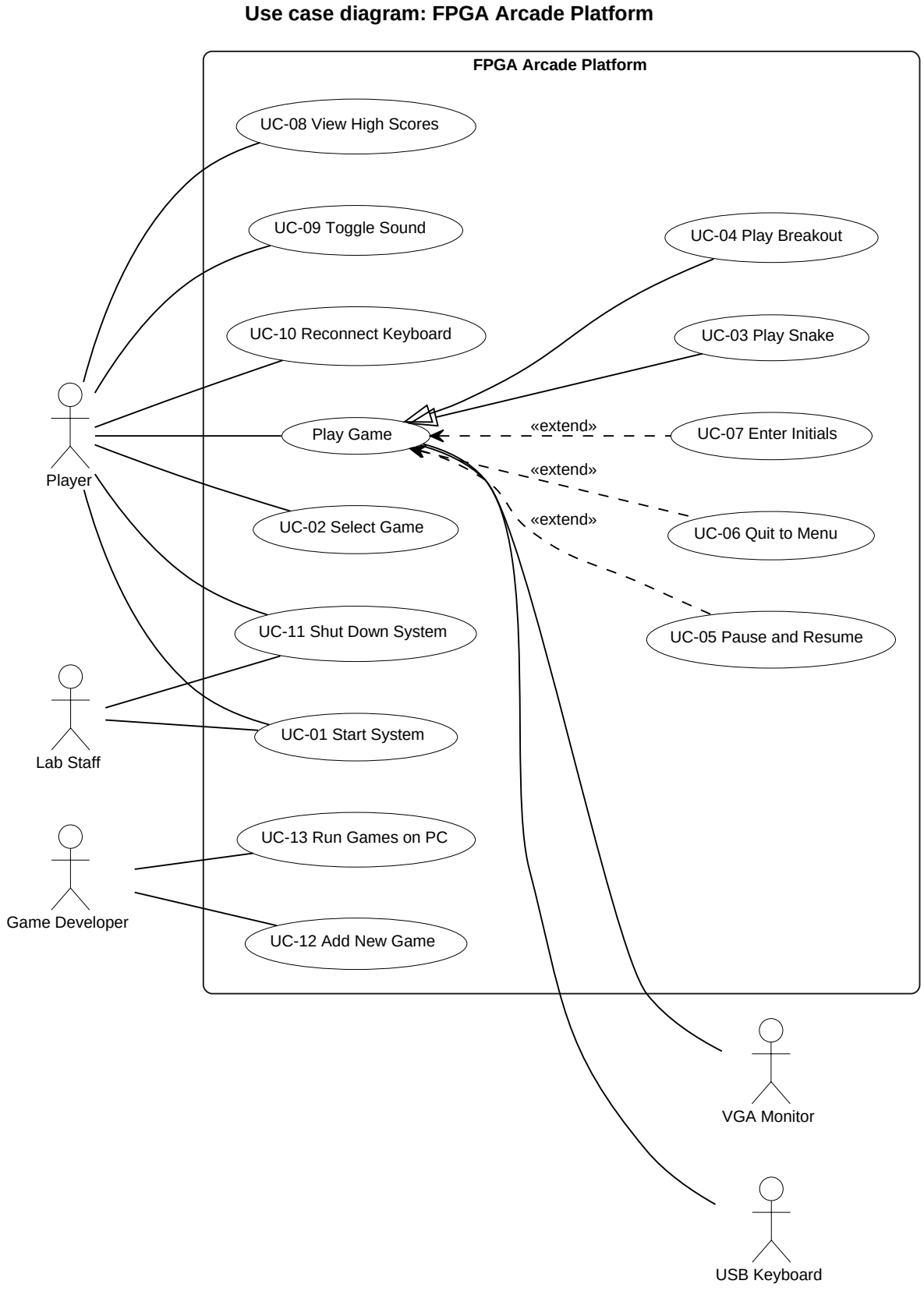


Figure 3.12: Use case diagram

Table 3.5: Behaviour models and their figures

Figure	Model	Owner
3.13, 3.14	Application state machine: start-up, menu and shutdown; gameplay	Muhammad
3.15, 3.16	Game state machines	Muhammad
3.20, 3.17	Input Manager state machine; keypress sequence	Huzaifa
3.18, 3.19	VGA Controller timing state machine	Huzaifa
3.21, 3.22	HAL lifecycle; Tile Renderer	Mujtaba
3.23, 3.24, 3.25	Frame-update sequence	Mujtaba

Application state machine. The Game Engine is always in exactly one state: Boot, HardwareNotReady, Menu, HighScores, ShutdownConfirm, ShuttingDown, Playing, Paused, KeyboardLost, GameOver or Initials. Part 1 covers start-up, the menu and shutdown; part 2 covers gameplay. The transitions implement SR-FW-01, SR-FW-13, SR-UI-01 to SR-UI-15.

Snake and Breakout state machines. Each game has its own internal states below the application's Playing state: Snake moves through Ready, Running, Dead and Over; Breakout through Serve, Running, LevelClear and Over. A game reports the end of play through `isOver()`, which the Game Engine uses to leave Playing (SR-UI-08).

Keypress sequence diagram. Figure 3.17.

The diagram follows one keypress (LEFT in Breakout) from the player to the Game Engine:

- **Path:** USB keyboard → Linux USB HID and evdev → Input Manager → Game Engine. It ends when `pollInput()` returns the KeyDown event and the Action to the Game Engine.
- **Hand-off:** from there the Game Engine runs `update` → `render` → `present`, which is the frame-update sequence diagram (Figures 3.23 to 3.25). How the Game Engine uses each action is in SR-UI and SR-GAM.
- **Timing:** notes give the worst-case time of each stage up to the hand-off. Together with the frame-update path they add up to the latency budget in PR-02.

VGA Controller state machine. Figures 3.18 and 3.19.

Two concurrent regions:

- **Horizontal:** H_ACTIVE (1024 px) → H_FRONT_PORCH (24) → H_SYNC (136) → H_BACK_PORCH (160). Advances every pixel clock.
- **Vertical:** V_ACTIVE (768 lines) → V_FRONT_PORCH (3) → V_SYNC (6) → V_BACK_PORCH (29). Advances at the end of each line.

Each state lists its output levels (HS, VS, BLANK_N). The V_ACTIVE → V_FRONT_PORCH transition raises `vblank_start`, which triggers the tilemap bank swap, the vsync flag and the frame counter (SR-VGA-08).

Input Manager state machine. Figure 3.20.

- **Start-up:** the Input Manager scans for a keyboard, then enters READY or NO_KEYBOARD.

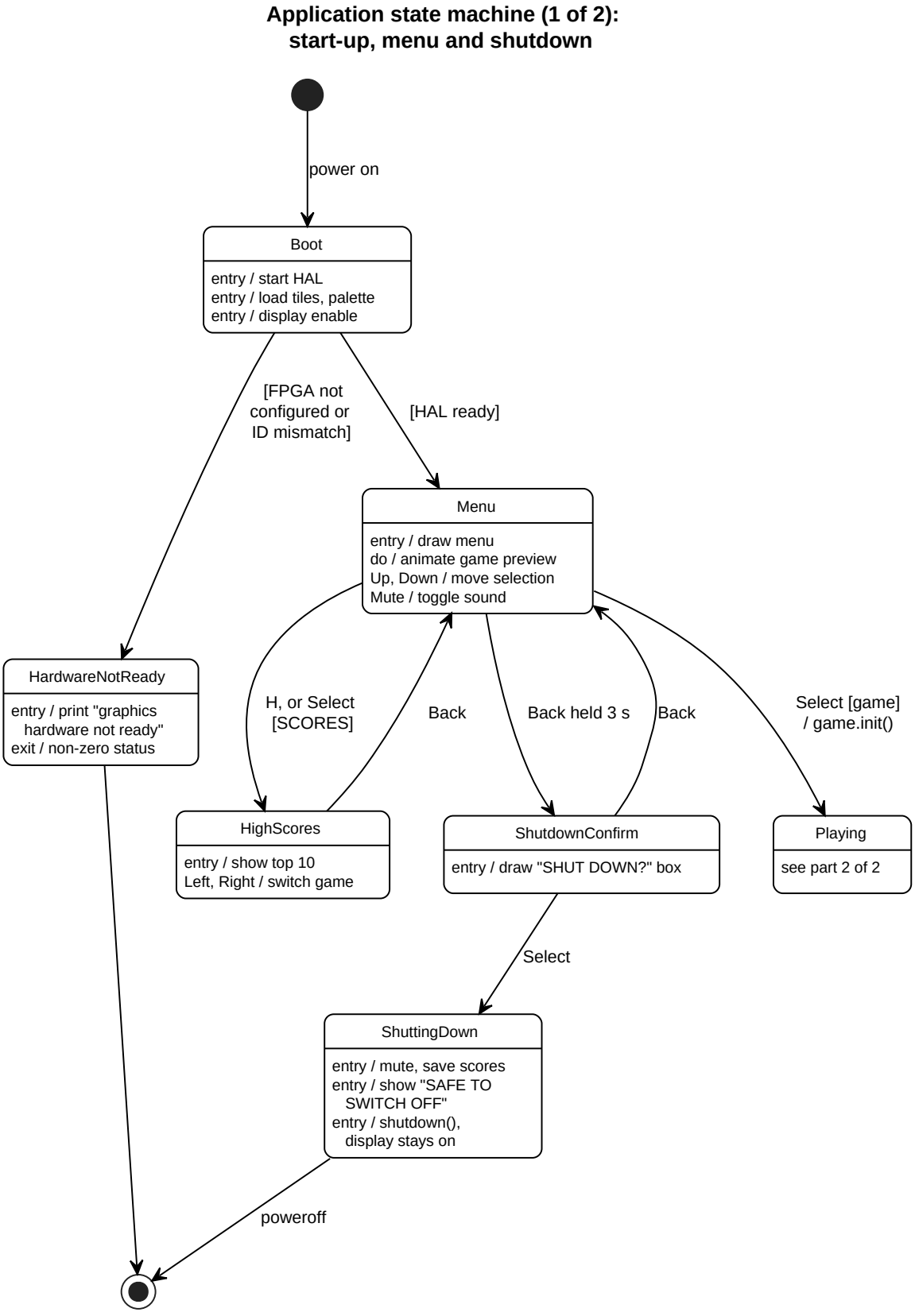


Figure 3.13: Application state machine, part 1: start-up, menu and shutdown

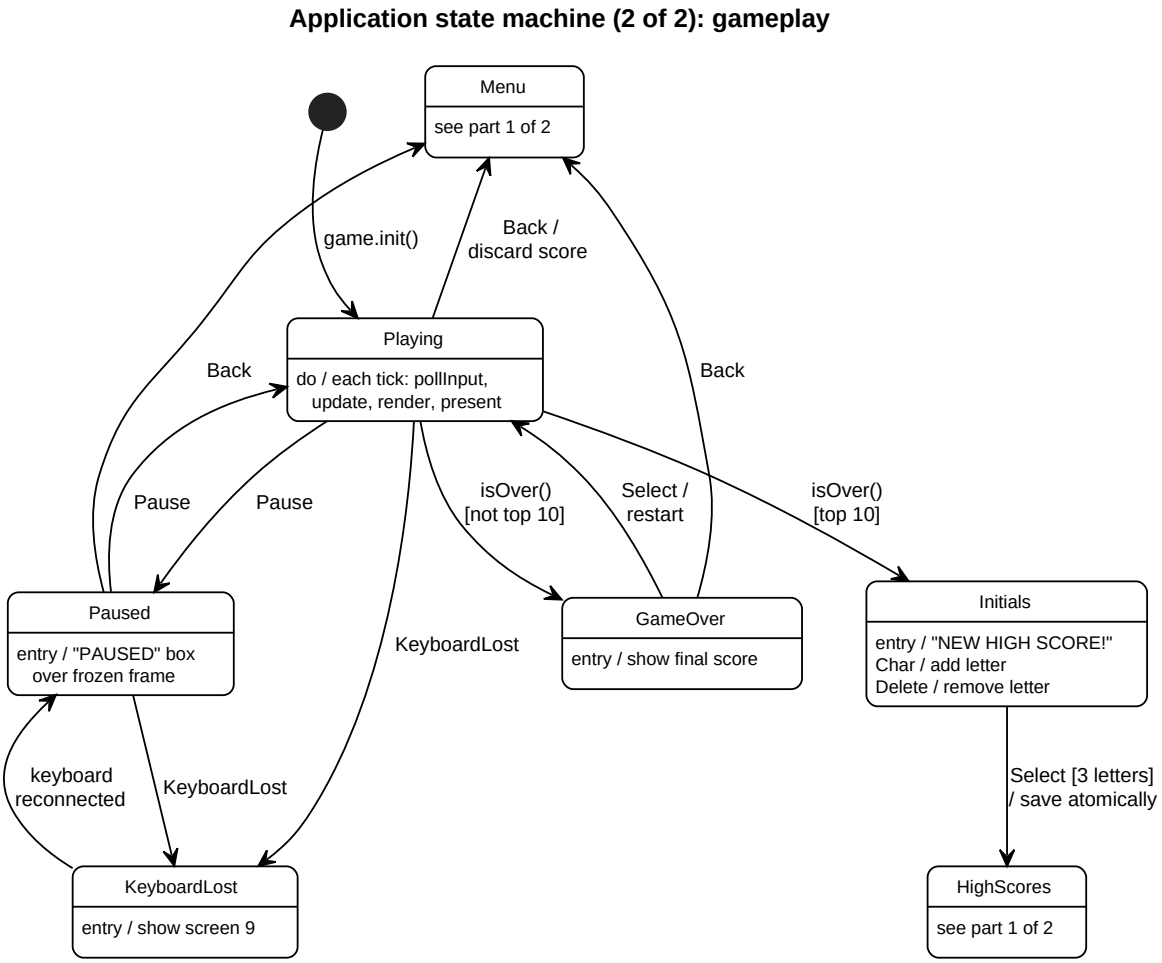


Figure 3.14: Application state machine, part 2: gameplay

- **Unplug:** READY → DISCONNECTED when `read()` returns ENODEV.
- **Reconnect:** NO_KEYBOARD or DISCONNECTED → READY when inotify reports a new keyboard.
- **Hand-off:** on entering DISCONNECTED, the Input Manager reports KeyboardLost to the Game Engine; what the game does with it is SR-UI-11. Per decision D5, the app enters its KeyboardLost state (pause and screen 9) only during play.

The entry and do actions trace to SR-INP-01 to SR-INP-16.

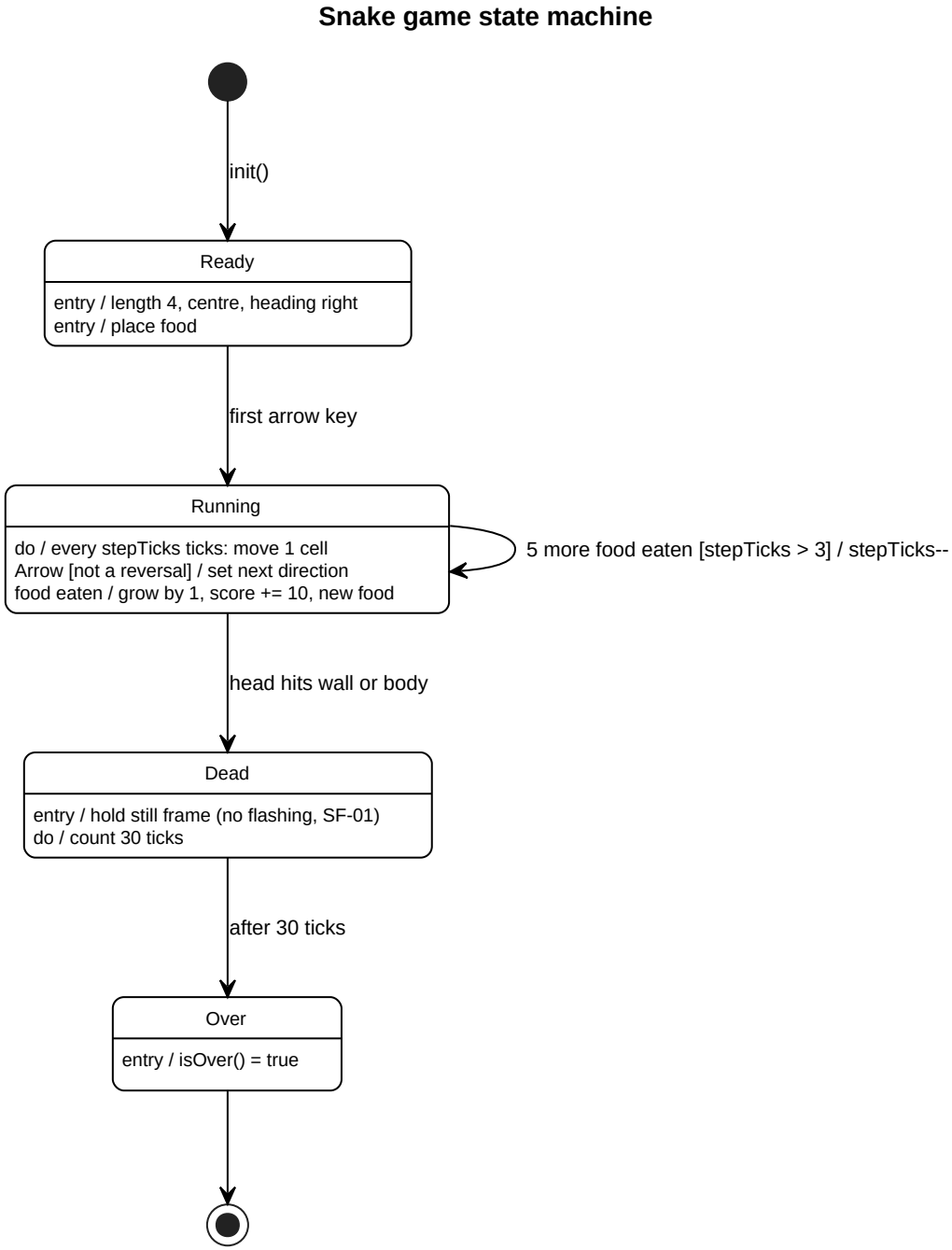


Figure 3.15: Snake state machine

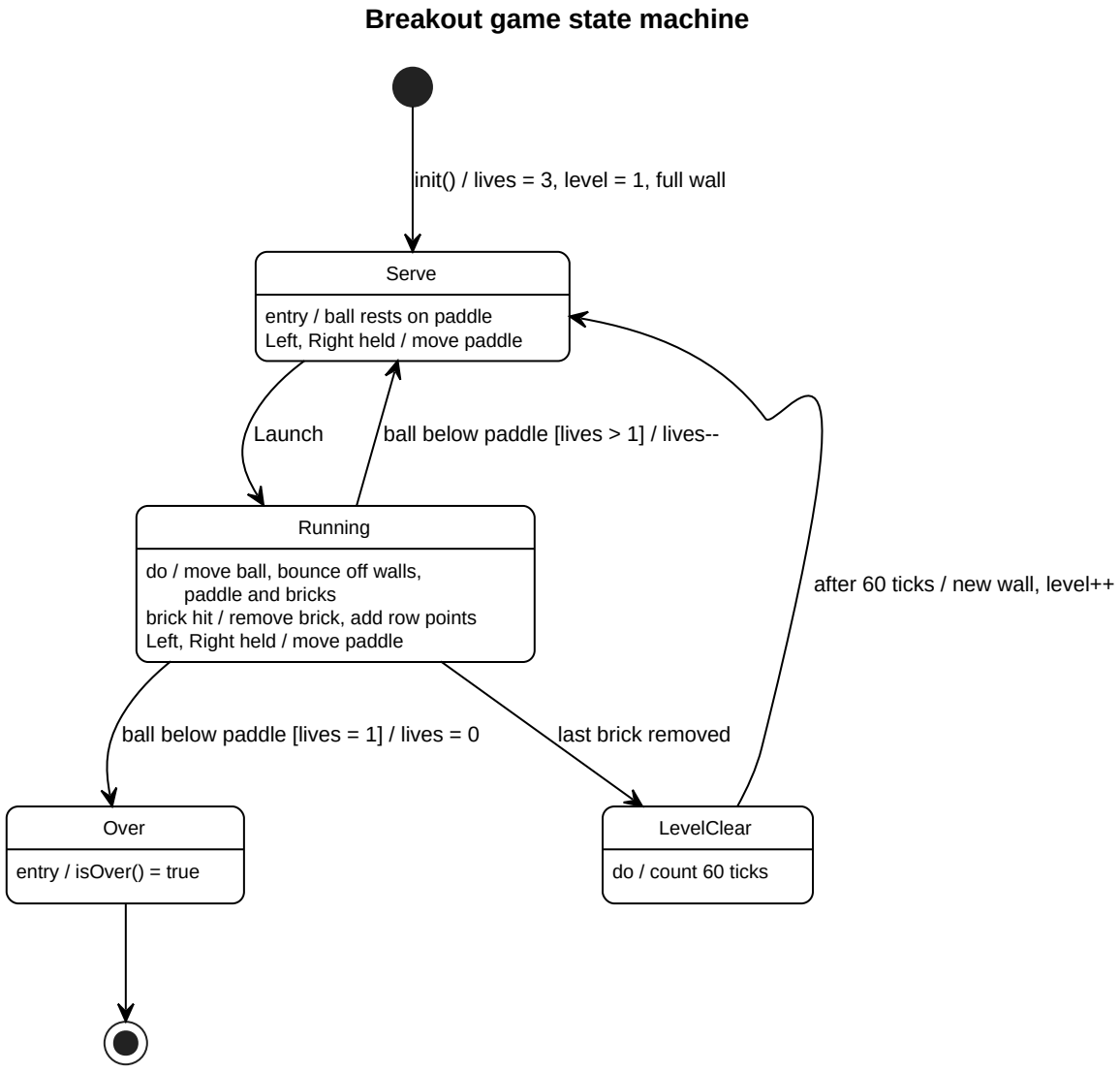


Figure 3.16: Breakout state machine

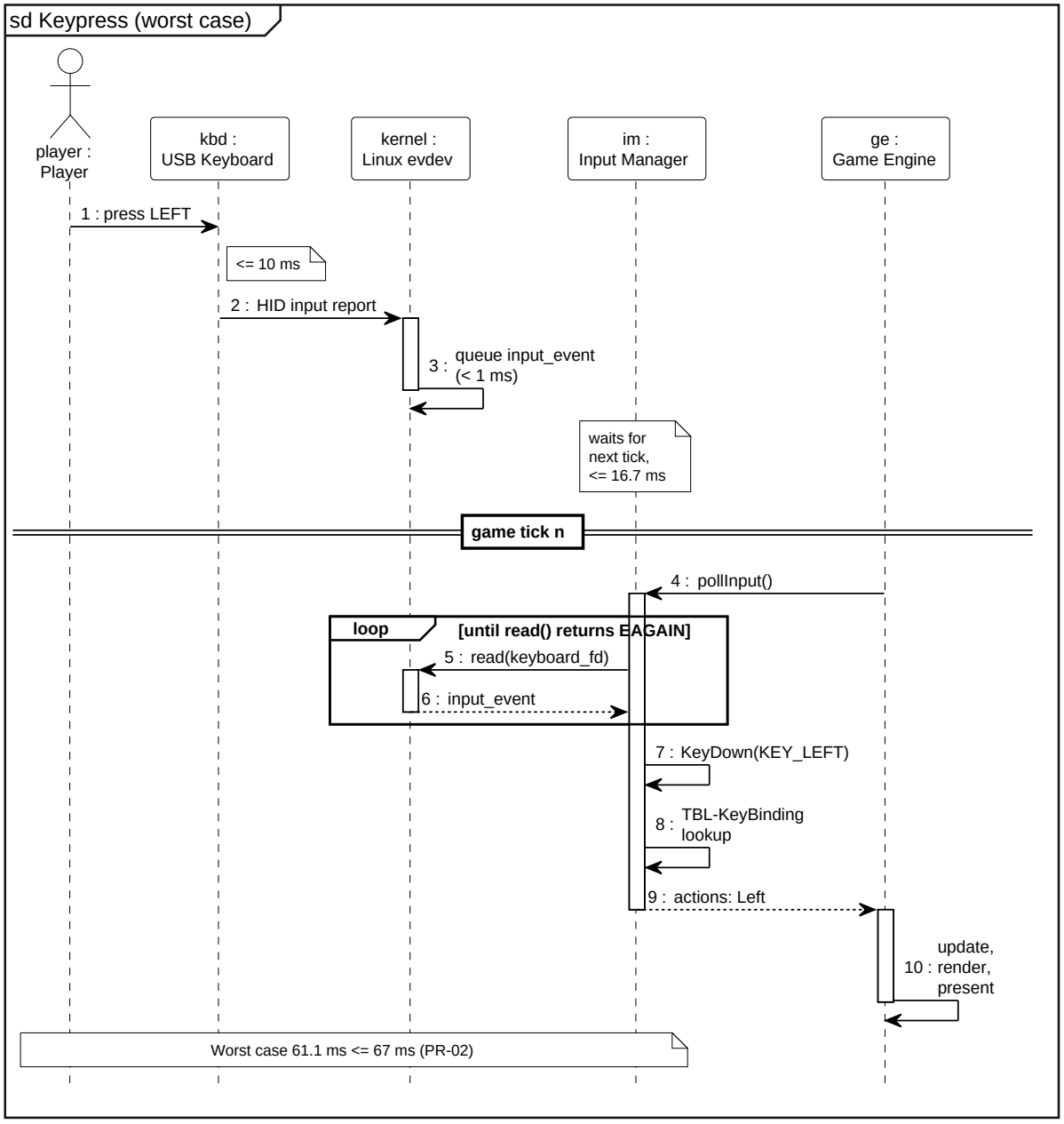


Figure 3.17: Keypress sequence: one key press (LEFT in Breakout) to the Game Engine

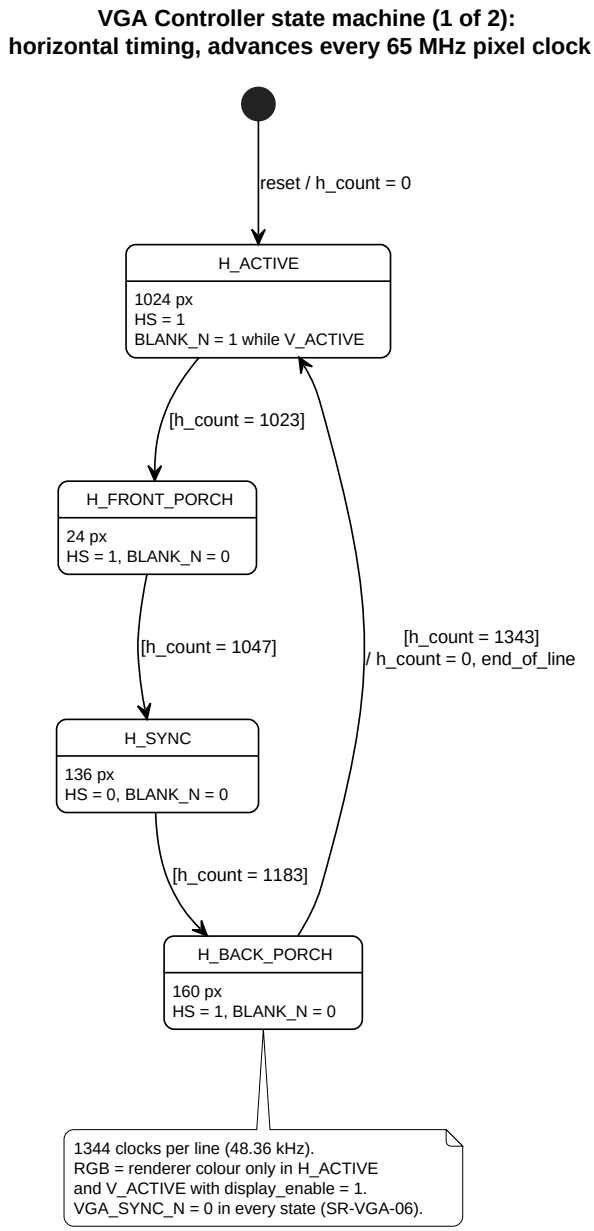


Figure 3.18: VGA Controller timing state machine, part 1: horizontal

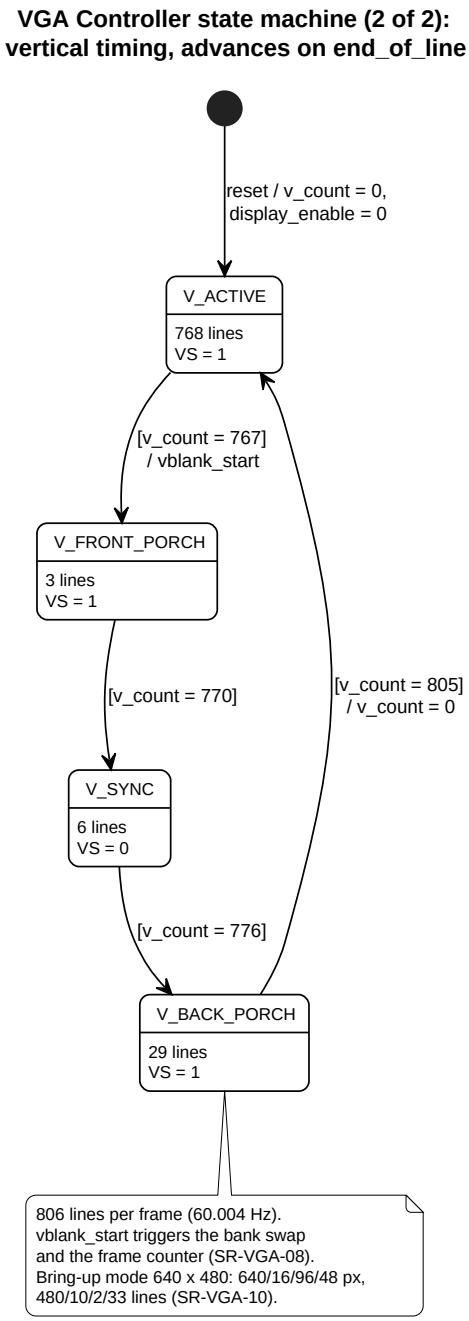


Figure 3.19: VGA Controller timing state machine, part 2: vertical

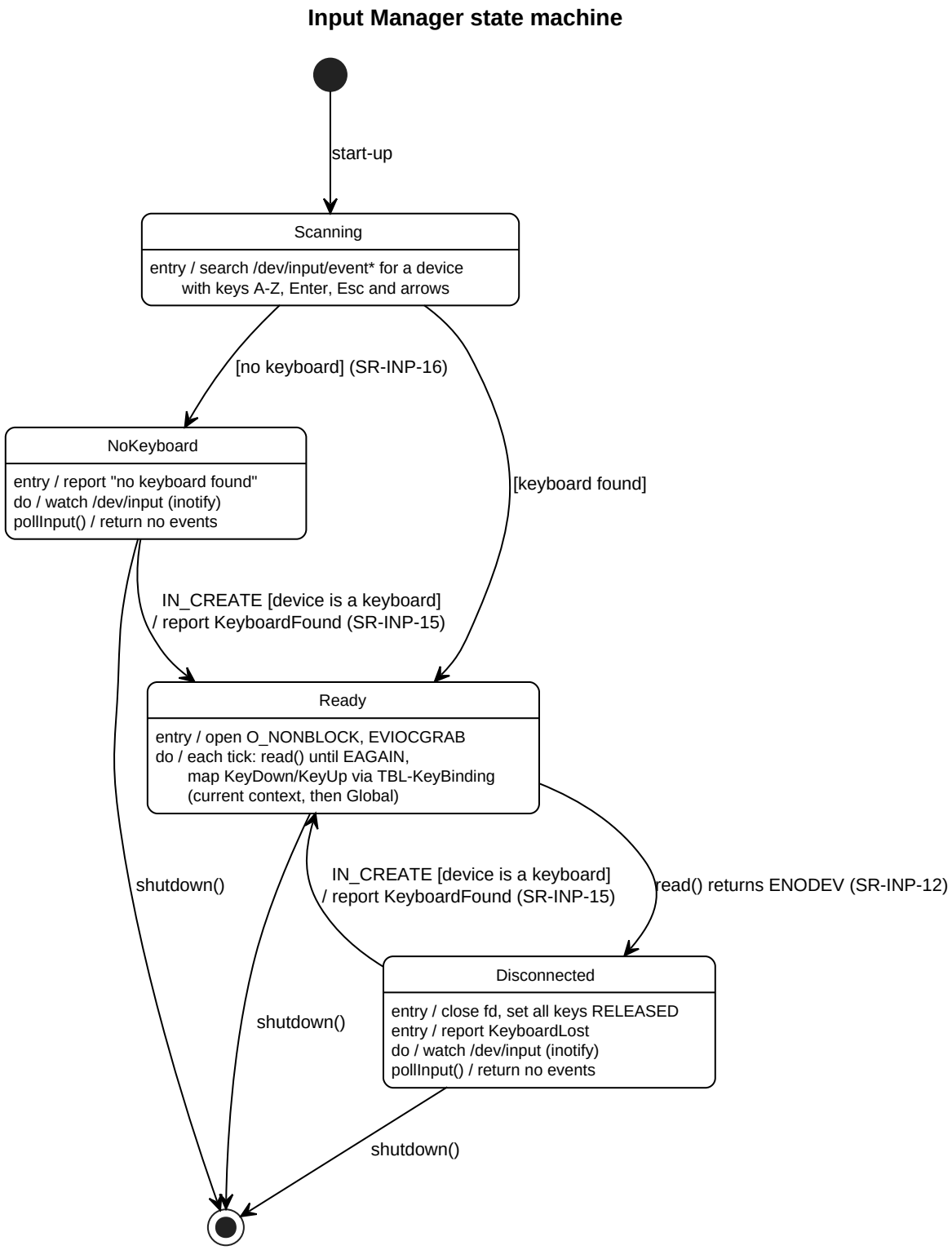


Figure 3.20: Input Manager state machine

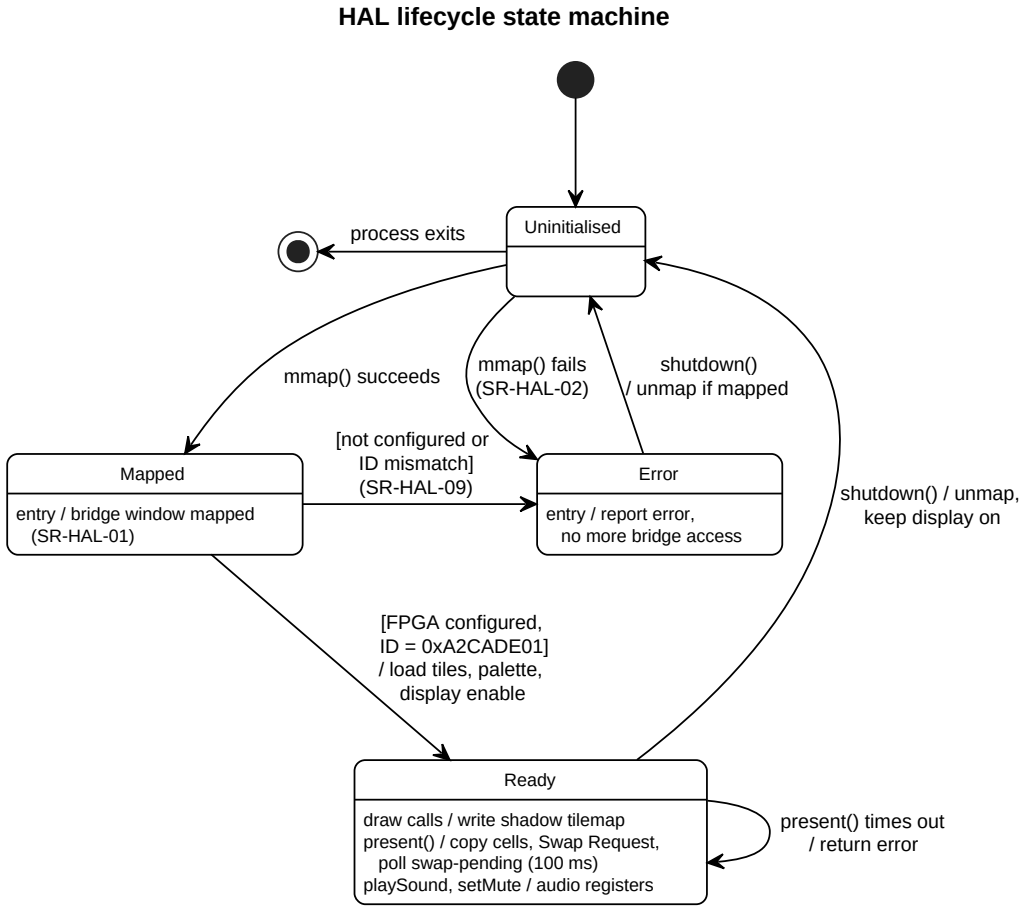


Figure 3.21: HAL lifecycle state machine

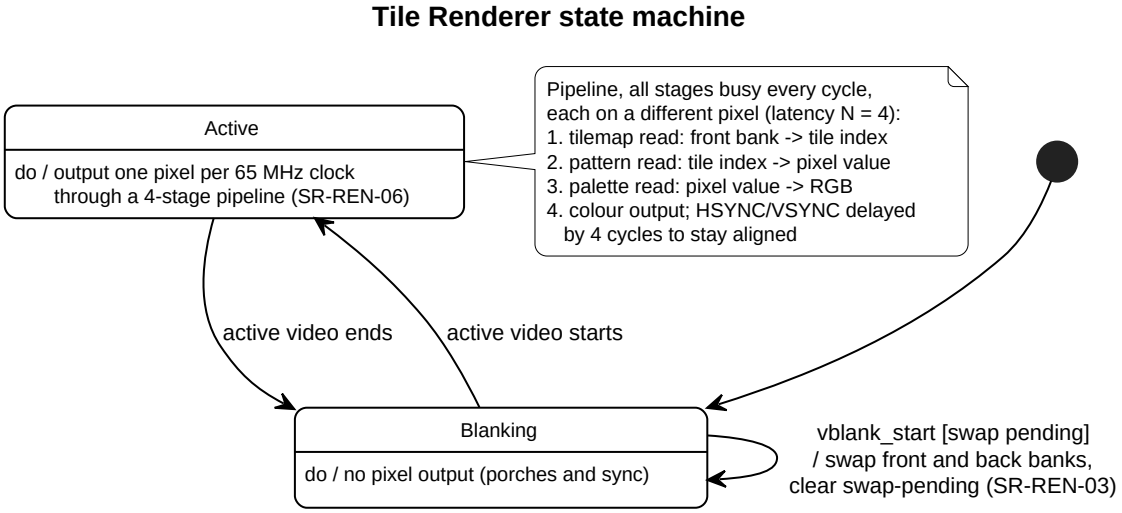


Figure 3.22: Tile Renderer state machine

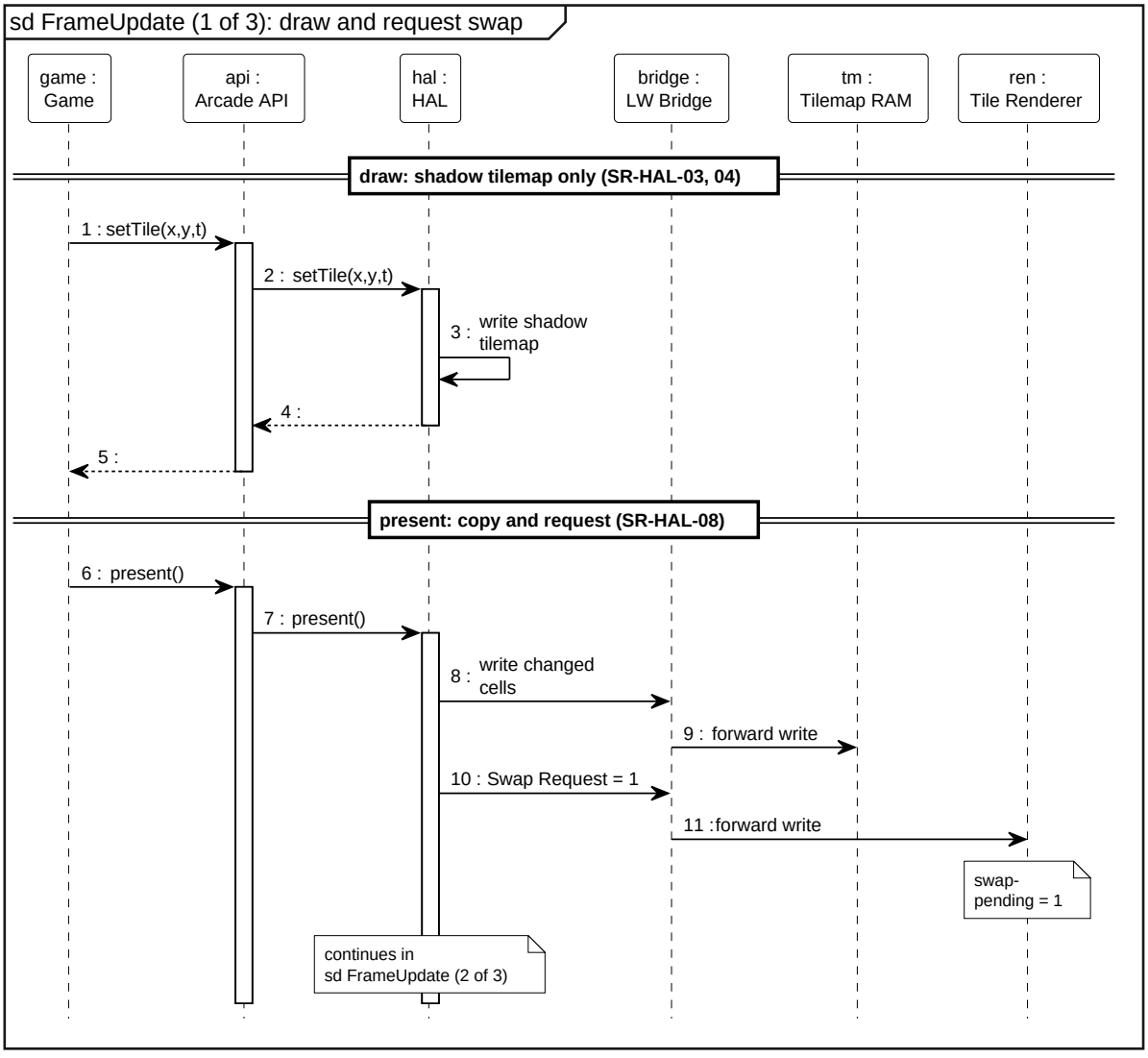


Figure 3.23: Frame-update sequence, part 1: request

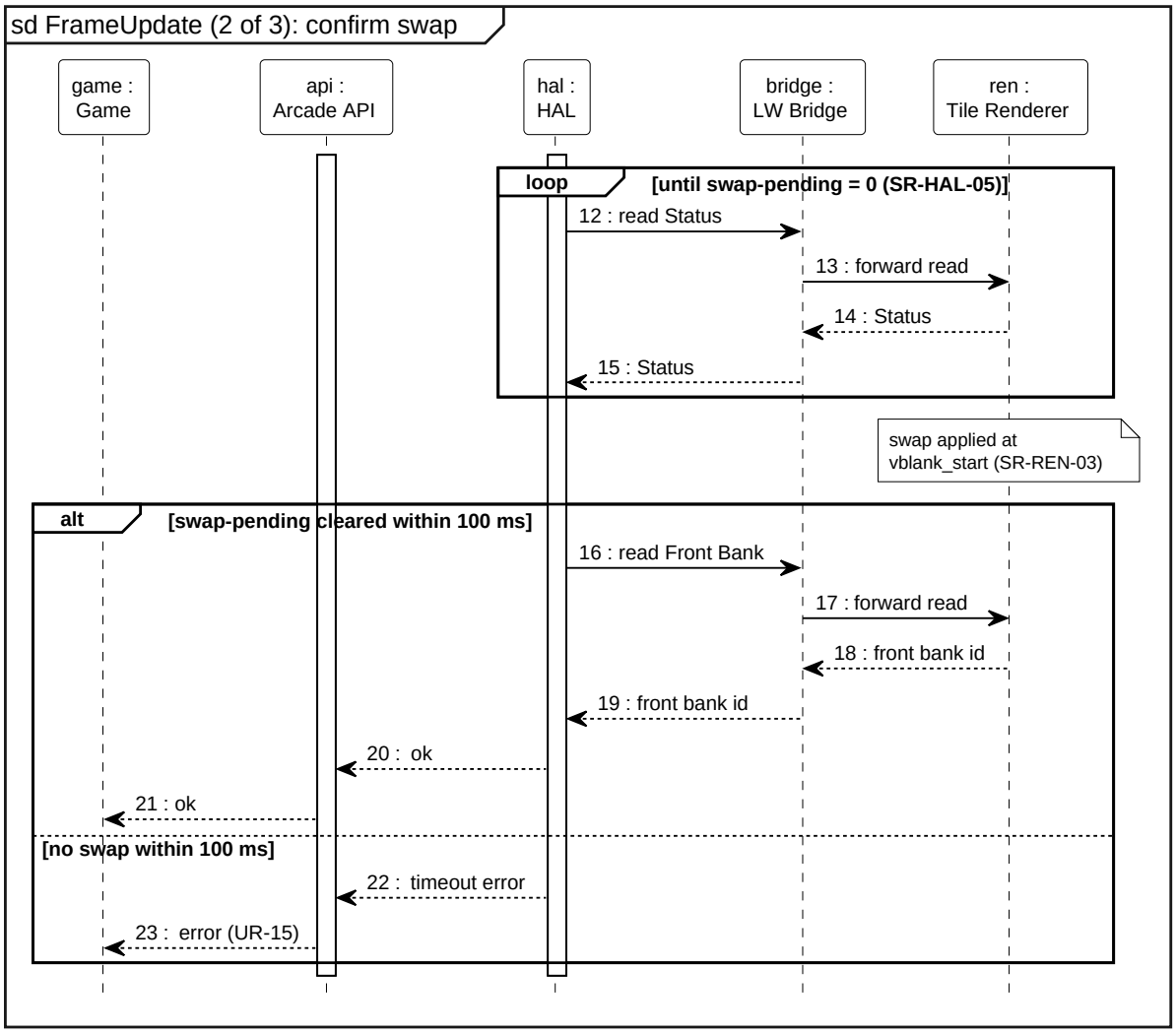


Figure 3.24: Frame-update sequence, part 2: confirm (messages numbered from 12)

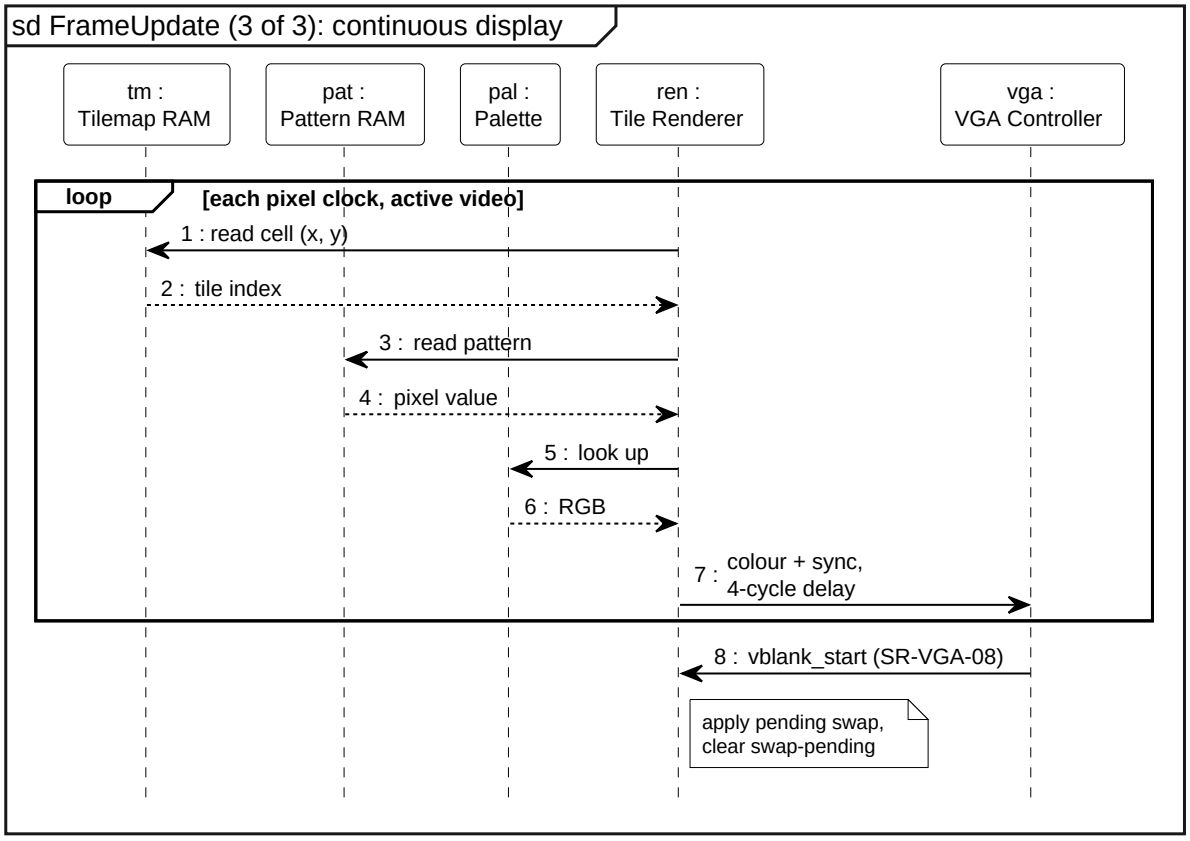


Figure 3.25: Frame-update sequence, part 3: display

4 Other Non-functional Requirements

4.1 Performance Requirements

Latency budget (used by PR-02). The worst case is a key pressed just after the game loop has read input:

Table 4.1: Keypress-to-screen latency budget (PR-02)

Stage	Worst case	Why
USB poll	10 ms	The keyboard reports at its next poll; USB keyboards are typically polled every 8–10 ms
Kernel HID → evdev	< 1 ms	Driver work on an 800 MHz Cortex-A9
Wait for the next game tick	16.7 ms	`pollInput()` runs once per tick, just after vsync (SR-INP-04)
Tick work and bank swap	16.7 ms	`update`, `render` and `present()` (copy changed cells, Swap Request, poll swap-pending); the banks swap at the next vblank (PR-03, PR-04; frame-update sequence diagram)
Scan-out to the changed row	16.7 ms	The new frame is drawn top to bottom; the bottom row appears last
Total	**61.1 ms**	Below 67 ms (4 frames at 60 Hz) with 5.6 ms of margin

PR-01 The VGA Controller shall produce a frame rate of 60 Hz ± 0.5 % (59.7 Hz to 60.3 Hz).

Rationale 65 MHz ÷ (1344 × 806) = 60.004 Hz (SR-VGA-01 to 03), well inside the ±0.5 % that monitors accept for this mode.

Parent UR-06 *Verification* Test (measure VGA_VS with an oscilloscope or frequency counter)

PR-02 The time from a key press to the first displayed frame showing its effect shall be 67 ms (4 frames) or less.

Rationale the latency budget above gives 61.1 ms in the worst case. This turns UR-05's "without noticeable delay" into a number.

Parent UR-05 *Verification* Analysis (latency budget) and Test (film the key and the screen with a 240 frames/s phone camera, 4.2 ms per frame; 20 presses; maximum ≤ 67 ms)

PR-03 The Game Engine shall run exactly one game tick per displayed frame, with no missed frames over 10 minutes of play.

Rationale game speeds are counted in ticks (baseline, SR-FW-02), so a missed frame would slow the game and add 16.7 ms of input latency.

Parent UR-05 *Verification* Test (after 10 minutes of Snake and of Breakout, the tick counter equals the change in `frameCount()`)

PR-04 One game tick of work (`pollInput`, `update`, `render` and copying changed cells to the back bank) shall take 8.3 ms or less in 99.9 % of ticks on the HPS.

Rationale 8.3 ms is half of a 16.7 ms frame. The other half is margin for Linux scheduling delays, so `present()` issues its Swap Request before the next vblank and PR-03 holds. The game loop itself is SR-FW-02.

Parent UR-05 *Verification* Test (time each tick with `clock_gettime(CLOCK_MONOTONIC)` over 10 minutes of each game; report the 99.9th percentile)

PR-05 A tilemap change written before `present()` shall appear on screen in the first frame

scanned out after the bank swap, within 16.7 ms of the swap.

Rationale double-buffered banks swap at vertical blanking (SR-VGA-08), so a frame is never half-updated and is never more than one frame late.

Parent UR-13 *Verification* Test (QuestaSim: write the back bank, swap, check that the next frame's pixels match)

PR-06 The Input Manager shall report every key press and release, with none lost, when keys are pressed at 30 keys per second for 60 s.

Rationale 30 keys/s is above fast typing speed; draining the event queue every tick (SR-INP-04) means at most a few events wait per tick.

Parent UR-05 *Verification* Test (inject 1,800 press/release pairs at 30/s with a Linux `uinput` test program; compare the KeyDown and KeyUp counts with 1,800 each)

PR-07 The system shall show the main menu 60 s or less after power is applied.

Rationale decision D1 sets boot-to-menu at 60 s or less (SR-FW-01). The time is mostly Linux boot from the microSD card and FPGA configuration.

Parent UR-01 *Verification* Test (stopwatch from power switch to menu, 5 power cycles, maximum $\leq$ 60 s)

PR-08 A sound effect should start playing within 16.7 ms (one frame) of the `playSound(effectId)` call made for a sound event (SR-GAM-15 to SR-GAM-22).

Rationale the HAL writes the tone registers straight away and the Tone Generator starts within one sample (SR-AUD-06), so the sound comes at most one frame before the picture that shows the event.

Parent UR-18 *Verification* Test (toggle a test GPIO in `playSound()`; on an oscilloscope, measure from the GPIO edge to the first non-zero sample on line-out)

4.2 Safety and Security Requirements

SF-01 No area of 341 × 256 px or larger shall flash more than 3 times in any 1 s period.

Rationale

- WCAG 2.3.1 limits content to three flashes in any one-second period.
- For a 1024 × 768 screen, it estimates the flash area that matters as a 341 × 256 pixel rectangle anywhere on screen (WCAG 2.1 Understanding 2.3.1).
- This protects players with photosensitive epilepsy. It limits effects such as a flashing "GAME OVER" or a whole-screen flash when a life is lost. The Snake death screen holds a still frame instead of flashing, so it meets the limit by design.

Parent UR-06 *Verification* Inspection (review each game's effects against the limit) and Test (record each effect and count luminance changes per second)

SF-02 If the HAL reports "graphics hardware not ready", the Game Engine shall print that message and the failed step to the serial console and exit with a non-zero status within 5 s.

Rationale UR-15 says the system must report the problem instead of hanging. Until the HAL sets display-enable, the monitor shows a stable black screen (SR-VGA-09), so the player never sees random video.

Parent UR-15 *Verification* Test (start the program without loading the FPGA bitstream; check the message and exit status)

SF-03 Unplugging and reconnecting the keyboard 20 times, during play and on other screens, shall cause no crash, no hang and no loss of the current game or score.

Rationale covers the whole unplug path together (SR-INP-12 to 16 and SR-UI-11). Per decision D5, losing the keyboard pauses the game only during play; on other screens the screen simply stays. A pulled cable is the most likely fault during a lab demo.

Parent UR-14 *Verification* Test (20 unplug/replug cycles in each game and 5 on the main menu; the game resumes with the same score after P, and the menu stays usable)

SF-04 After a power loss at any moment, the high-score file shall contain either the complete

old table or the complete new table.

Rationale
- Decision D2 and SR-FW-09: the table is a CSV file, written to a temporary file, flushed with `fsync()`, then renamed over the old file.
- `rename()` replaces an existing file atomically, so no process ever finds it missing (rename(2)).
- `fsync()` makes sure the data is on the card before the rename (fsync(2)).

Parent UR-12 *Verification* Test (a test program rewrites the table in a loop; cut power 20 times; after each boot the file loads with 10 valid entries per game)

SF-05 The sound output should not exceed 25 % of the codec's full-scale level.

Rationale per decision D3, sound is on at boot, so the first tone can reach a player's headphones without warning. The line output has fixed gain, so the amplitude cap in the Tone Generator (SR-AUD-07) is the safety measure, and no game can exceed it.

Parent UR-18 *Verification* Test (measure line-out amplitude with an oscilloscope at the highest-amplitude effect; $\leq$ 25 % of full scale)

SF-06 The quick-start card shall tell users to place the board on a non-conductive surface, handle it only by its edges, and connect or disconnect the 12 V supply only with the power switch off.

Rationale the DE1-SoC is a bare board with exposed components and pins, so static discharge or a short on a metal desk can damage it.

Parent UR-01 *Verification* Inspection (review the quick-start card, 2.6)

SF-07 The quick-start card shall tell users to switch off power only after "SAFE TO SWITCH OFF" appears.

Rationale pulling power while Linux is running can corrupt the microSD card, which holds the Linux image, the game and the high scores. The shutdown sequence (SR-UI-14, SR-FW-13/14) saves and powers Linux off, then shows "SAFE TO SWITCH OFF" (3.1.1 screen 10b). The card sits next to the SF-06 handling instructions.

Parent UR-19 *Verification* Inspection (review the quick-start card, 2.6)

4.2.1 Security

Expected level. The arcade runs offline in a teaching lab. It has no network services (CON-09), stores no personal data beyond three-letter initials, and can only be used by someone standing at the board. The realistic threats are:

- a malformed or tampered file on the microSD card (the high-score file or the keymap override);
- someone using the serial console or keyboard to get a Linux shell.

Remote attack is out of scope because no network interface is in use.

Privileges. The program needs privileged access to three device files:

- `/dev/mem`, to map the HPS-to-FPGA bridge (owned by root, group kmem, mode 660 (mem(4)));
- `/dev/input/event*`, for the keyboard;
- `/dev/i2c-0`, for the codec.

The requirements below keep that privilege as small as possible.

SC-01 After mapping the bridge and opening `/dev/i2c-0`, the game program should drop root privileges and run as a dedicated user `arcade` that belongs to the `input` group.

Rationale only start-up needs root. Membership of `input` is still needed to open a reconnected keyboard (SR-INP-15). If the program is attacked through a malformed file, it then runs without root. Running `poweroff` for the shutdown (SR-FW-13/14) also needs root; with SC-01, the program can exit with a "shutdown requested" status and let its root start-up script run `poweroff`.
Parent UR-01 *Verification* Inspection (`ps -o user` shows `arcade` during play)

SC-02 No process on the board shall listen on any TCP or UDP port while the arcade is running.
Rationale matches CON-09. Some DE1-SoC Linux images start network services by default, and these must be disabled.
Parent UR-01 *Verification* Test (`ss -tuln` on the serial console lists no listening sockets)

SC-03 The high-score CSV file (decision D2) shall store only game ID, rank, three-letter initials, score and entry order for each entry.
Rationale TBL-HighScore holds no names, dates or other personal data, so the file reveals nothing about players beyond their chosen initials.
Parent UR-12 *Verification* Inspection (file format against TBL-HighScore)

SC-04 On start-up, the Game Engine shall discard any high-score CSV line that fails validation and load the rest. A valid entry has initials of exactly three letters A–Z, a score from 0 to 999,999 (SCORE_MAX) and a rank from 1 to 10.
Rationale a corrupted or hand-edited file must not crash the program or show invalid text on screen. Loading the table is SR-FW-08; SCORE_MAX also fits the 6-digit score shown in the HUD.
Parent UR-12 *Verification* Test (start with a file containing bad initials, out-of-range scores and binary junk; valid entries load and the program runs)

SC-05 On start-up, the Input Manager shall ignore any line of `/etc/arcade/keymap.conf` that names an unknown context, key code or action, and use the default binding for that key.
Rationale a broken override file must not leave a screen without working keys.
Parent UR-02 *Verification* Test (keymap file with invalid lines; all default keys still work)

SC-06 The Linux image shall have a non-empty root password.
Rationale the serial console and any local login are the remaining ways into the board. Some board images ship with an empty root password.
Parent UR-01 *Verification* Inspection (log in on the serial console; a password is required)

4.3 Software Quality Attributes

The quality attributes below are listed in priority order. Where attributes conflict, correctness of the real-time display and reliability come before extensibility, and extensibility before ease of learning.

4.3.1 Maintainability

QA-01 The software shall be layered as Game → Arcade API → HAL → FPGA registers, and no layer shall call a layer above it or bypass the layer below it.
How achieved games include only the Arcade API header; only the HAL includes the register map; a build check fails if any other module includes it. Each layer has one owner and its own unit tests.
Parent UR-16 *Verification* Inspection (include-dependency report)

4.3.2 Reusability (product line)

QA-02 Adding a new game shall require exactly one new class implementing the game interface and one TBL-GameRegistry row, with zero changes to the Game Engine, the HAL or the FPGA design.

How achieved the frozen game interface (SR-FW-03), the registry-built menu (SR-FW-04) and tile graphics loaded at start-up rather than fixed at synthesis (SR-REN-04). Design for change: the register map, tile size and palette are frozen design decisions recorded in Chapter 5.

Parent UR-16 *Verification* Demonstration (a third, minimal test game added by changing only those two items)

4.3.3 Portability

QA-03 The same game source code shall build and run unchanged on the FPGA backend on the DE1-SoC and on the SDL2 backend on a Linux or Windows PC.

How achieved all hardware access sits behind the Arcade API (SR-FW-05); the game code uses only standard C++17.

Parent UR-17 *Verification* Test (both games built for both backends from one source tree)

4.3.4 Testability

QA-04 Game logic shall be unit-tested on a PC with GoogleTest to at least 80 % line coverage, and every SR-VGA, SR-REN and SR-AUD requirement verified by Test shall have a QuestaSim testbench.

How achieved games are pure state machines driven by `update(input)` with no hardware calls, so tests feed recorded inputs and check state; the SDL2 backend and a mock HAL replace the hardware.

Parent UR-17 *Verification* Inspection (coverage report and testbench list)

4.3.5 Reliability

QA-05 The system shall run 2 hours of continuous play, alternating both games, without a crash, a hang or corruption of the high-score file.

How achieved atomic high-score saves (SR-FW-09), validated loading (SC-04), the HAL's bounded timeouts (SR-HAL-05) and the keyboard-loss handling of SR-INP and SR-UI-11.

Parent UR-12 *Verification* Test (2-hour soak test with a scripted input program)

4.3.6 Usability

QA-06 A first-time player shall be able to start a game within 30 s of seeing the main menu, using only the on-screen hint bar.

How achieved a hint bar on every screen (SR-UI-13), a live game preview (SR-UI-02) and the same Select/Back keys on every screen.

Parent UR-02 *Verification* Test (5 first-time users, timed)

5 Other Requirements

The system has no database engine. Its persistent and configuration data are specified as tables so that each field, key and constraint can be traced to requirements. Section 5 has two parts: the hardware tables held in FPGA memory (TBL-Tilemap, TBL-TilePattern, TBL-Palette, TBL-RegisterMap) and the software tables below. Figure 5.2 shows how the software tables relate.

5.1 Database Tables (Hardware)

Note: these hardware "tables" are a documentation convention for describing fixed-layout FPGA block RAM (M10K) contents, not literal relational tables implemented by a database engine. Software tables (TBL-HighScore, TBL-GameRegistry, TBL-KeyBinding, and TBL-SoundEffect) are documented separately in the software-tables part of Section 5.

5.1.1 TBL-Tilemap

One row per cell, per bank. Backed by the two 12,288-byte regions at offsets `0x0000--0x2FFF` (bank 0) and `0x3000--0x5FFF` (bank 1) in the register map.

Table 5.1: TBL-Tilemap fields

Field	Type	Width	Constraint
bank_id	UINT8	1 bit (0 or 1)	Part of PK; identifies which tilemap bank
cell_x	UINT8	7 bits	Part of PK; 0–127
cell_y	UINT8	7 bits	Part of PK; 0–95
tile_index	UINT8	8 bits	FK → TBL-TilePattern.tile_index; 0–255

- **PK:** (bank_id, cell_x, cell_y)
- **Row count:** 2 banks × 128 × 96 = 24,576 rows (12,288 bytes/bank)
- **Used by:** SR-HAL-03, SR-HAL-04, SR-HAL-08, SR-REN-01, SR-REN-03, SR-REN-07, SR-REN-08

5.1.2 TBL-TilePattern

One row per tile design. Backed by the 8 KB region at offset `0x6000--0x7FFF`.

Table 5.2: TBL-TilePattern fields

Field	Type	Width	Constraint
tile_index	UINT8	8 bits	PK; 0–255
pattern_data	BLOB (4-bit packed)	32 bytes	8×8 pixels, 4 bits/pixel, 2 pixels/byte (low nibble = left pixel); each nibble is a palette_index FK → TBL-Palette.palette_index

- **PK:** tile_index
- **Row count:** 256 (8,192 bytes total)
- **Used by:** SR-REN-04, SR-REN-06

5.1.3 TBL-Palette

One row per colour slot. Backed by the 64-byte region at offset `0x8000`.

Table 5.3: TBL-Palette fields

Field	Type	Width	Constraint
palette_index	UINT8	4 bits	PK; 0–15
rgb	UINT32	32 bits	Packed as `0x00RRGGBB`, 8 bits per channel (matches the ADV7123 DAC's 8 wired bits/channel, CON-06)

- **PK:** palette_index
- **Row count:** 16 (64 bytes total)
- **Used by:** SR-REN-05, SR-REN-06

5.1.4 TBL-RegisterMap

One row per addressable register (the control/status/ID/swap registers only — not the bulk memory regions already covered above). Holds the same rows as the register map in 3.1.4, which is the normative definition; this table models them as data.

Table 5.4: TBL-RegisterMap fields

Field	Type	Width	Constraint
offset	UINT32	32 bits	PK; byte offset within the 2 MB LW bridge window
name	VARCHAR(32)	—	Unique; e.g. "Control Register", "Swap Request / Front Bank"
access	ENUM (R, W, RW)	—	Per-bit access may differ (e.g. Swap Request/Front Bank is W on write, R on read of a different meaning) — noted in description
width_bits	UINT8	8 bits	Register width in bits
reset_value	UINT32	32 bits	Value on power-on/reset
description	VARCHAR(256)	—	What the register does

- **PK:** offset
- **Row count:** 8 (Control, Status, Frame Counter, Swap Request/Front Bank, ID/Version, Tone Frequency, Tone Duration, Audio Control)
- **Used by:** SR-HAL-01, SR-HAL-02, SR-HAL-05, SR-HAL-06, SR-HAL-07, SR-HAL-08, SR-HAL-09, SR-HAL-10, SR-HAL-11, SR-HAL-12, SR-REN-03

Figure 5.1 shows how the hardware tables relate.

5.2 Database Tables (Software)

5.2.1 TBL-GameRegistry

Compiled into the program, read-only. One row per game; the main menu is built from it (SR-FW-04).

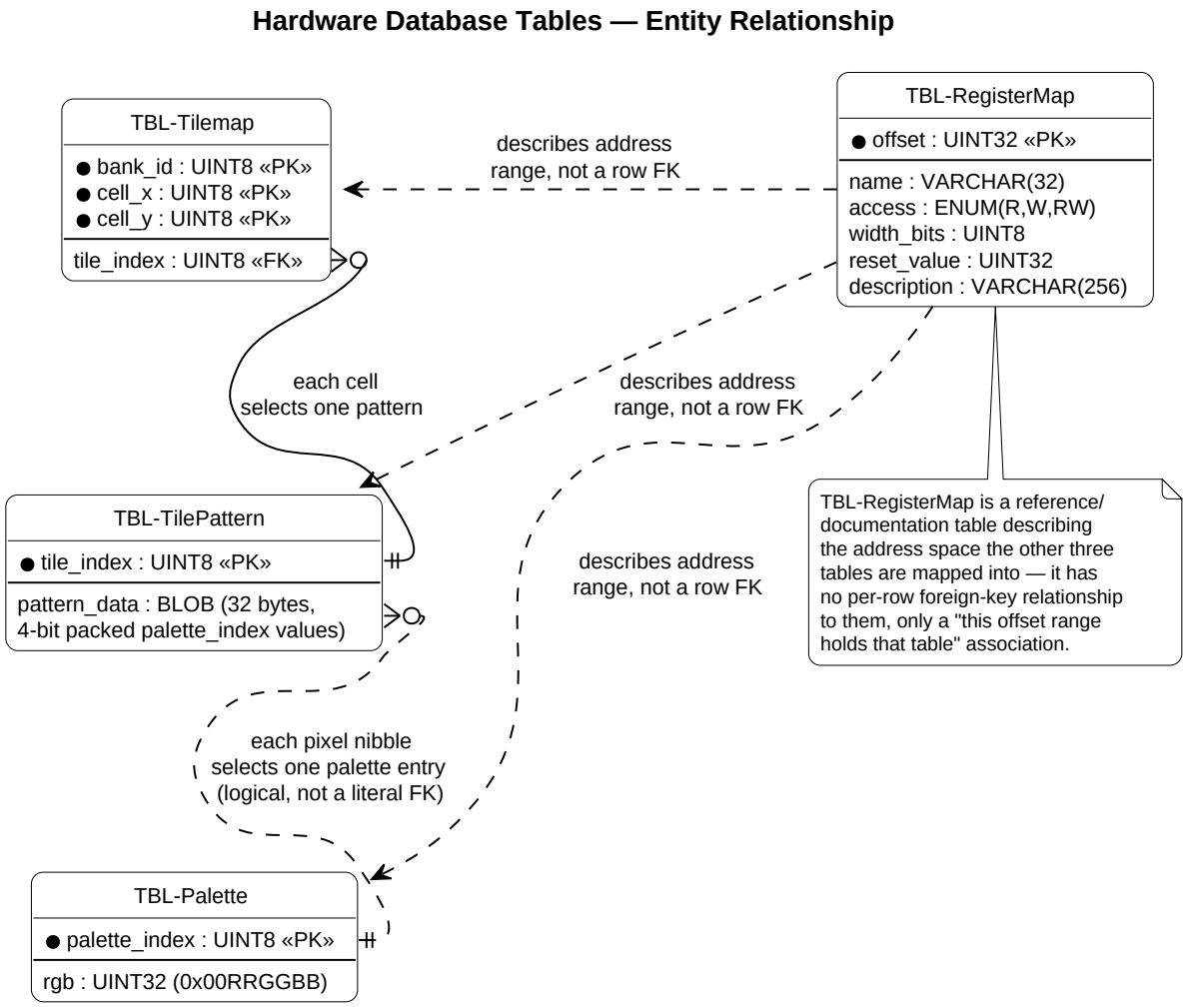
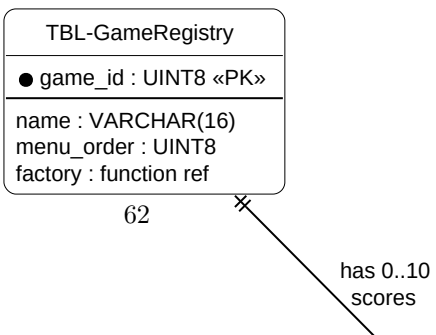


Figure 5.1: Hardware tables entity-relationship diagram

Table 5.5: TBL-GameRegistry fields

Field	Type	Constraint
game_id	UINT8	PK
name	VARCHAR(16)	Unique; shown in the menu and HUD
menu_order	UINT8	Unique; position in the menu
factory	function reference	Creates an object implementing the game interface (SR-FW-03)

- **Rows (release 1.0):** (0, “SNAKE”, 0), (1, “BREAKOUT”, 1)
- **Used by:** SR-FW-03, SR-FW-04, SR-UI-01, QA-02



(continued)

Field	Type	Constraint
score	UINT32	0–999,999 (SCORE_MAX); ranks sorted descending
seq	UINT32	Entry order; breaks ties, older entry ranks higher

- **Rows:** at most 10 per game (20 in release 1.0)
- **No date field:** the board has no battery-backed clock and no network time, so a date would be wrong after every power cycle.
- **Used by:** SR-FW-08 to SR-FW-10, SR-UI-08, SR-UI-10, SR-UI-12, SF-04, SC-03, SC-04

5.2.3 TBL-KeyBinding

Default rows compiled in; an optional override file `/etc/arcade/keymap.conf` may replace individual rows (SC-05).

Table 5.7: TBL-KeyBinding fields

Field	Type	Constraint
context	ENUM (Global, Menu, Snake, Breakout, Paused, GameOver, Initials, HighScores, ShutdownConfirm)	PK (with key_code)
key_code	UINT16	PK (with context); Linux evdev code `KEY_*`
action	ENUM (Up, Down, Left, Right, Select, Back, Pause, Launch, Char, Delete, Mute)	One action per key per context

A key is looked up in the current context first and then in Global, so a screen can override a global key (SR-INP-09).

Default bindings

Table 5.8: TBL-KeyBinding default bindings

Context	Keys → action
Global	M → Mute
Menu	↑ → Up, ↓ → Down, Enter → Select, H → Select (SCORES), Esc → Back (held 3 s: shutdown)
Snake	↑ ↓ ← → → Up, Down, Left, Right; P → Pause; Esc → Back
Breakout	← → → Left, Right; Space → Launch; P → Pause; Esc → Back
Paused	P → Pause; Esc → Back
GameOver	Enter → Select; Esc → Back
Initials	A–Z → Char (overrides Global M); Backspace → Delete; Enter → Select
HighScores	← → → Left, Right; Esc → Back
ShutdownConfirm	Enter → Select; Esc → Back

- **Used by:** SR-INP-09 to SR-INP-11, SR-UI-01 to SR-UI-14, SC-05

5.2.4 TBL-SoundEffect

Compiled into the program, read-only. One row per note; the key is effect_id with note_no.

Table 5.9: TBL-SoundEffect fields

Field	Type	Constraint
effect_id	UINT8	PK (with note_no); 0–6
name	VARCHAR(12)	Same for all notes of one effect
note_no	UINT8	PK (with effect_id); 1–4, play order
freq_hz	UINT16	100–4,000 (SR-AUD-04)
dur_ms	UINT16	10–2,000 (SR-AUD-05); a whole number of game ticks

Contents

Table 5.10: TBL-SoundEffect contents

effect_ id	Effect	Notes in order: pitch, Hz, ms (ticks)	Total
0	Eat	G5 784 Hz, 33 ms (2) → C6 1,047 Hz, 50 ms (3)	83 ms
1	BrickHit	E6 1,319 Hz, 33 ms (2)	33 ms
2	PaddleHit	C5 523 Hz, 33 ms (2)	33 ms
3	LifeLost	G4 392 Hz, 100 ms (6) → E4 330 Hz, 100 ms (6) → C4 262 Hz, 150 ms (9)	350 ms
4	GameOver	C5 523 Hz, 117 ms (7) → G4 392 Hz, 117 ms (7) → E4 330 Hz, 117 ms (7) → C4 262 Hz, 133 ms (8)	484 ms
5	MenuMove	A5 880 Hz, 33 ms (2)	33 ms
6	MenuSelect	E5 659 Hz, 50 ms (3) → C6 1,047 Hz, 83 ms (5)	133 ms

Every note lies within 262–1,319 Hz and every effect lasts 500 ms or less. The HAL plays an effect's notes in order, writing the next note in the tick in which the previous note ends (SR-HAL-11); a new `playSound()` call replaces any effect still playing.

- **Used by:** SR-HAL-11, SR-GAM-15 to SR-GAM-22, SR-AUD-04, SR-AUD-05, PR-08

5.3 Future Scope

Release 1.0 deliberately leaves out: Space Invaders and other sprite-based games (needs a sprite renderer in the FPGA); multiplayer; recorded or mixed audio; and network features. The frozen Arcade API and register map are designed so that a sprite layer can be added later as new registers without changing existing ones.

Appendix A Data Dictionary

Data types. UINT*n*: unsigned integer of n bits (UINT8, UINT10, UINT11, UINT16, UINT32). INT16: signed 16-bit integer, two's complement. INT: signed integer of the C `int` type (32 bits on the HPS). BOOL: a single bit or flag, 0 or 1. ENUM: one value from the listed set. BLOB: a fixed-size block of raw bytes. CHAR(n) / VARCHAR(n): text of exactly / at most n characters. real: a derived quantity that is calculated, not stored.

A.1 Constants

Table A.1: Constants

Name	Type / range	Value	Description	Related IDs
TICK_RATE	—	60 Hz	Game-loop ticks per second, locked to vsync	SR-FW-02
GRID_COLS × GRID_ROWS	—	128 × 96 tiles	Tilemap size	SR-FW-12, UR-06
BOOT_LIMIT / BOOT_TO_MENU_MAX_S	UINT8, s	60 s	Maximum power-on to main menu	SR-FW-01, PR-07
TILE_SIZE	—	8 × 8 px	Tile dimensions	SR-REN-04
TILEMAP_BANK_SIZE	—	12,288 bytes	Size of one tilemap bank (128×96×1 byte)	SR-REN-01, SR-HAL-03/04
TILE_PATTERN_COUNT	—	256	Max distinct tile patterns	SR-REN-04
TILE_PATTERN_RAM_SIZE	—	8,192 bytes	256 tiles × 64 px × 4 bit	SR-REN-04
PALETTE_SIZE	—	16 entries	Colour palette size	SR-REN-05
BRIDGE_BASE	—	0xFF20_0000	LW HPS-to-FPGA bridge physical base address	SR-HAL-01
BRIDGE_WINDOW_SIZE	—	2 MB	LW bridge window size	SR-HAL-01
VSYNC_TIMEOUT	—	100 ms (6 frame periods)	present() swap-pending poll timeout before reporting error	SR-HAL-05
HAL_ID_MAGIC	—	0xA2CADE01	Expected value of the ID/Version register, confirms correct bitstream	SR-HAL-09
PIXEL_CLK_HZ	UINT32, Hz	65,000,000 (± 0.5%)	Pixel clock, operating mode 1024×768	SR-VGA-01, CON-02, CON-03
H_ACTIVE	UINT11, px	1024	Visible pixels per line	SR-VGA-02
H_FRONT_PORCH	UINT11, px	24	Blank pixels, active video → HSYNC	SR-VGA-02
H_SYNC	UINT11, px	136	HSYNC pulse width (active low)	SR-VGA-02, SR-VGA-04
H_BACK_PORCH	UINT11, px	160	Blank pixels, HSYNC → active video	SR-VGA-02
H_TOTAL	UINT11, px	1,344	Pixel clocks per line	SR-VGA-02
V_ACTIVE	UINT10, lines	768	Visible lines per frame	SR-VGA-03
V_FRONT_PORCH	UINT10, lines	3	Blank lines, active video → VSYNC	SR-VGA-03

(continued)

Name	Type / range	Value	Description	Related IDs
V_SYNC	UINT10, lines	6	VSYNC pulse width (active low)	SR-VGA-03, SR-VGA-04
V_BACK_PORCH	UINT10, lines	29	Blank lines, VSYNC → active video	SR-VGA-03
V_TOTAL	UINT10, lines	806	Lines per frame	SR-VGA-03
LINE_RATE_HZ	real, Hz	48,363	PIXEL_CLK_HZ ÷ H_TOTAL	SR-VGA-03
FRAME_RATE_HZ	real, Hz	60.004	LINE_RATE_HZ ÷ V_TOTAL	PR-01
FRAME_PERIOD_MS	real, ms	16.67	1 ÷ FRAME_RATE_HZ; also one game tick	PR-02, PR-03
BRINGUP_PIXEL_CLK_HZ	UINT32, Hz	25,000,000–25,200,000	Pixel clock, bring-up mode 640×480	SR-VGA-10
BRINGUP_H_TOTAL / V_TOTAL	UINT11 / UINT10	800 / 525	Bring-up line and frame length	SR-VGA-10
VIDEO_MODE	ENUM	XGA_1024x768 (default), VGA_640x480	Synthesis-time parameter of the VGA Controller	SR-VGA-11
BRINGUP_VISIBLE_TILES	—	80 × 60 tiles (cols 0–79, rows 0–59)	Tilemap area shown in 640×480 bring-up mode	SR-REN-08
AUD_MCLK_HZ	UINT32, Hz	12,288,000 (± 0.5%)	Codec master clock on AUD_XCK (256 × 48 kHz)	SR-AUD-01
AUD_SAMPLE_RATE_HZ	UINT32, Hz	48,000 (± 0.5%)	Stereo frames per second sent to the codec	SR-AUD-02
AUD_SAMPLE_BITS	UINT8, bits	16	Bits per channel, two's complement, I^2S	SR-AUD-03
TONE_FREQ_MIN / MAX	UINT16, Hz	100 / 4,000	Allowed tone-frequency register range	SR-AUD-04, SR-HAL-11
TONE_DURATION_MIN / MAX	UINT16, ms	10 / 2,000	Allowed tone-duration register range	SR-AUD-05, SR-HAL-11
TONE_AMPLITUDE_MAX	INT16	8,192 (25% of full scale)	Largest square-wave sample magnitude	SR-AUD-07, SF-05
KEY_LATENCY_MAX_MS	real, ms	67 (4 frames)	Worst-case keypress-to-screen time	PR-02
RECONNECT_MAX_MS	UINT16, ms	1,000	Time to select a newly plugged-in keyboard	SR-INP-15
SNAKE_BOARD	—	42 × 30 cells	Snake playfield	SR-GAM-01
SNAKE_CELL	—	3 × 3 tiles	Size of one snake cell on screen	SR-GAM-01
SNAKE_START_LEN	—	4 cells	Initial snake length	SR-GAM-02
SNAKE_STEP_START / _MIN	—	6 / 3 ticks	Ticks per snake move, start and fastest	SR-GAM-03
FOOD_POINTS	—	10	Points per food	SR-GAM-05
SNAKE_DEATH_TICKS	—	30 ticks	Still frame after the snake dies	Snake state machine
BRICK_COLS × BRICK_ROWS	—	21 × 6	Breakout wall	SR-GAM-07
PADDLE_W	—	16 tiles	Paddle width	SR-GAM-07
BALL_SIZE	—	2 × 2 tiles	Ball size	SR-GAM-07
START_LIVES	—	3	Breakout lives at start	SR-GAM-13
ROW_POINTS	—	60, 50, 40, 30, 20, 10	Brick points, top row to bottom row	SR-GAM-12

(continued)

Name	Type / range	Value	Description	Related IDs
LEVEL_CLEAR_TICKS	—	60 ticks	Pause before a new wall	SR-GAM-14
HISCORE_SLOTS	—	10 per game	Size of the high-score table	SR-FW-10
INITIALS_LEN	—	3 letters (A–Z)	Initials length	SR-UI-10
SCORE_MAX	—	999,999 (6 digits)	Highest displayable score	SR-UI-04
SHUTDOWN_HOLD	—	3 s (180 ticks)	Back hold time to open the shutdown confirmation	SR-UI-14
HISCORE_FILE	—	`/var/lib/arcade/highscores.csv`	High-score file path (decision D2)	SR-FW-08, SR-FW-09

A.2 State variables

Table A.2: State variables

Name	Type / range	Possible values or meaning	Related IDs
appState	enum	Boot, HardwareNotReady, Menu, ShutdownConfirm, ShuttingDown, Playing, Paused, KeyboardLost, GameOver, Initials, HighScores	App state machine
menuSelection	0 … (games + 1)	Highlighted menu entry; the last entry is SCORES	SR-UI-01
activeGame	UINT8	game_id of the running game (TBL-GameRegistry)	SR-FW-04
soundOn	bool	true at start-up; toggled by Mute	SR-FW-11
score	0 … 999,999	Current game score	SR-UI-04
shadowTilemap	128×96 byte array, per bank role	Software-side copy of the tilemap; drawing calls write here	SR-HAL-03, SR-HAL-04
swapPending	bool (hardware status bit)	Set on Swap Request, cleared by hardware at vblank	SR-HAL-08, SR-REN-03, Status Register bit 2
frontBank	0 or 1	Which tilemap bank is currently displayed	SR-HAL-06, Swap Request/Front Bank register
h_state	ENUM	ACTIVE, FRONT_PORCH, SYNC, BACK_PORCH Horizontal timing state of the VGA Controller	SR-VGA-02, VGA state machine
h_count	UINT11	0–1343 Pixel clock within the current line	SR-VGA-02
v_state	ENUM	ACTIVE, FRONT_PORCH, SYNC, BACK_PORCH Vertical timing state of the VGA Controller	SR-VGA-03, VGA state machine
v_count	UINT10	0–805 Line within the current frame	SR-VGA-03
display_enable	BOOL	0 (black, reset value), 1 (show tilemap) Set by the HAL after tiles and palette are loaded (SR-HAL-13)	SR-VGA-09
im_state	ENUM	NO_KEYBOARD, SCANNING, READY, DISCONNECTED Input Manager state	SR-INP-12–16, Input Manager state machine

(continued)

Name	Type / range	Possible values or meaning	Related IDs
key_state[key_code]	ENUM per key	RELEASED (default), PRESSED Held-key table, updated by KeyDown/KeyUp	SR-INP-08, SR-INP-13
input_context	ENUM	MENU, SNAKE, BREAKOUT, PAUSED, GAMEOVER, INITIALS, HIGHSCORES, SHUTDOWN_CONFIRM Current screen's key-binding context; GLOBAL is searched second and is never current	SR-INP-09, SR-INP-10, SR-UI-14
keyboard_fd	INT	-1 (none) or ≥ 0 File descriptor of the open evdev keyboard	SR-INP-01, SR-INP-12
audio_enable	BOOL	0 (silent, reset value), 1 Set by the HAL after codec configuration (SR-HAL-10)	SR-AUD-09
mute	BOOL	0 (sound on, default), 1 (sound off) Toggled by the Mute action; lasts until power-off	SR-AUD-08
tone_active	BOOL	0, 1 1 while a tone is playing	SR-AUD-04, SR-AUD-05
tone_remaining_ms	UINT16, ms	0–2,000 Time left in the current tone	SR-AUD-05, SR-AUD-06
snakeState	enum	Ready, Running, Dead, Over	Snake state machine
snakeDir / nextDir	enum	Up, Down, Left, Right	SR-GAM-04
snakeBody	list of cells, length 4 … 1,260	Head first; 1,260 = 42 × 30	SR-GAM-03, SR-GAM-05
foodPos	cell (0–41, 0–29)	Current food cell	SR-GAM-05
stepTicks / tickCount	3 … 6 / 0 … stepTicks − 1	Snake speed and tick counter	SR-GAM-03
breakoutState	enum	Serve, Running, LevelClear, Over	Breakout state machine
paddleX	tile column 1 … 111	Paddle left edge	SR-GAM-09
ballPos / ballVel	tile position / dx, dy $\in \{-1, +1\}$	Ball position and direction	SR-GAM-10, SR-GAM-11
bricks	21 × 6 bit array	true = brick present	SR-GAM-12, SR-GAM-14
lives	0 … 3	Breakout lives left	SR-GAM-13
level	1 …	Breakout level	SR-GAM-10, SR-GAM-14
initialsBuf	0–3 letters	Initials typed so far	SR-UI-10
highScores	TBL-HighScore rows	In-memory copy of the high-score table	SR-FW-08 … SR-FW-10

A.3 Inputs and outputs

Table A.3: Inputs and outputs

Name	Direction	Type	Description	Related IDs
Action events	In	Up, Down, Left, Right, Select, Back, Pause, Launch, Char(A–Z), Delete, Mute	From the Input Manager via `pollInput()`, once per tick	SR-INP-09, SR-FW-02
KeyboardLost / KeyboardFound	In	event	From the Input Manager	SR-INP-14, SR-UI-11
Held-key state	In	per-key bool	Used for paddle movement	SR-INP-08, SR-GAM-09
input_event	In	struct {time, type, code, value}	Raw keyboard event from `/dev/input/eventN`; only EV_KEY used	SR-INP-04–07
read() result	In	errno	EAGAIN (queue empty), ENODEV (keyboard removed)	SR-INP-04, SR-INP-12
inotify event	In	IN_CREATE on `/dev/input`	New input device node appeared	SR-INP-15
Tilemap drawing	Out	`setTile`, `fillRect`, `drawText`, `clear`	To the shadow tilemap only — no bridge access	SR-HAL-03, SR-HAL-04
Frame end	Out	`present()`	Copy changed shadow cells to back bank + request swap	SR-HAL-08, SR-FW-02
Sound	Out	`playSound(effectId)`, `setMute(on)`	To the HAL / Tone Generator	SR-HAL-11 (playSound), SR-HAL-12 (setMute), SR-FW-11
High-score file	Out	CSV file	Atomic save (decision D2: temp file, fsync, rename)	SR-FW-09
FPGA-ready check	In/Out	boolean + magic value	HAL checks FPGA manager state + ID/Version register before any bridge access	SR-HAL-09
tone-frequency / tone-duration registers	Out	UINT16 Hz / UINT16 ms	Written by HAL from TBL-SoundEffect on playSound()	SR-AUD-04, SR-AUD-05, SR-HAL-11
audio control register	Out	bits {audio_enable, mute}	Written by HAL	SR-AUD-08, SR-AUD-09, SR-HAL-10, SR-HAL-12
VGA_R/G/B, VGA_HS/VS, VGA_BLANK_N, VGA_SYNC_N, VGA_CLK	Out	per 3.1.2.1	VGA signals to the ADV7123 DAC / monitor	SR-VGA-01, 04–07
vblank_start	Out	1-bit pulse	First pixel clock after last active pixel of line 767; bank-swap trigger	SR-VGA-08, SR-REN-03
AUD_XCK, AUD_BCLK, AUD_DACLRCK, AUD_DACDAT	Out	per 3.1.2.4	Audio signals to the WM8731 codec	SR-AUD-01–03

Appendix B Traceability Matrix

Each user requirement (UR) is listed with every system requirement (SR) and non-functional requirement (PR, SF, SC, QA) whose parent it is. Every UR has at least one child, and every one of the 134 SR, PR, SF, SC and QA requirements has exactly one parent UR, so there are no orphan requirements and no unrealised user needs. Constraints (CON) and assumptions (AS) limit the design rather than derive from a single UR and are not listed.

Table B.1: Traceability matrix: user requirements to derived requirements

UR	Count	Derived requirements
UR-01	7	SR-FW-01, SR-HAL-01, PR-07, SF-06, SC-01, SC-02, SC-06
UR-02	7	SR-UI-01, SR-UI-02, SR-UI-03, SR-UI-13, SR-INP-09, SC-05, QA-06
UR-03	5	SR-GAM-01, SR-GAM-02, SR-GAM-03, SR-GAM-05, SR-GAM-06
UR-04	6	SR-GAM-07, SR-GAM-08, SR-GAM-10, SR-GAM-11, SR-GAM-13, SR-GAM-14
UR-05	16	SR-FW-02, SR-GAM-04, SR-GAM-09, SR-INP-01, SR-INP-02, SR-INP-03, SR-INP-04, SR-INP-05, SR-INP-06, SR-INP-07, SR-INP-08, SR-INP-10, PR-02, PR-03, PR-04, PR-06
UR-06	15	SR-FW-12, SR-VGA-01, SR-VGA-02, SR-VGA-03, SR-VGA-04, SR-VGA-05, SR-VGA-06, SR-VGA-07, SR-VGA-10, SR-VGA-11, SR-REN-05, SR-REN-06, SR-REN-08, PR-01, SF-01
UR-07	2	SR-UI-04, SR-GAM-12
UR-08	2	SR-UI-05, SR-UI-06
UR-09	1	SR-UI-07
UR-10	2	SR-UI-08, SR-UI-09
UR-11	2	SR-UI-10, SR-INP-11
UR-12	8	SR-FW-08, SR-FW-09, SR-FW-10, SR-UI-12, SF-04, SC-03, SC-04, QA-05
UR-13	10	SR-VGA-08, SR-HAL-03, SR-HAL-04, SR-HAL-05, SR-HAL-06, SR-HAL-08, SR-REN-01, SR-REN-03, SR-REN-07, PR-05
UR-14	8	SR-UI-11, SR-UI-15, SR-INP-12, SR-INP-13, SR-INP-14, SR-INP-15, SR-INP-16, SF-03
UR-15	5	SR-VGA-09, SR-HAL-02, SR-HAL-09, SR-HAL-13, SF-02
UR-16	5	SR-FW-03, SR-FW-04, SR-REN-04, QA-01, QA-02
UR-17	6	SR-FW-05, SR-FW-06, SR-FW-07, SR-HAL-07, QA-03, QA-04
UR-18	23	SR-FW-11, SR-GAM-15, SR-GAM-16, SR-GAM-17, SR-GAM-18, SR-GAM-19, SR-GAM-20, SR-GAM-21, SR-GAM-22, SR-AUD-01, SR-AUD-02, SR-AUD-03, SR-AUD-04, SR-AUD-05, SR-AUD-06, SR-AUD-07, SR-AUD-08, SR-AUD-09, SR-HAL-10, SR-HAL-11, SR-HAL-12, PR-08, SF-05
UR-19	4	SR-FW-13, SR-FW-14, SR-UI-14, SF-07

Coverage summary: 19 of 19 user requirements are realised; 134 derived requirements (107 SR, 8 PR, 7 SF, 6 SC and 6 QA), of which 24 are desirable “should” requirements (the 23 children of UR-18, plus SC-01). Section 3.3 maps the same URs to use cases UC-01 to UC-13.